



Amazon Connect Bootcamp 2021



Amazon Connect

Lab Guide

Table of Contents

Introduction	4
Case Study	5
Hands-on Module 1: Amazon Connect basic setup	6
Create your Amazon Connect instance.....	6
Amazon Connect Queues and Routing Profiles.....	17
Routing Profiles.....	19
Permissions.....	21
Receive the First Call.....	28
Hands-on Module 2: Amazon Connect and Amazon Lex.....	30
Build your Lex bot.....	31
Integrate Your Lex Bot with Amazon Connect.....	42
Testing it all together	51
Hands-on Module 3: Amazon Connect Dynamic Contact Flow.....	52
AWS Lambda role and policy for Amazon DynamoDB access.....	52
Creating and configuring the DynamoDB table.....	53
AWS Lambda Integration.....	54
Giving Amazon Connect permission to invoke your Lambda function.....	63
Employee Authentication Contact Flow	64
Hands-on Module 4: Amazon Lex Enhanced	82
Build your Lambda function	82
Creating and configuring the DynamoDB table.....	86
Reconfigure your Lex bot.....	87
Read booking price in your Amazon Connect Contact Flow	92
Testing it all together	93
Adding Lex Variables to your Contact Flow	93
Hands-on Module 5 – Send SMS Confirmation	93
AWS Lambda role and policy for AWS SNS Publish.....	94
Giving Amazon Connect permission to invoke your Lambda function.....	99
Update Amazon Connect Contact Flow.....	100
Hands-on Module 6 – Integrate CCP into a custom web page.....	107
Amazon Connect Streams	108
Getting Started – Downloading Streams.....	112

Initialization	112
Event Subscription.....	114
Simple – Serverless – Custom CCP	114
Hands-on Module 7 – Trying it all Together	123

Introduction

The purpose of this guide is to provide you with an introduction to Amazon Connect, Amazon Web Services cloud-based virtual contact centre offering. Leveraging the sample case study provided, you are expected to participate in seven (7) hands-on lab modules:

- Module 1 - Amazon Connect basic setup
- Module 2 - Amazon Connect and Amazon Lex
- Module 3 - Amazon Connect Dynamic Contact Flow
- Module 4 – Amazon Lex Enhanced
- Module 5 – Send SMS confirmation
- Module 6 – Integrate CCP into a custom web page
- Module 7 – Tying it all together

Each module will typically be supported by a short introductory 15-minute lecture followed by a session where you will get actual hands-on experience with Amazon Connect.

For this Bootcamp, you will need access to the AWS Console. It is highly recommended that you leverage an existing AWS Account that is not used for production purposes.

Case Study

Our sample case study will focus on Acme Travel, a travel company based in San Diego, California. Acme Travel specializes in supporting corporate events, such as conferences, seminars, and other internal meetings in several cities within the U.S.

Typical Acme Travel services include: hotel accommodations and car rental.

Acme Travel has engaged you to help them launch a new customer service contact center, that will allow client's employees to book their travel accommodations to attend a company sponsored event.

Based upon requirements provided by Acme Travel, you will need to implement the following:

1. The self-service application will need to recognize the employee's phone number and authenticate via their previously provided PIN. If the employee phone number is not recognized, IVR will ask for the Employee ID and PIN.
2. The application must have different queues for authenticated and unknown employees and will prioritize authenticated employees first.
3. Once verified, employee will be greeted by their name
4. Employee will then be able to book a hotel accommodation by way of a Natural Language Understanding (NLU) chat-bot.
5. Employee will then have to provide location, check-in date, and number of nights staying. This booking must then be stored in a database table.
6. If the Employee asks to speak to an agent, the agent has to greet the Employee by their name.
7. All calls must be recorded and encrypted.

Hands-on Module 1: Amazon Connect basic setup

Objective: Create an Amazon Connect Instance, claim a phone number and set up a basic contact flow, with basic queue setup and call recording.

Amazon Connect is a cloud-based contact center solution which makes it easy to set up and manage a customer contact center and provide reliable customer engagement at any scale. You can deploy a contact center in just a few steps, onboard agents from anywhere, and begin to engage with your customers.

More information:

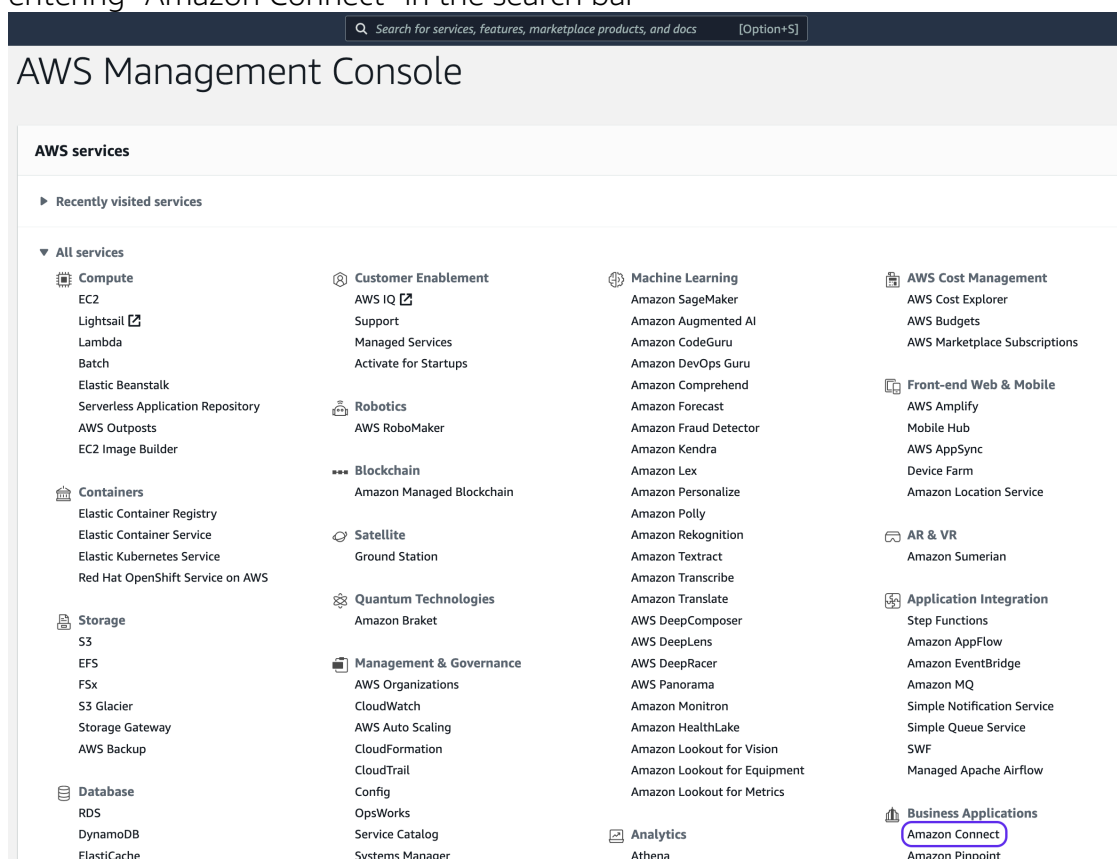
<https://docs.aws.amazon.com/connect/latest/adminguide/what-is-amazon-connect.html>

Create your Amazon Connect instance

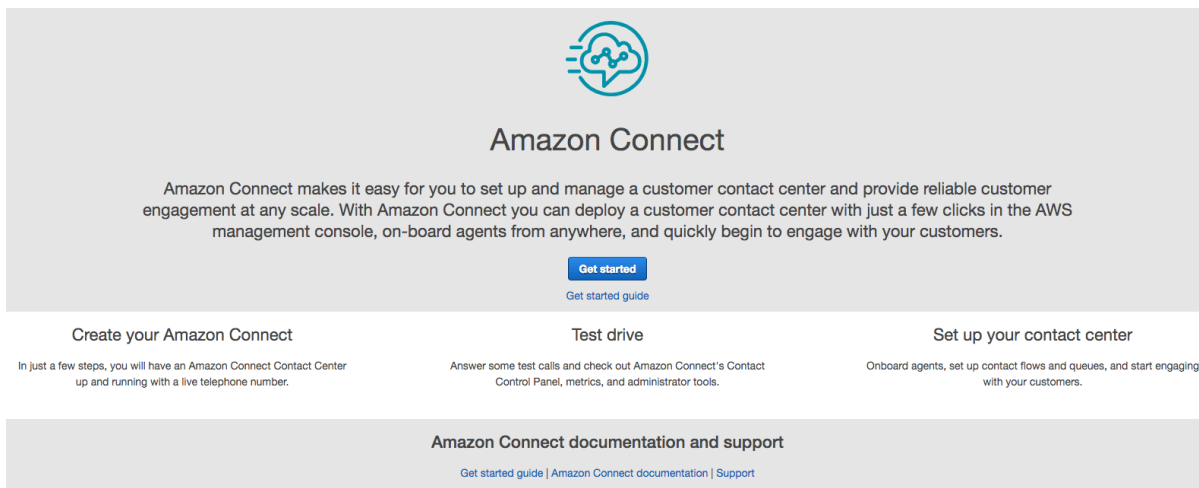
More information on how to start with Amazon Connect can be found in the official documentation:

<https://docs.aws.amazon.com/connect/latest/adminguide/gettingstarted.html>

1. Navigate to AWS console and login using your AWS account
2. Ensure that your selected region is **"US East (N. Virginia)"** by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.
3. Select Amazon Connect either from the Business Applications section, or by entering "Amazon Connect" in the search bar



4. If this is your first time navigating to Amazon Connect, click **Get Started**. Otherwise, you will see a list of instances that have already been built with an **Add an Instance** button. Click either **Add an Instance** or **Get Started**.



5. The first step is to select the option for *Identity Management*. This is where you select to either:
- Provision Simple AD bundled with your Amazon Connect instance
 - Link to an existing directory, whether AWS Cloud Directory or an on-premises directory through the AD Connector.
 - Amazon Connect supports identity federation with Security Assertion Markup Language (SAML) 2.0 to enable web-based single sign-on (SSO) from your organization to your Amazon Connect instance. This allows your users to sign in to a portal in your organization hosted by a **SAML 2.0**-compatible identity provider (IdP). The IdP includes an option to log in to Amazon Connect, which redirects the user to your Amazon Connect instance without having to provide separate credentials for Amazon Connect. More information can be found in the official documentation:
<https://docs.aws.amazon.com/connect/latest/adminguide/configure-saml.html>
6. For the purpose of this training, we will select the first option to store our Amazon Connect users with the instance through Simple AD. Type in a name for your instance and click Next step to continue.

Note: Amazon Connect Instances require globally unique names. Select a unique name else it will be rejected.

Amazon Connect resource configuration

Get up and running in a few easy steps

Step 1: Identity management

Step 2: Administrator

Step 3: Telephony options

Step 4: Data storage

Step 5: Review and create

Identity management

Amazon Connect can be configured to manage your users directly or to leverage an existing directory. This cannot be changed once your instance is created. [Learn more](#)

- ☒ Store users within Amazon Connect
Users will be created and managed by you within Amazon Connect. Note: you will not be able to share users with other applications.
- ☐ Link to an existing directory
Amazon Connect will leverage an existing directory. You create users within the directory and then add and configure them within Amazon Connect. Note: you can only associate a directory with a single Amazon Connect instance. [Learn more](#)
- ☐ SAML 2.0-based authentication
AWS supports identity federation with [SAML 2.0 \(Security Assertion Markup Language 2.0\)](#), an open standard that many identity providers (IdPs) use. This feature enables federated single sign-on (SSO), so users can log into the AWS Management Console or call the AWS APIs without you having to create an IAM user for everyone in your organization. [Learn more](#)

Access URL .my.connect.aws ⓘ

[Cancel](#) [Previous](#) [Next step](#)

7. The next screen asks you to enter a local Administrator for your instance. While you are able to bypass this if you access the instance from the AWS console account used to provision the instance, it is recommended that you create an administrator account at the time of instance creation.

Please note that this will not create an AWS IAM user, but a user in the Amazon Connect related AD.

Passwords have to meet basic complexity criteria: minimum of 8 characters, at least 1 uppercase, 1 lowercase, and 1 number. *For simplicity, enter the same credentials used for the AWS login account.*

Amazon Connect resource configuration

Get up and running in a few easy steps

Step 1: Identity management

Step 2: Administrator

Step 3: Telephony options

Step 4: Data storage

Step 5: Review and create

Create an Administrator

Specify an administrator for this instance of Amazon Connect; this could be you or someone else. You will be able to manage permissions and add more users from within Amazon Connect.

- ☒ Add a new admin

First Name

Last Name

Username

Password

Password (verify)

Email Address

- ☐ Skip this

[Cancel](#) [Previous](#) [Next step](#)

8. Next, we have the option of enabling or disabling the ability to accept incoming and outbound calls. Please note that you can enable both at this time and modify these settings after the instance has been launched. Leave

the Incoming and Outbound options checked and click Next Step.

Amazon Connect resource configuration

Get up and running in a few easy steps

[Step 1: Identity management](#)

[Step 2: Administrator](#)

Step 3: Telephony options

[Step 4: Data storage](#)

[Step 5: Review and create](#)

Telephony Options

Amazon Connect offers the ability to accept inbound calls, make outbound calls, or both. You will claim a telephone number later.
Note: You will not be able to place or receive phone calls if you don't select the corresponding telephony options.

Incoming calls

☒ I want to handle **incoming** calls with Amazon Connect

Outbound calls

☒ I want to make **outbound** calls with Amazon Connect

Note: You can set which users can place outbound calls in user permissions.

[Cancel](#)

[Previous](#)

[Next step](#)

- The Data Storage screen allows you to customize where call reports, recordings, chat transcripts and exported reports are to be stored. The first option is to accept the default values for S3 where Amazon Connect automatically creates your S3 bucket, configures encryption for your bucket.

Amazon Connect resource configuration

Get up and running in a few easy steps

[Step 1: Identity management](#)

[Step 2: Administrator](#)

[Step 3: Telephony options](#)

Step 4: Data storage

[Step 5: Review and create](#)

Data storage

Call recordings, scheduled reports, and chat transcripts are stored in an Amazon S3 bucket that is created for you when you create an Amazon Connect instance. The stored data is encrypted by the AWS Key Management Service using a key specific to your Amazon Connect instance. Contact flow logs are stored in Amazon CloudWatch Logs in a Log Group created for you.

To successfully create an Amazon Connect instance, you need to use an AWS account that has access to both Amazon S3 and Amazon CloudWatch

Important: By choosing **Next step** you are granting Amazon Connect the following permissions:

- Read and write access to your S3 bucket to save and manage your data
- Encrypt/decrypt permissions to encrypt data
- Read and write access to CloudWatch Logs

Your data will be encrypted and stored here `amazon-connect-be5c4a4bb561/connect/amazonconnect-bootcamp1`

Your Contact flow logs will be stored here `/aws/connect/amazonconnect-bootcamp1`

[Customize settings](#)

[Cancel](#)

[Previous](#)

[Next step](#)

You also have the option of choosing a previously created S3 bucket and selecting your own KMS keys. Click on the "Customize settings" link and confirm the appropriate information in the designated sections are similar to the screenshots captured below.

AWS Key Management Service (KMS) is a managed service that makes it easy for you to create and control the encryption keys used to encrypt your data. AWS Key Management Service works with AWS CloudTrail to provide you with logs of API calls made to or by KMS. These logs help you meet compliance and regulatory requirements by providing details of when keys were accessed and who accessed them.

Call recordings

☒ Enable call recording
 Cancel Save

☒ Create a new S3 bucket for me (recommended)

Name ⓘ

Path prefix ⓘ

Location: connect-da085b77fdbc/connect/amazonconnectbootcamp1/CallRecordings

☐ Select an existing S3 bucket

Encryption options for call recordings

☒ Enable encryption

☒ Select KMS key by name
 ☐ Enter key ARN/ID

KMS master key ⓘ

ARN / ID

Description Default master key that protects my Connect calls when no other key is defined

Account 109779297014

Chat transcripts

☒ Enable chat transcripts

Cancel

Save

☒ Create a new S3 bucket for me (recommended)

Name connect-e71d0396db16 

Path prefix ChatTranscripts 

Location: connect-e71d0396db16/connect/amazonconnect-bootcamp1/ChatTranscripts

☐ Select an existing S3 bucket

Encryption options for chat transcripts

☒ Enable encryption

☒ Select KMS key by name

☐ Enter key ARN/ID

KMS master key aws/connect 

ARN / ID arn:aws:kms:eu-west-2:150934812383:key/67beee68-7613-47d8-a6da-148f46a22c0e

Description Default master key that protects my Connect calls when no other key is defined

Account 150934812383

Exported reports

☒ Enable exported reports

Cancel

Save

☒ Create a new S3 bucket for me (recommended)

Name connect-da085b77fdbc 

Path prefix Reports 

Location: connect-da085b77fdbc/connect/amazonconnectbootcamp1/Reports

☐ Select an existing S3 bucket

Encryption options for exported reports

☒ Enable encryption

☒ Select KMS key by name

☐ Enter key ARN/ID

KMS master key aws/connect 

ARN / ID arn:aws:kms:eu-central-1:109779297014:key/f66d31aa-c8b5-49ec-9668-262bf1d8cc90

Description Default master key that protects my Connect calls when no other key is defined

Account 109779297014

You can also enable Contact flow logs streaming to AWS Cloud Watch:

Contact flow logs

☒ Enable Contact flow logs
Your logs will be stored here /aws/connect/amazonconnectbootcamp1

If you make any changes, please be sure that you click on save otherwise "Next" button will be on grey.

By choosing **Next Step** you are granting Amazon Connect the following permissions:

- Read and write access to your S3 bucket to save and manage your data
- Encrypt/decrypt permissions to encrypt the data
- Read and write access to CloudWatch Logs

The permissions are given by auto-generated AWS IAM Role for Amazon Connect service, with attached appropriate Policy. The Policy can be modified after the instance has been launched.

10. Review the details listed on the final confirmation screen and click Create Instance.

Amazon Connect resource configuration

Get up and running in a few easy steps

Step 1: Identity management

Step 2: Administrator

Step 3: Telephony options

Step 4: Data storage

Step 5: Review and create

Review and Create

Review your choices. If everything is correct, you can finish configuring resources for Amazon Connect.

Identity management [Change](#)

Storing users within Amazon Connect

Access URL: https://amazonconnect-bootcamp1.my.connect.aws

Administrator [Change](#)

jsmith

Telephony [Change](#)

Inbound and outbound calls

Data storage [Change](#)

Call Recording S3 Locationamazon-connect-be5c4a4bb561

Encryption Keyaws/connect

Chat Transcript S3 Locationamazon-connect-be5c4a4bb561

Encryption Keyaws/connect

Exported Reports S3 Locationamazon-connect-be5c4a4bb561

Encryption Keyaws/connect

Attachments S3 LocationDisabled

Encryption KeyDisabled

Contact Flow Logs Location/aws/connect/amazonconnect-bootcamp1

Cancel

Previous

Create instance

11. Once complete, click Get Started on the Success! page.

Success!

Your Amazon Connect has been created.
Now you can begin to choose phone numbers, accept calls, and engage with your customers.

[Get started](#)

Users can access Amazon Connect at: <https://amazonconnectbootcamp1.aws...>
Please allow up to 15 minutes for the login page to become accessible.

Configure additional resources such as data streaming and CRM integrations within [Amazon Connect AWS console](#).

12. The first time you log in to your instance you will be greeted with a getting started wizard that will walk you through some basic steps to set up your call center. You can continue using the wizard by clicking "Let's go" or you can skip for now. Click Let's go.



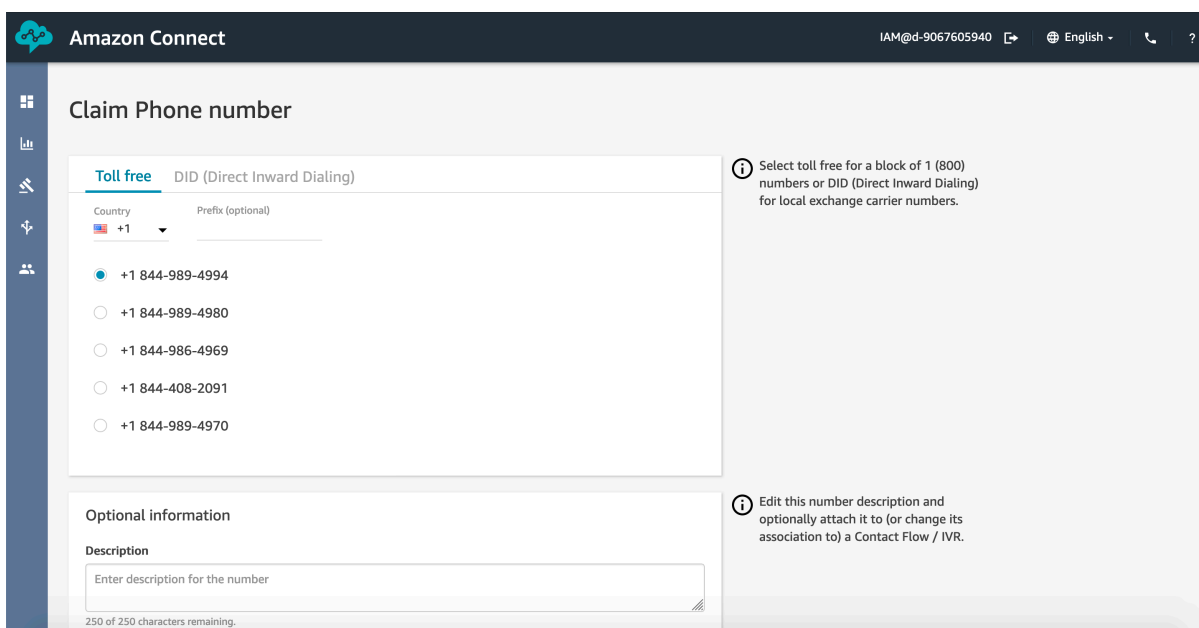
Welcome to Amazon Connect

We're excited to provide you a reliable communication service that connects with your customers.
Our first order of business is to help you claim a phone number. Are you ready?

[Let's go](#)

[Skip for now](#)

13. The first step in setting your call center is to claim a phone number. You have the option of a local Direct Dial number or a Toll-Free number. For the purpose of this lab, let's start with a tollfree US number. In some countries you can find a message like this "To claim a number in the selected country, please provide a valid business address in that country. Numbers that are claimed without providing a valid local business address may be revoked. To provide the address, please create a support case. Click here to create a support case now." Just click in create a support case and come back to the Claim number screen. Is not needed to create the case just now. For this lab, unless otherwise instructed please claim a tollfree US number.



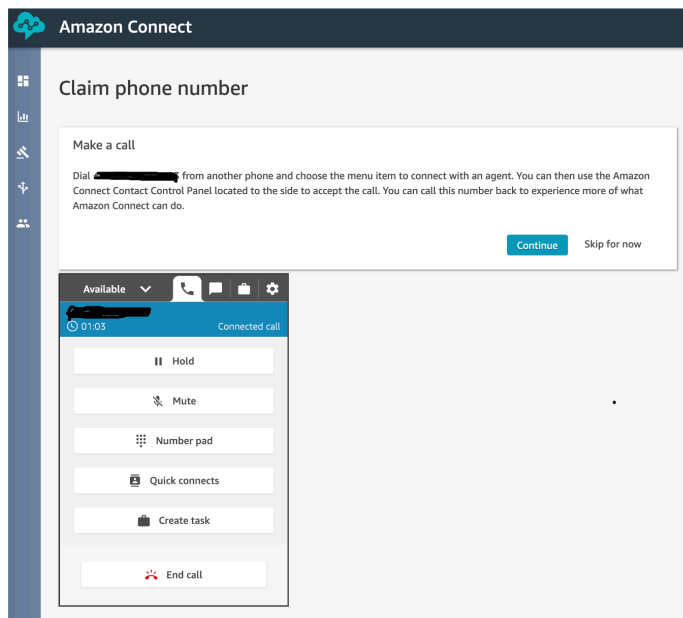
14. After you have chosen your phone number, you can make and accept calls using Contact Control Panel (CCP)- *it may take a couple of minutes for the number claim process to complete*. Please be sure that you are using a WebRTC enabled browser like Firefox or Chrome and that you have accepted the access to the microphone.

- Dial your phone number from your mobile phone.
- When prompted, choose option 1 to be placed in queue for an agent.
- A sample flow for testing changes in priorities is launched. Choose 1.
- The prompt will ask for callback request or keep in queue. Choose 1 again
- To accept an incoming call, choose **Accept call**
- To place the call on hold, choose **Hold**
- To end the call, choose **End Call**
- To change a status, choose **Change Status**
- To edit settings (Language), click on Gear icon (top left corner)
- To dial a number, click on **Dial number**

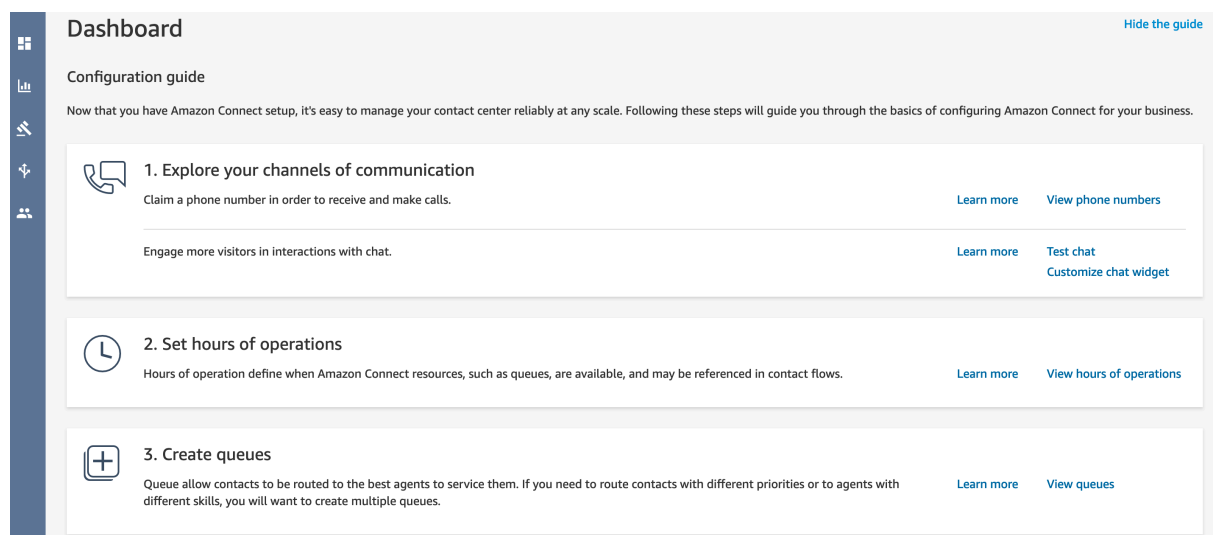
More information about Agent Console can be found in the official documentation:

<https://docs.aws.amazon.com/connect/latest/userguide/agentconsole-guide.html>

Note: Depending on the browser you are using, you may be asked if you would like your Contact Center instance to access your default microphone. If you do not have a headset available, you can allow this.



Once the test call has been finalized you can click on Continue button and a dashboard with a configuration guide will be shown.



It is possible to test the chat as well by clicking on the link "Test chat" or Customize chat widget, but we will skip that for now.

15. You can claim a phone number when you first create an instance, from the Dashboard, or you can claim one from the Manage Phone Numbers screen.

You can also release a number that you no longer use. There is a soft-limit of ten numbers per contact center. If you reach your limit and no longer need a number, you can release it to make space for a new number. To do this, choose Release and then Release. You cannot claim the same number again. Amazon Connect uses the E.164 format for international dialing. This formatting is pre-configured in the CCP. *(E.164 defines a general format for international telephone numbers, more information: <https://en.wikipedia.org/wiki/E.164>)*

You can find more information about Amazon Connect Instance Configuration in the official documentation:

<https://docs.aws.amazon.com/connect/latest/adminguide/amazon-connect-instance.html>

Amazon Connect Queues and Routing Profiles

For the purpose of our case study, we need to create 2 queues, one for authenticated customers and another for unknown customers. Both queues are going to use the same Hours of Operation: Monday to Friday, 8:00 AM until 7:00 PM.

1. Navigate to Routing > Hours of operation
2. Click **Add New Hours**
3. Set Name as **EmployeeBooking_DefaultHours**
4. Set the appropriate time zone
5. Set working days and hours according to requirements
6. Remove Saturday and Sunday
7. Click on **Save**

Hours of operation

Name
EmployeeBooking_DefaultHours

Description
Enter your description...
250 of 250 characters remaining.

Time zone:
Europe/London

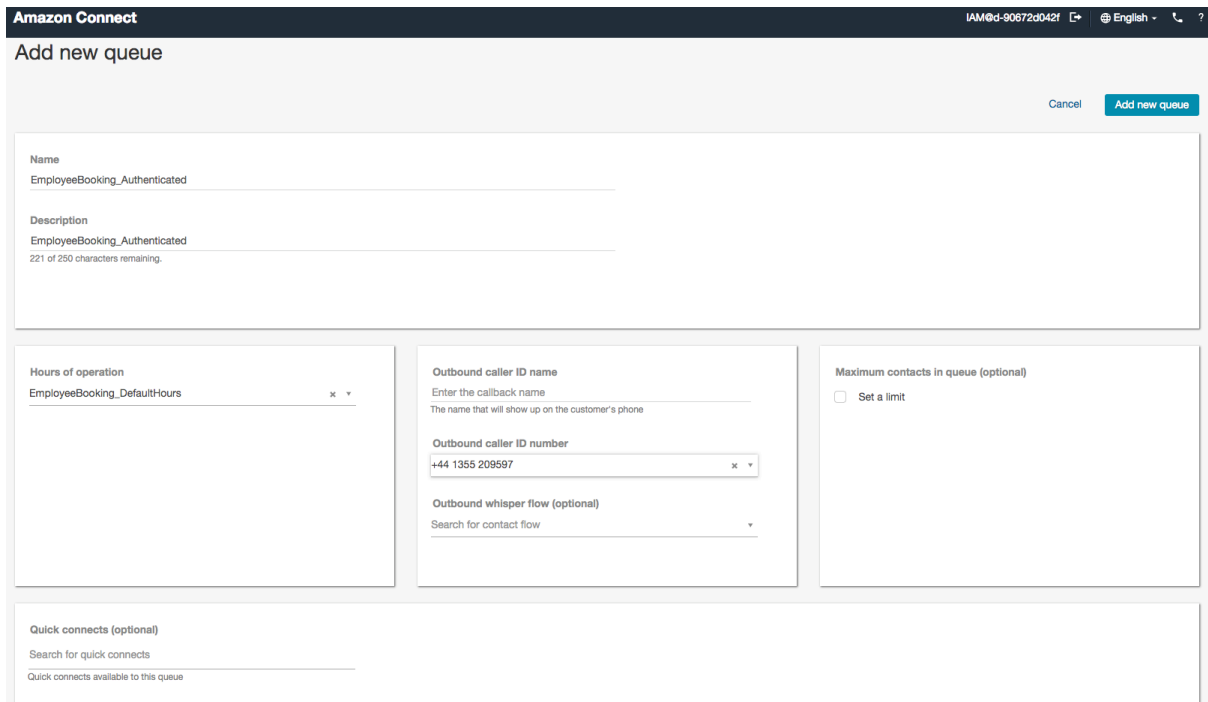
Add new **Remove** **Edit**

Day	Start	End
☐ Monday	08 : 00 AM	07 : 00 PM
☐ Tuesday	08 : 00 AM	07 : 00 PM
☐ Wednesday	08 : 00 AM	07 : 00 PM
☐ Thursday	08 : 00 AM	07 : 00 PM
☐ Friday	08 : 00 AM	07 : 00 PM

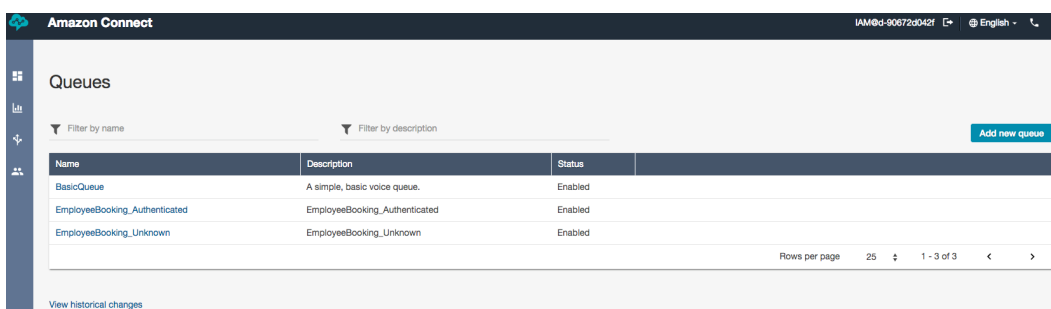
Save **Cancel**

8. Navigate to Routing > Queues
9. Click **Add New Queue**
10. Enter the queue name as **EmployeeBooking_Authenticated**
Queue names should reflect the skills required to solve problems. When naming, ensure the name aligns with the skill required for agents who work this queue.
11. Enter a description
12. Select Hours of Operation - **EmployeeBooking_DefaultHours**
13. Select the Outbound caller ID number (This number will also be used for callbacks).
It is possible to set Outbound caller ID dynamically – please check the official documentation:
<https://docs.aws.amazon.com/connect/latest/adminguide/queues-callerid.html>
Outbound Whisper flow is optional, and it's used when an agent is dialing out, to greet a customer, or to play any custom message, for example, if the phone call will be recorded. For the purpose of this lab, please leave it blank.
14. Maximum contacts in queue is optional, and it's used to limit the number of calls in queue – the contact flow can take different branch if the limit is reached. For the purpose of this lab, please leave it blank.

15. Quick Connects are optional. Amazon Connect uses Quick Connects as a one-click field for contacts, DID numbers, or queues that an agent can transfer calls to. Quick connects are available based on the queues agents are configured to be working. For the purpose of this lab, please leave it blank.
16. Click on **Add New Queue**
17. Repeat steps 8-17 for **EmployeeBooking_Unknown** queue



18. Both queues are now created, along with default BasicQueue



Name	Description	Status
BasicQueue	A simple, basic voice queue.	Enabled
EmployeeBooking_Authenticated	EmployeeBooking_Authenticated	Enabled
EmployeeBooking_Unknown	EmployeeBooking_Unknown	Enabled

Routing Profiles

Amazon Connect implements the concept of Routing Profiles. A Routing Profile is a collection of queues that determines how contacts are routed to agents.

Routing Profiles are used to prioritize contacts across specific queues and manage the priority in which contacts are handled based on the queues they are routed to. This can be used to ensure alignment with service SLAs. Routing Profiles are managed and assigned to agents by the Administrator.

An agent can only be assigned a single Routing Profile at a time; however, they may serve multiple queues, based on rules defined in the Routing Profile.

Once a Routing Profile is created, it becomes easy to assign this profile to an agent who inherits the queues contained within the profile. Pending calls become available for agents based on priority and delay values configured within the Routing Profile.

Routing Profiles make it easy to make bulk changes that impact many users with a single configuration change. Sudden changes in call volume can be addressed by adjusting delay values that traditional would require rebasing in order to compensate for the sudden increase in call volume.

Lastly, Routing Profile values are limited to the discrete queues that have been created. This prevents significant overhead common in skills management and allows routing administrators and IVR administrators collaboration when designing a customer-centric contact center.

For the purpose of our lab, we need to create a single **Routing Profile**, giving the priority to Authenticated users.

1. Navigate to Users > Routing Profiles
2. Click **Add Profile**
3. Enter the Name **EmployeeBooking_Default**
4. Enter the Description (this is required)
5. Set first queue **EmployeeBooking_Authenticated** with priority 1
6. Add second queue **EmployeeBooking_Unknown** with priority 2
7. *Sub-priority values are set in order of importance from 1 to 99. The lowest number is the highest sub-priority.*
8. Select Default outbound queue **EmployeeBooking_Unknown** (When an outbound call is placed by an agent, it must be associated with a queue)
9. Leave all the Chat and Task checkboxes unchecked.
10. Click **Add new profile**

Add new routing profile

Name: EmployeeBooking_Default

Description: EmployeeBooking_Default (227 of 250 characters remaining)

Set channels and concurrency: Defines which channels agents can handle in the CCP. Note: If all channels are selected, agents will have access to only one channel at a time for inbound contacts. [Learn more](#)

☒ Voice
☐ Chat
☐ Task

Routing profile queues

If no queue is added, the agent will only be able to make outbound calls. At least one queue is needed for inbound calls, and must not be a duplicate.

Name	Channels	Priority	Delay (in seconds)
EmployeeBooking_Authenticated	☒ Voice ☐ Chat ☐ Task	1	0
EmployeeBooking_Unknown	☒ Voice ☐ Chat ☐ Task	2	0

Default outbound queue: EmployeeBooking_Unknown

11. Your Routing Profile is now created, alongside default **Basic Routing Profile**

Routing profiles

Filter by name: Filter by description

Name	Description	No. of associated queues	No. of agents staffed
Basic Routing Profile	A simple routing profile.	1	2
EmployeeBooking_Default	EmployeeBooking_Default	2	0

Rows per page: 25 1 - 2 of 2

Permissions

Permissions are required for users to access, view, and edit certain administrative functionality and tools. Security profiles determine which users and roles can view and perform specific tasks. Default security profiles:

- **Admin** – can access and edit all available resources and actions
- **Agent** – can access CCP only
- **CallCenter Manager** – can access user management, metrics, and routing settings
- **Quality Analyst** – can access metrics only

More information can be found in the official documentation:

<https://docs.aws.amazon.com/connect/latest/userguide/users.html>

For the purpose of our lab, we need to create a single **agent**:

1. Navigate to Users > User Management
2. *You have the option of creating individual users or upload multiple users using a CSV template. It is recommended that you have Security Profiles, Routing Profiles, Queues, and other elements configured before adding users.*
3. Select **Add New Users > Create** and set up a new user
4. Enter **First Name, Last Name, LoginName, EmailAddress and Password** (minimum of 8 characters, at least 1 uppercase alphabet, 1 lowercase alphabet and 1 number)
5. Select **Routing Profile: EmployeeBooking_Default**
6. Select **Security Profiles: Agent**
7. Select **Phone Type: Soft Phone**
8. You can set **Auto-Accept Call** option, if desired
9. Set After Call Work timeout: **5** seconds.

Note: After a call is terminated, the agent will be set in After Call Work mode, not available for another call. Once the timer expires, the agent will become available automatically.

Note: Agents and teams can be organized into groupings based on their location and their skill sets. Hierarchies can be configured with up to five levels and allow you to segment agents or teams. You can create large groups, such as all agents who work on a specific continent, or smaller groups such as all agents working in a specific department. The hierarchies are reflected in reports and historical metrics to allow for more granular reporting.

10. Click on **Save**, then **Create users**. **NOTE:** If you don't click on Create users the user won't be created.

11. The agent is created, alongside Contact Center administrator (created during instance launch wizard)

Contact Flow Design

Amazon Connect end-customer interactions are enabled via Contact flows. These are similar in concept to Interactive Voice Response (IVR) functionality. Contact flows comprise blocks of functionality such as Play Prompt, Branch on Customer Input, Invoke External Functionality, Transfer to Queue, Securely Take Customer Sensitive Information, or Terminate Call. Contact flows determine the automated end-customer experience, and every contact that Amazon Connect handles is processed by at least one Contact Flow.

Amazon Connect Contact Flows are very flexible and extensible. You can use them to play a prompt to a customer, ask a customer a question, make a call to an external data source (a “data dip”), branch to different paths within the Contact Flow, or transfer to an agent or another Contact Flow. Contact Flows are based on a simple drag and drop graphical user interface (GUI) in the Amazon Connect console.

You can save the flow as you build it and it's only updated in production when you publish it. You can revert to the most recent published version and also save-as to create a new flow built on a published Contact Flow while maintaining the published one.

More information about Contact Flows can be found in the official documentation: <https://docs.aws.amazon.com/connect/latest/userguide/contactflow.html>

Amazon Connect contains a set of different flow types:

- **Contact Flow** is the main type and it's used to implement the self-service application for our Contact Center. You can link contact flows to create smaller, more manageable contacts flows. For example, we can use one contact flow for customer authentication, then another contact flow for chatbot interaction.
- **Customer Queue Flow** gives the wait treatment to the customer while waiting in a queue. Usually, once the customer completes the self-service application (**Contact Flow**), we transfer it to **Customer Queue Flow**.
- **Customer Hold Flow** gives the wait treatment while the customer is on hold (agent puts customer on-hold). It can contain simple music-on-hold, or some more advanced, dynamic and personalized treatment.
- **Customer Whisper Flow** gives a Whisper message (announcement) to the customer once the agent is connected. Default customer Whisper flow gives just a simple beep, but it can be configured with more complex and dynamic announcements.
- **Outbound Whisper Flow** is used when an agent is dialing out, to greet a customer, or play any custom message (for example, highlighting when a phone call will be recorded).
- **Agent Hold Flow** gives the wait treatment while the agent is on hold. It can contain simple music-on-hold, or some more advanced and dynamic treatment.
- **Agent Whisper Flow** gives a Whisper message (announcement) to the agent once the customer is connected. Default agent Whisper flow gives a Queue name, but it can be configured with more complex and dynamic announcements. In our example, we will be using this flow to Whisper the employee's name to the agent.

- **Transfer to Agent Flow** gives a voice treatment when transferring to another agent. Default one gives a simple message "Transferring now...", but it can be configured with more complex and dynamic announcements.
- **Transfer to Queue Flow** give a voice treatment to the caller, when transferring to another queue, but also implements a logic to check hours of operation for the destination queue and also if there are staffed agent in that queue. It can be configured with more complex and dynamic announcements.

Amazon Connect comes with all default contact flows, ready to use, but you can design your own queues, according to your requirements. The Contact Flow Palette contains nodes to set hold flow, queue flow or Whisper flow dynamically, during the call processing.

Amazon Connect supports text-to-speech, including SSML or plaintext with (or without) dynamic attributes. You can enter text-to-speech prompts in any of the contact flow blocks that support prompt entry, such as **Play prompt** and **Get customer input**. The text-to-speech voice is selected in the **Set voice** contact block. More information about using text-to-speech can be found in the official documentation:

<https://docs.aws.amazon.com/connect/latest/adminguide/prompts.html>

Amazon Connect uses Amazon Polly, a service that converts text into lifelike speech using Speech Synthesis Markup Language (SSML). More information about SSML tags can be found in the Amazon Polly Developer Guide:

<https://docs.aws.amazon.com/polly/latest/dg/ssml.html> and

<https://docs.aws.amazon.com/polly/latest/dg/supportedtags.html>

Amazon Polly can be tested [directly from AWS Console](https://console.aws.amazon.com/polly/home/SynthesizeSpeech), by navigating to <https://console.aws.amazon.com/polly/home/SynthesizeSpeech>

You can test both Plain text and SSML tags. For example, enter the following text in the Plaintext tab and click **Listen to Speech**:

We can speak at normal volume, or we can whisper.

We can change the way we say certain words, such as tomato or tomato.

Here is a number 5551212, here it is differently 5551212.

Here it is as a UK Phone number 5551212.

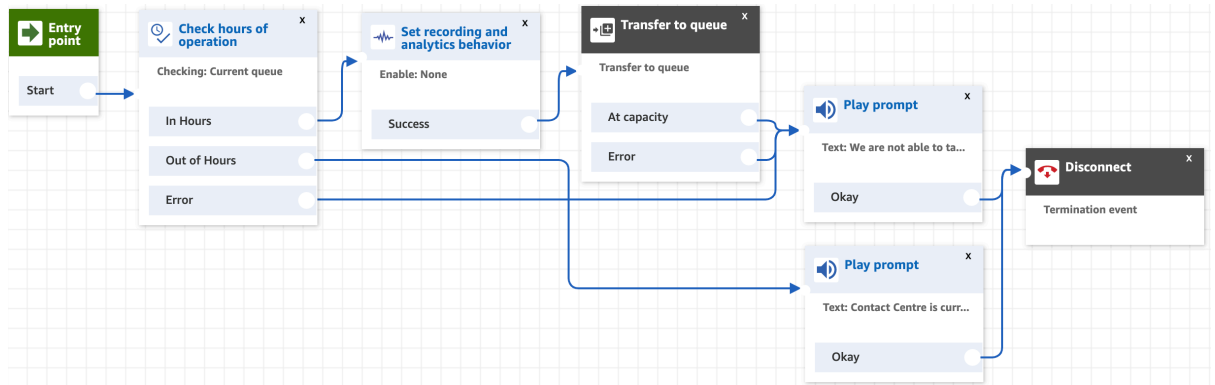
As a SSML test, enter the following text in the SSML tab and click **Listen to Speech**:

```
< speak > < p > We can speak at normal volume < amazon:effect name="whispered"> or we can whisper. < /amazon:effect > < /p > < p > We can change the way we say certain words, such as tomato. or < phoneme alphabet="ipa" ph="tʊ'mɑ:təʊ"> tomato < /phoneme > < /p > Here is a number, < say-as interpret-as="digits"> 5551212 < /say-as >, here it is differently < say-as interpret-as="cardinal"> 5551212 < /say-as >. Here it is as a UK Phone number. < lang xml:lang="en-GB"> < say-as interpret-as="telephone"> 5551212 < /say-as > < /lang > < /speak >
```

First, we are going to create a Contact Flow which will support checking the Hours of Operation and transferring to a queue.

This Contact Flow has been designed as a module, and can be reused anytime you want to put call in a queue. The best practice is to always design reusable modules, rather than creating one large, monolithic flow for each line of business.

1. Navigate to Routing > Contact Flows > Create Contact Flow
2. Enter contact flow name: **EmployeeBooking_TransferToQueue**
3. Enter a description
4. Place **Check Hours of Operation** node found under the Branch Section.
5. Place **Set Call Recording and Analytics Behaviour** node and set **Agent and Customer option**
6. Place **Transfer to Queue** node and connect the previous node to this node
7. Place **Play Prompt** node, select **Text-to-Speech** and type in "We are not able to take your call right now, please try again later."
8. Place another **Play Prompt** node, select **Text-to-Speech** and type in "Contact Centre is currently closed. Working hours are Monday to Friday, from 8AM until 7PM. Goodbye."
9. Place **Disconnect** node
10. Connect nodes according to the given flow:



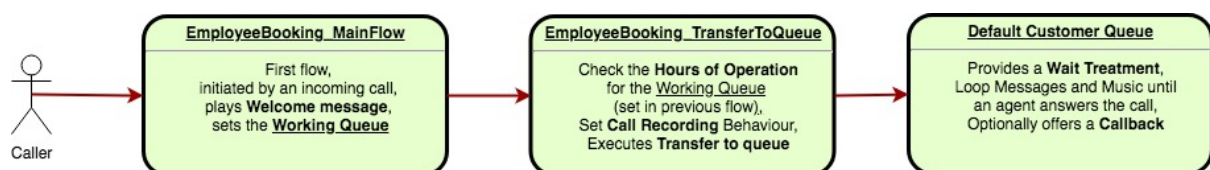
11. Save and Publish flow

Once the call is transferred to the queue, the wait treatment will be given by the **Default Customer Queue** flow. *This is also a module, which can be reused for different lines of businesses.*

Please open the **Default Customer Queue** flow and review its design. It contains a **Loop Prompts** node, which uses Amazon Polly text-to-speech to say "Thank you for calling. Your call is very important to us and will be answered in the order it was received." And then plays a static announcement – a .wav file that contains music.

You may have completely different wait treatments for different queues. For that purpose, you would use different **Customer Queue** type flows. However, for our lab, we will use the default one for both our queues.

Next, we need to create our **main Contact Flow**, that will be invoked when a customer calls in. The sequence of flows will look like this:

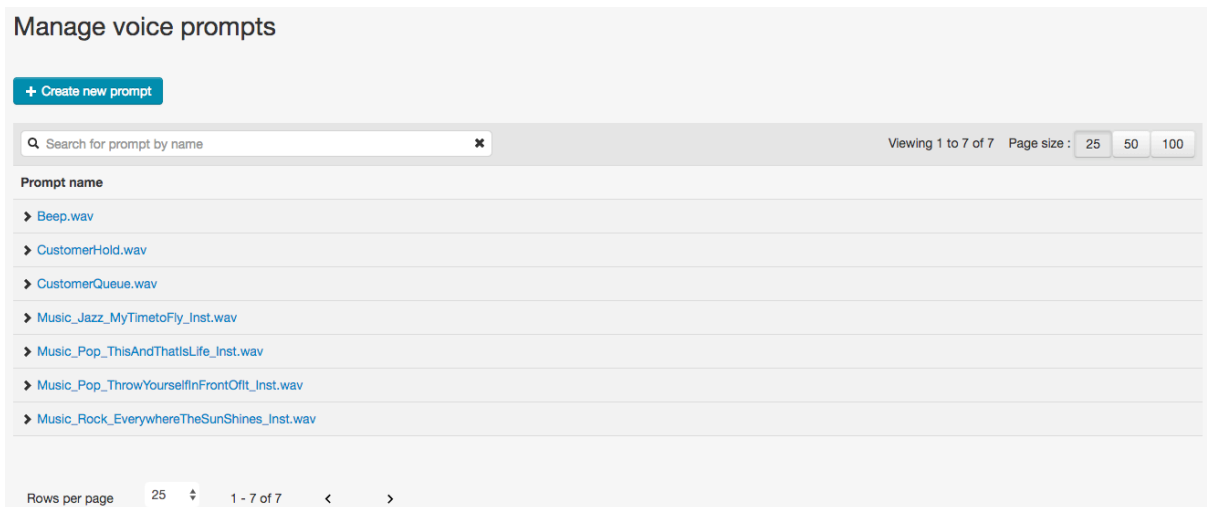


In this module, our main flow will stay simple, giving a welcome message and transferring to our **EmployeeBooking_TransferToQueue** flow, which we created in the previous step. Before the welcome message, we want to give a 'chime', which will be uploaded as a static announcement (prompt).

Prompts are audio files played in call flows. You can upload a pre-recorded .wav file prompt, or record one in the web application. Prompts and routing policies should be aligned with each other to ensure a smooth call flow for customers.

To create a prompt:

1. Choose Routing>Prompts
2. You'll be able to see existing (default) prompts



Manage voice prompts

[+ Create new prompt](#)

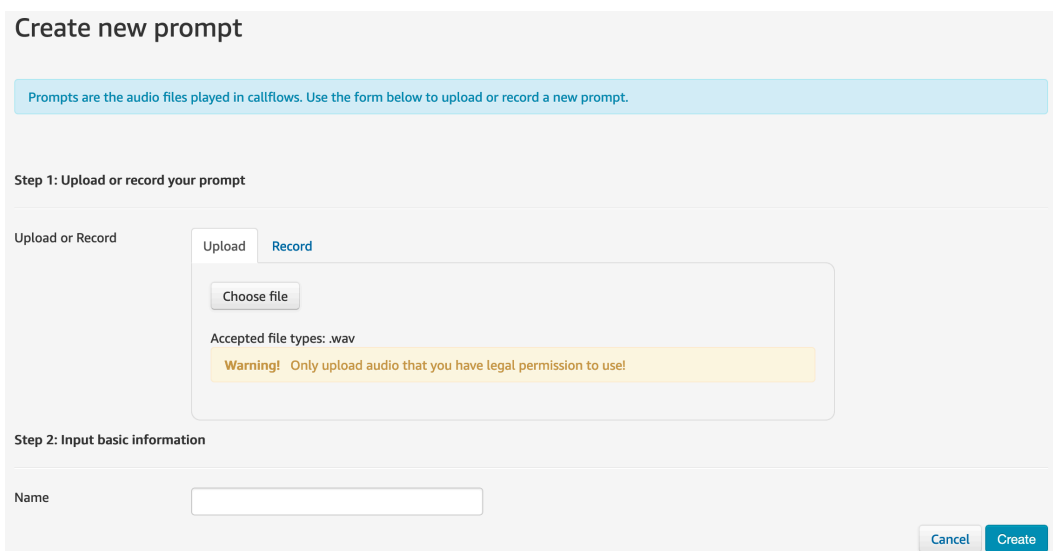
Search for prompt by name

Viewing 1 to 7 of 7 Page size: 25 50 100

Prompt name
> Beep.wav
> CustomerHold.wav
> CustomerQueue.wav
> Music_Jazz_MyTimetoFly_Inst.wav
> Music_Pop_ThisAndThatIsLife_Inst.wav
> Music_Pop_ThrowYourselfInFrontOfIt_Inst.wav
> Music_Rock_EverywhereTheSunShines_Inst.wav

Rows per page: 25 1 - 7 of 7 < >

3. On the Manage voice prompts screen, choose **Create New Prompt**
4. You can choose the following actions:
 - a. **Upload**—Choose the file to upload
 - b. **Record**—Choose the red circle to begin recording. Use the red square to stop. You can choose Crop to cut the recorded prompt or Discard to record a new prompt
5. For our lab purposes, upload the given .wav file
<https://s3.amazonaws.com/amazonconnectbootcampresources/WelcomeChime.wav>
6. Set Name as **WelcomeChime**
7. Click on **Create**



Create new prompt

Prompts are the audio files played in callflows. Use the form below to upload or record a new prompt.

Step 1: Upload or record your prompt

Upload or Record: Upload Record

Choose file

Accepted file types: .wav

Warning! Only upload audio that you have legal permission to use!

Step 2: Input basic information

Name:

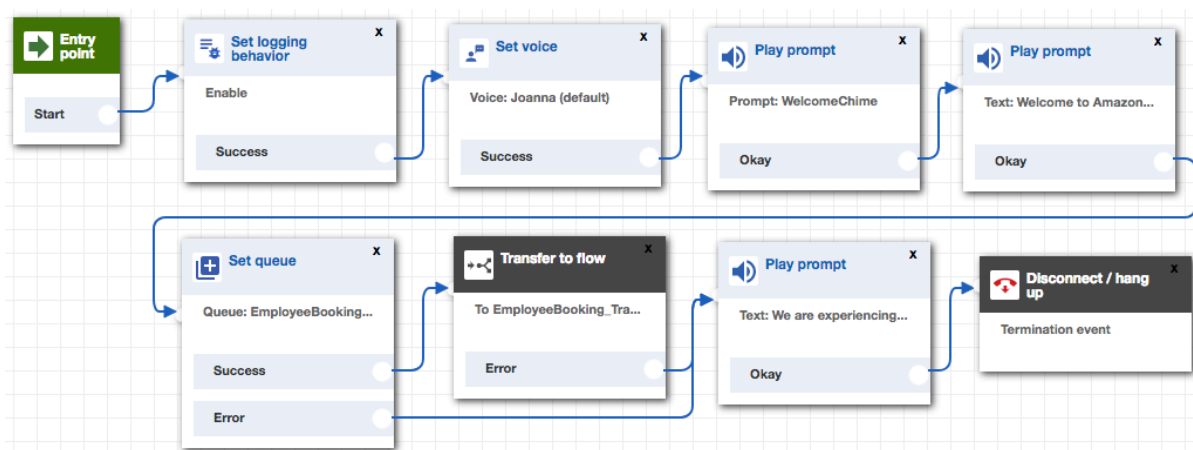
Cancel Create

8. To manage existing prompts, choose Routing > Prompts and select the appropriate prompt. You can choose Play, Download, Edit, or Delete.

Choose Save to apply the changes.

We are now ready to create our main Contact Flow:

1. Navigate to Routing > Contact Flows > Create Contact Flow
2. Set name as **EmployeeBooking_MainFlow**
3. Enter a description for the contact flow
4. Place **Set Logging Behavior** node and **Enable** it (*all contact flow logs will be available in AWS Cloud Watch*)
5. Place **Set Voice** node and set desired language and voice for Amazon Polly text-to-speech (*English,US > Joanna is the default*)
6. Place **Play Prompt** node and select **WelcomeChime** prompt
7. Place another **Play Prompt** node and enter the text: "Welcome to the Blunder Mifflin Employee Booking service."
8. At this point, we want to simply transfer our call to our queue. First, we have to set Queue, hence place **Set Working Queue** node and select **EmployeeBooking_Unknown** queue.
9. Place **Transfer to Flow** node and select our **EmployeeBooking_TransferToQueue** flow.
10. Place another **Play Prompt** node which will handle errors that might occur ("We are experiencing technical difficulties. Please try again later.")
11. Place **Disconnect** node



12. Save and Publish the flow

Next step is to connect our main Contact Flow to our Phone Number:

1. Navigate to Routing > Phone numbers
2. Select your phone number, this will take you to the Edit Phone number screen
3. Select **EmployeeBooking_MainFlow**
4. Save

Edit Phone number

+44 1355 209597

Description

First Phone Number

232 of 250 characters remaining.

Contact flow / IVR

EmployeeBooking_MainFlow

Save Cancel

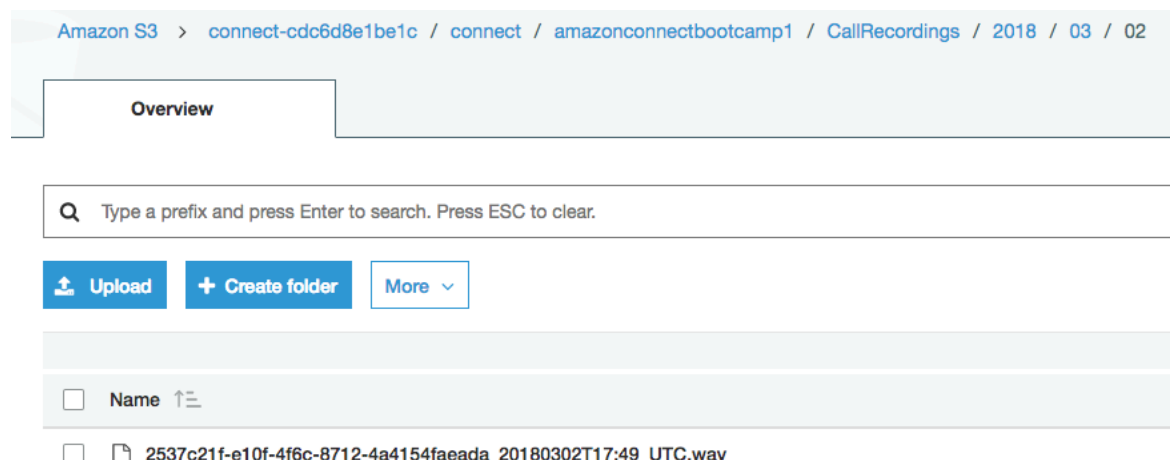
Receive the First Call

To receive your first call, you have to log the agent into the CCP application:

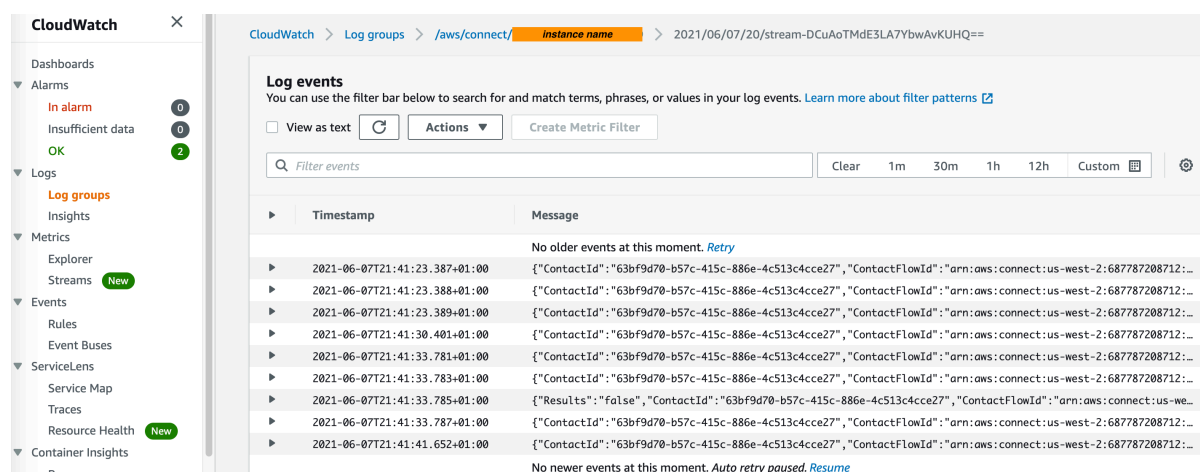
1. Open another instance of your browser in Incognito/Private mode
2. Navigate to your CCP URL: (<https://instancename.my.connect.aws/ccp-v2/softphone>), enter credentials and set your agent to Offline.
3. When prompted, allow the CCP to use your microphone and send you notifications.
4. Place a call to your phone number
5. You should be able to hear the Welcome Chime, Welcome Message, Queue Message, and music while waiting in the queue
6. Change the agent's status to **Available** and answer the call
7. Speak so that you can listen to the recording later
8. Hang up the call
9. The agent will be in After Call Work state for 5 seconds (if configured on agent's form) then automatically to Available State.

To find your recording, navigate to Metrics and Quality > Contact Search, find your contact and click on **Play** button. It is a stereo recording, with the customer audio on one channel and the agent on another. You can also download the recording.

Additionally, you can check your S3 bucket; select S3 service from AWS console, click on **connect-***** bucket, and drill down until you find your .wav file.



From the [AWS Console](#), navigate to [AWS Cloud Watch](#) > [Log groups](#) > [/aws/connect/instancename](#) and drill down through log files. You'll be able to see one record per each node visited in Contact Flow.



Amazon Connect integrates with CloudWatch so that you can collect, view, and analyze CloudWatch metrics for your Amazon Connect virtual contact centre. More information can be found in the official documentation: <https://docs.aws.amazon.com/connect/latest/adminguide/monitoring-cloudwatch.html>

Hands-on Module 2: Amazon Connect and Amazon Lex

Objective: Allow authenticated employees to book a hotel, by using Amazon Lex chatbot with Natural Language Understanding.

Amazon Lex

Amazon Lex is a service for building conversational interfaces into any application using voice and text. Amazon Lex provides the advanced deep learning functionalities of automatic speech recognition (ASR) for converting speech to text, and natural language understanding (NLU) to recognize the intent of the text, to enable you to build applications with highly engaging user experiences and lifelike conversational interactions.

With Amazon Lex, the same deep learning technologies that power Amazon Alexa are now available to any developer, enabling you to quickly and easily build sophisticated, natural language, conversational bots (“chatbots”).

Speech recognition and natural language understanding are some of the most challenging problems to solve in computer science, requiring sophisticated deep learning algorithms to be trained on massive amounts of data and infrastructure. Amazon Lex democratizes these deep learning technologies by putting the power of Amazon Alexa within reach of all developers. Harnessing these technologies, Amazon Lex enables you to define entirely new categories of products made possible through conversational interfaces.

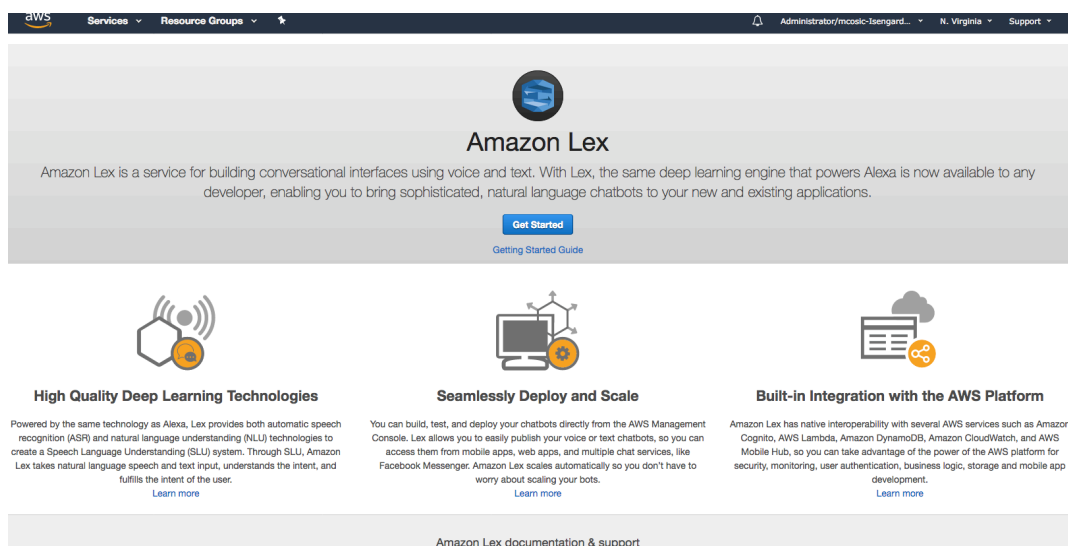
More information about Amazon Lex can be found in the official documentation: <https://docs.aws.amazon.com/lex/latest/dg/what-is.html>

For our use-case, we will create a Lex chatbot which will allow employees to book a hotel. The Lex bot will interact with the voice caller, giving options to choose Location (City), Check-in date and number of nights the caller wants to stay in. If the caller cannot execute their requirements through chat self-service, the bot will allow a transfer-to-agent function to connect the caller with the contact center agent.

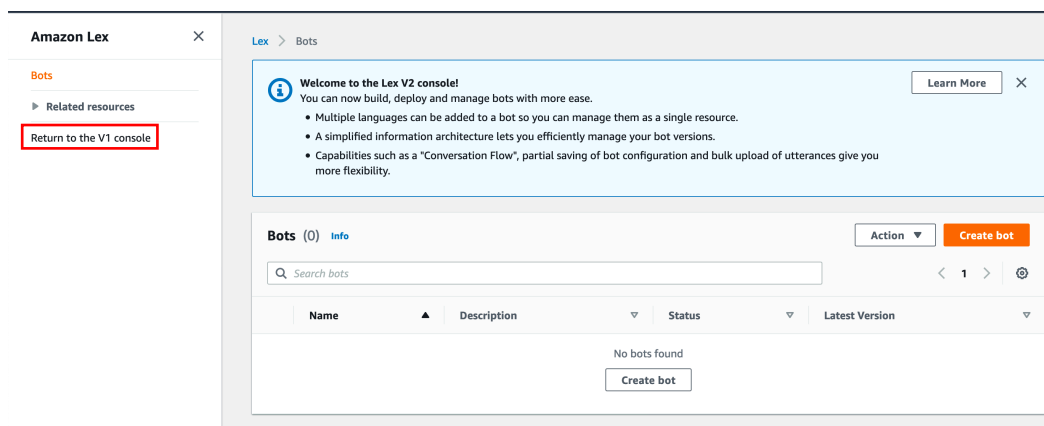
Build your Lex bot

In our use-case, we are going to implement an Amazon Lex bot with two intents – 'book a hotel', and 'speak to an agent'. In a real use-case scenario, the bot can be improved to support booking a flight, checking the status, rescheduling a flight, or even renting a car. Each of these functions would be represented as separate intents.

1. In your AWS console search bar, type **"Lex"** and select **Amazon Lex** from the results
2. *Ensure that your selected region is "US East (N. Virginia)" by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.*
3. Click **Get Started**

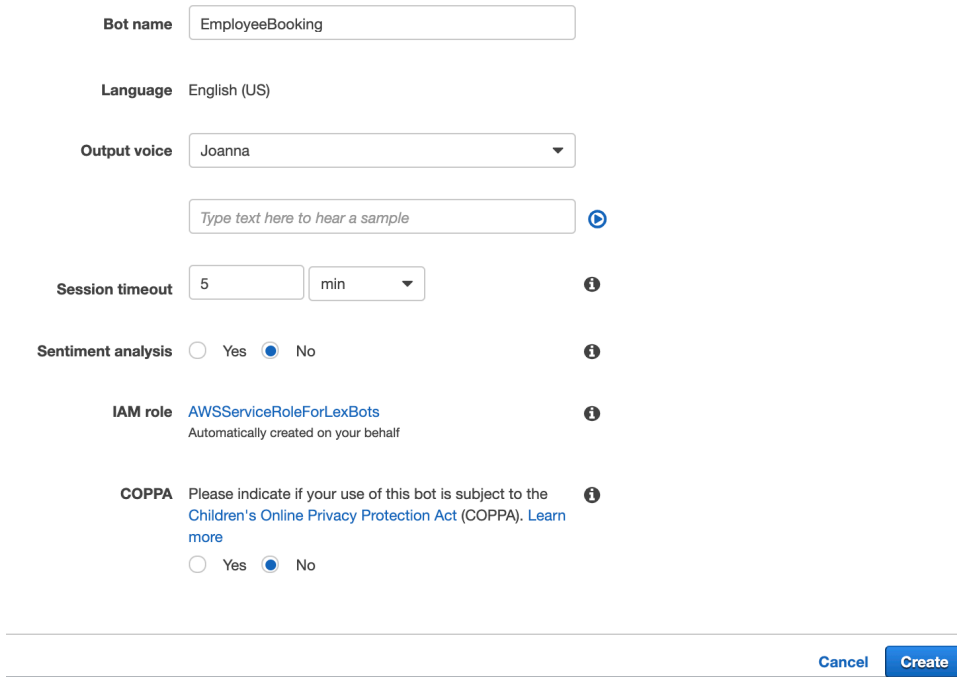


4. This workshop works with Amazon Lex v1, please make sure that you are using Amazon Lex v1. You can go to the v1 console by clicking on "Return to v1 console". Screen shot below for reference



5. Once you are on the lex v1 console
6. Choose **Custom Bot**
7. Bot name: **EmployeeBooking**
8. Language is limited to **English (US)**
9. Output voice: **Joanna**
10. Session timeout: **5 minutes**
11. Sentiment Analysis: **No**
12. You will need to select "**No**" for the COPPA option prior to being allowed to create the bot.

13. Click **Create**



Bot name EmployeeBooking

Language English (US)

Output voice Joanna

Type text here to hear a sample

Session timeout 5 min

Sentiment analysis ☐ Yes ☒ No

IAM role [AWSServiceRoleForLexBots](#)
Automatically created on your behalf

COPPA Please indicate if your use of this bot is subject to the [Children's Online Privacy Protection Act \(COPPA\)](#). [Learn more](#)
☐ Yes ☒ No

[Cancel](#) [Create](#)

14. The console sends Amazon Lex a request to create a new bot.

15. Next to the bot name in the console is the bot version, Latest.

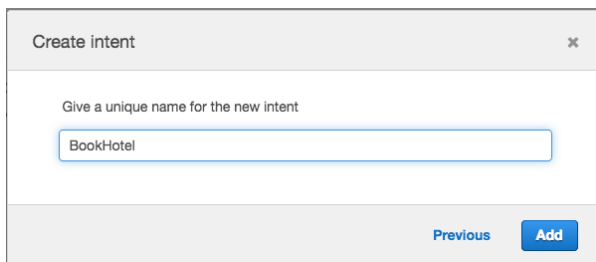
New Amazon Lex resources have **LATEST** as the version.

There are no Intents or Slot types created at this time.

16. The **Build** and **Publish** are bot level activities. After you configure the entire bot, you'll learn more about these activities.

17. Choose **Create Intent > Create Intent**

18. Name: **BookHotel**



Create intent

Give a unique name for the new intent

BookHotel

[Previous](#) [Add](#)

19. Click **Add**

20. Next step is to add **Sample utterances**

- a. "Book a hotel"
- b. "I want to make hotel reservations"
- c. "Book a {Nights} night stay in {Location}"

21. Create **Slots** (data the user must provide to fulfill the intent). For each slot, you have to provide **Name**, **Slot type** and **Prompt**. All 3 slots are required (Required by checkbox).

Slots						
Priority	Required	Name	Slot type		Prompt	
		e.g. Location	e.g. AMAZON.US_CITY		e.g. What city?	
1.	☒	Location	AMAZON.US_CITY	Built-in	What city will you be staying in?	
2.	☒	CheckInDate	AMAZON.DATE	Built-in	What day do you want to check in?	
3.	☒	Nights	AMAZON.NUMBER	Built-in	How many nights will you be staying?	

22. Set **Confirmation prompt**

- a. **Confirm**
 - i. "Okay, I have you down for a {Nights} night stay in {Location} starting {CheckInDate}. Shall I book the reservation?"
- b. **Cancel**
 - i. "Okay, I have canceled your reservation in progress."

23. Set **Fulfillment** > **Return parameters to client** > **Add message** ("Thanks, I have placed your reservation.")

▼ Fulfillment ⓘ

☐ AWS Lambda function ☒ Return parameters to client

▼ Response ⓘ

[Preview](#)

|| ☒ Message ☐ Custom Markup ⓘ

One of these messages will be presented at random.

e.g. Thank you. Your {Drink_Name} has been ordered.

Thanks, I have placed your reservation.

[Add Message](#)

24. Save at the bottom of the page.

25. Review the **Error Handling** from the menu on the left-hand side of the screen.

Clarification prompts are going to be played in case the input was not understood, while **Hang-up phrase** is going to be played as a final message (after **Maximum number of retries** exceeds).

Error handling

Clarification prompts

e.g. Sorry, can you please repeat that?

Sorry, can you please repeat that?

Maximum number of retries

Hang-up phrase

e.g. Sorry, I could not understand. Please contact customer support.

Sorry, I could not understand. Goodbye.

[Save](#)

26. Click **Save**

27. Click the **“Plus”** button (icon) to add a new Intent

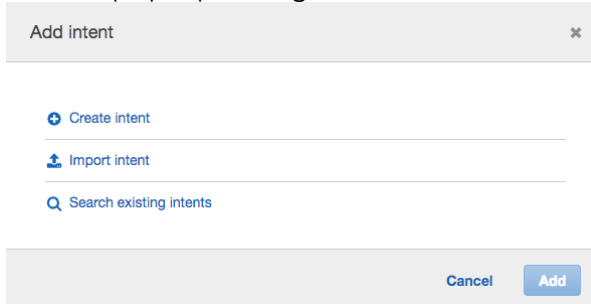
< EmployeeBooking Latest ▼

[Editor](#) [Settings](#) [Channels](#) [Monitoring](#)

Intents

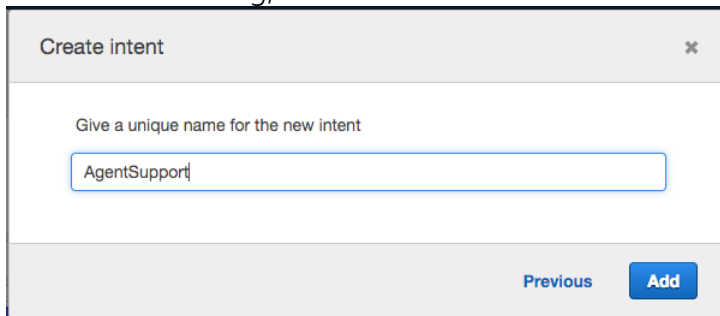
BookHotel

28. On the pop-up dialog, select **Create intent**



A dialog box titled "Add intent" with a close button (X) in the top right corner. It contains three options: "Create intent" (with a plus icon), "Import intent" (with a plus icon and a document icon), and "Search existing intents" (with a magnifying glass icon). At the bottom, there are "Cancel" and "Add" buttons.

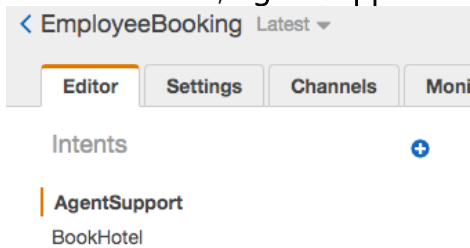
29. On the next dialog, enter a name for the new Intent – **AgentSupport**



A dialog box titled "Create intent" with a close button (X) in the top right corner. It contains a text input field with the placeholder text "Give a unique name for the new intent". The field contains the text "AgentSupport". At the bottom, there are "Previous" and "Add" buttons.

Click the **Add** button

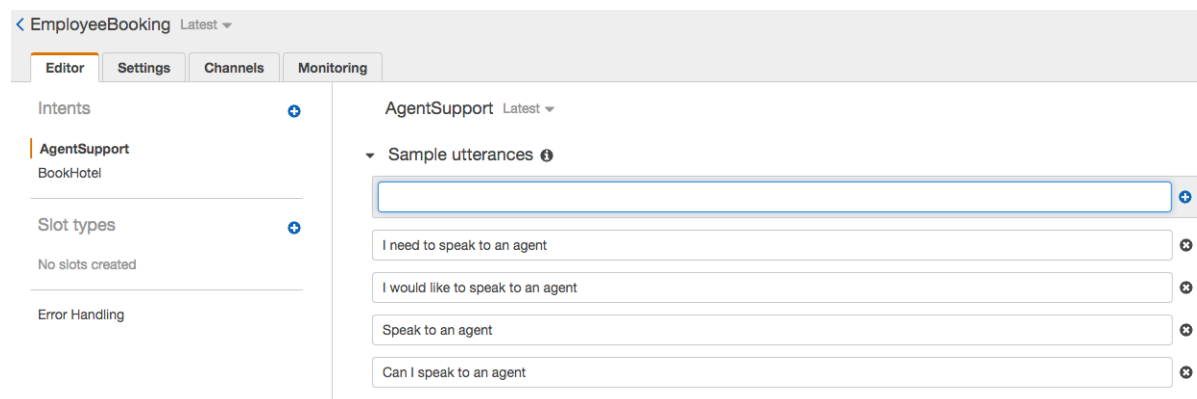
30. The new Intent, **AgentSupport** has been created



A screenshot of the "EmployeeBooking" interface. The top bar shows "EmployeeBooking" and "Latest" with a dropdown arrow. Below the bar are tabs: "Editor" (selected), "Settings", "Channels", and "Monitoring". Under the "Editor" tab, there is a section titled "Intents" with a plus icon. Below this, the "AgentSupport" intent is listed with the sub-label "BookHotel".

31. Next step is to add **Sample utterances**

- "Can I speak to an agent"
- "Speak to an agent"
- "I would like to speak to an agent"
- "I need to speak to an agent"

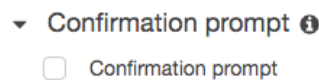


A screenshot of the "EmployeeBooking" interface, specifically the "AgentSupport" intent page. The top bar shows "EmployeeBooking" and "Latest" with a dropdown arrow. Below the bar are tabs: "Editor" (selected), "Settings", "Channels", and "Monitoring". Under the "Editor" tab, there is a section titled "Intents" with a plus icon. Below this, the "AgentSupport" intent is listed with the sub-label "BookHotel". To the right of the "Intents" list, there is a section titled "AgentSupport Latest" with a dropdown arrow. Below this, there is a section titled "Sample utterances" with a plus icon. Below this section, there are four text input fields containing the sample utterances: "I need to speak to an agent", "I would like to speak to an agent", "Speak to an agent", and "Can I speak to an agent". Each field has a plus icon on the right side.

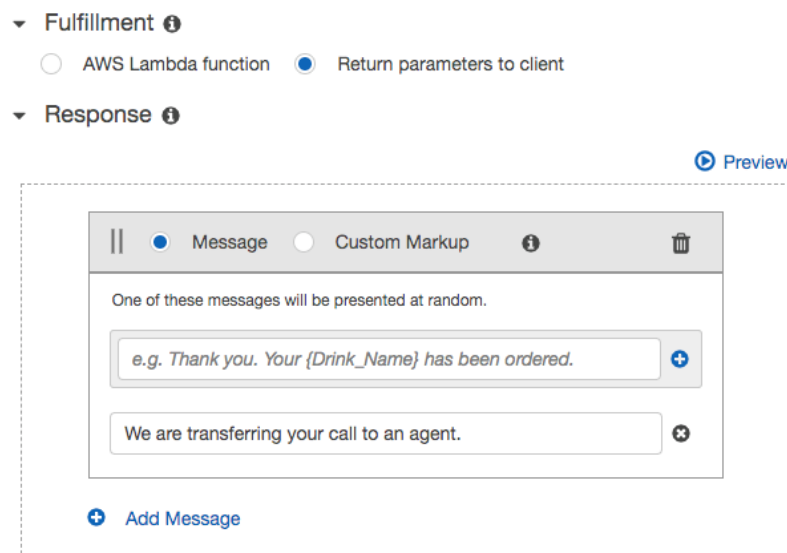
32. For this Intent, we don't need any **Slots** (as we don't need any data the user would need to provide, to fulfill the intent). We can skip the **Slots** section.



33. We can set a **Confirmation prompt**, asking caller to confirm that agent support is needed, but for the purpose of this exercise, we are going to leave it **blank** (un-checked).



34. Set Fulfilment > Return parameters to client > Enter your message ("*We are transferring your call to an agent*")



35. Review the **Error Handling** from the menu on the left-hand side of the screen. **Clarification prompts** are going to be played in case the input was not understood, while **Hang-up phrase** is going to be played as a final message (after **Maximum number of retries** exceeds).

Error handling

Clarification prompts

e.g. Sorry, can you please repeat that? 

Sorry, can you please repeat that? 

Maximum number of retries

5

Hang-up phrase

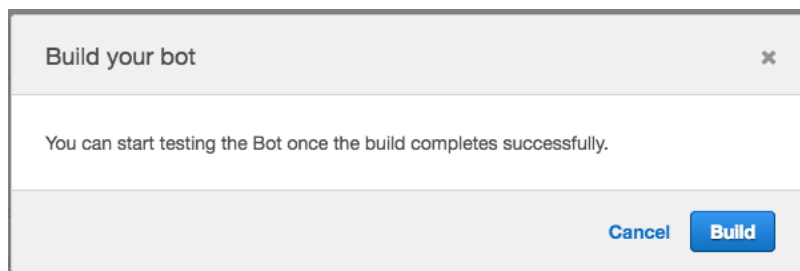
e.g. Sorry, I could not understand. Please contact customer support. 

Sorry, I could not understand. Goodbye. 

Save

36. Click **Save Intent**

37. Click **Build** to build your bot



38. Once the bot is successfully built, you can test it by typing an utterance in the console on the left-hand side. Start by typing "book a hotel"

> Test Bot (Latest) 🟢 READY

book a hotel

What city will you be staying in?

Clear

Chat to your bot...

Inspect Response

Dialog State: ElicitSlot

☒ Summary ☐ Detail

Intent: BookHotel

Slots	(0/3)
CheckInDate	null
Location	null
Nights	null

39. Enter a city (*Location slot*)

> Test Bot (Latest) 🟢 READY

book a hotel

What city will you be staying in?

New York

What day do you want to check in?

Clear

Chat to your bot...

Inspect Response

Dialog State: ElicitSlot

☒ Summary ☐ Detail

Intent: BookHotel

Slots	(1/3)
CheckInDate	null
Location	New York
Nights	null

40. Enter a Check-In Date (*CheckInDate slot*)

> Test Bot (Latest) READY

What city will you be staying in?

New York

What day do you want to check in?

Saturday

How many nights will you be staying?

[Clear chat history](#)

Chat to your bot...

Inspect Response

Dialog State: ElicitSlot

☒ Summary ☐ Detail

Intent: BookHotel

Slots (2/3)

CheckInDate	2018-05-26
Location	New York
Nights	null

41. Enter how many nights you want to stay (*Nights slot*)

> Test Bot (Latest) READY

Saturday

How many nights will you be staying?

3

Okay, I have you down for a 3 night stay in New York starting 2018-05-26. Shall I book the reservation?

[Clear chat history](#)

Chat to your bot...

Inspect Response

Dialog State: ConfirmIntent

☒ Summary ☐ Detail

Intent: BookHotel

Slots (3/3)

CheckInDate	2018-05-26
Location	New York
Nights	3

42. The Lex bot has confirmed your inputs, and it asks you for the confirmation. Enter "yes"

> Test Bot (Latest) ✓ READY

Saturday

How many nights will you be staying?

3

Okay, I have you down for a 3 night stay in New York starting 2018-05-26. Shall I book the reservation?

[Clear chat history](#)

Chat to your bot...

Inspect Response

Dialog State: ConfirmIntent

☒ Summary ☐ Detail

Intent: BookHotel

Slots	(3/3)
CheckInDate	2018-05-26
Location	New York
Nights	3

43. The Lex bot confirms your reservation.

> Test Bot (Latest) ✓ READY

3

Okay, I have you down for a 3 night stay in New York starting 2018-05-26. Shall I book the reservation?

yes

Thanks, I have placed your reservation.

[Clear chat history](#)

Chat to your bot...

Inspect Response

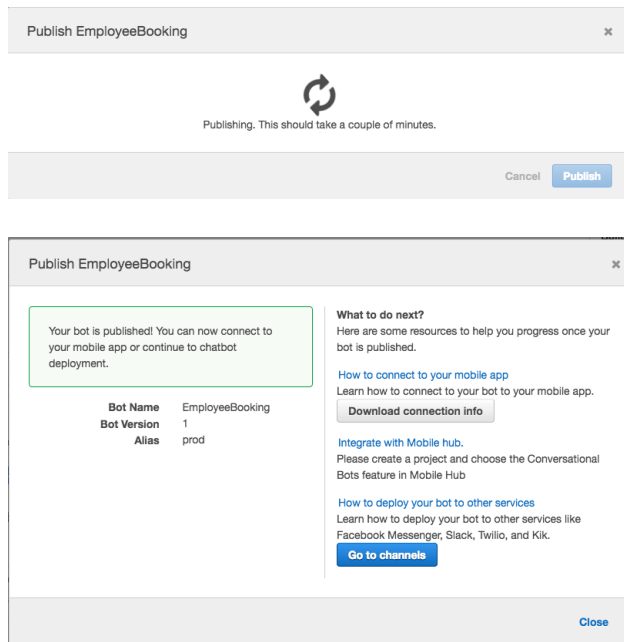
Dialog State: Fulfilled

☒ Summary ☐ Detail

Intent: BookHotel

Slots	(3/3)
CheckInDate	2018-05-26
Location	New York
Nights	3

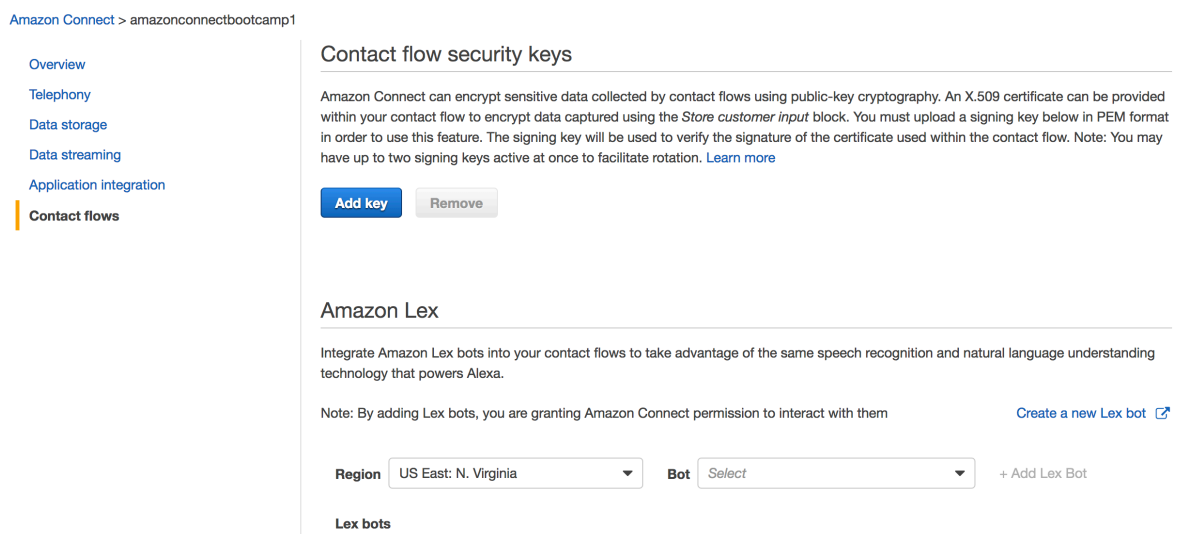
44. Click **Publish**, set an alias as **prod**, click **Publish** again



Integrate Your Lex Bot with Amazon Connect

As a first step, we have to enable our Amazon Lex bot in the Amazon Connect instance.

1. Navigate to Amazon Connect Service > Click on your instance name
2. From the left-hand menu, select **Contact Flows**
3. Select **Region** – in this case, **US East: N. Virginia**
4. Select **Bot** – in this case, **EmployeeBooking**
5. Click **+ Add Lex Bot**



Amazon Lex

Integrate Amazon Lex bots into your contact flows to take advantage of the same speech recognition and natural language understanding technology that powers Alexa.

Note: By adding Lex bots, you are granting Amazon Connect permission to interact with them

[Create a new Lex bot](#) 

Region US East: N. Virginia ▼

Bot Select ▼

+ Add Lex Bot

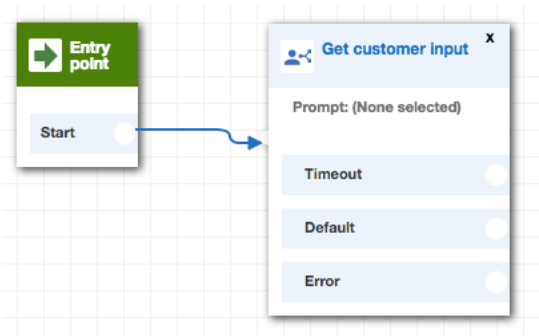
No Lex bots available to add [Create new](#).

Lex bots

EmployeeBooking [Remove](#)

The Amazon Lex bot is now assigned to your Amazon Connect instance. The next step is to design a Contact Flow that will invoke our bot.

1. In your Amazon Connect instance, navigate to Routing > Contact Flows
2. Click **Create Contact Flow**
3. Set Name: **EmployeeBooking_LexBot**
4. Enter description
5. Click **Save**
6. Place new **Get Customer Input** node



7. Set prompt text "How can I help you today?"

Get customer input
x

Configure the branches that a customer can choose based on their intent.

Plays an interruptible audio prompt and branches based on DTMF or Amazon Lex intents.

☐ Select from the prompt library
☒ Text to speech (Ad hoc)
[Learn more about Amazon Connect's TTS capabilities](#)
☒ Enter text

How can I help you today?

☐ Enter dynamically

Interpret as

Text

8. Scroll down, select **Amazon Lex**, set name as **EmployeeBooking**, alias **prod**

Lex bot

Name	Alias
<u>EmployeeBooking</u>	<u>prod</u>

9. Intents > Add an intent > **BookHotel**

×

[Add another intent](#)

Cancel
Save

10. Intents > Add another intent > **AgentSupport**

Intents

×

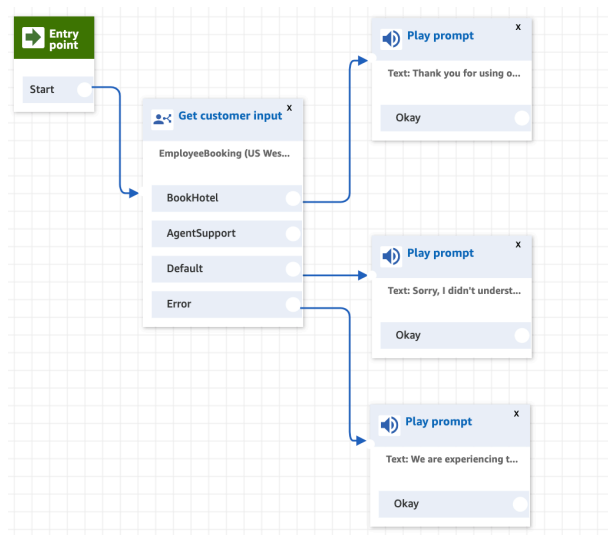
×

[Add another intent](#)

Cancel
Save

11. Click **Save**

12. Place three **Play Prompt** nodes which will play result message once Lex bot interaction is completed.



13. The first prompt will confirm a reservation by the Lex bot

☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☐ Enter text

14. The second prompt plays a Default message (if intent has not been matched):

- ☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

- ☒ Enter text

Sorry, I didn't understand you.

15. The third prompt will be played if an error occurs:

- ☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

- ☒ Enter text

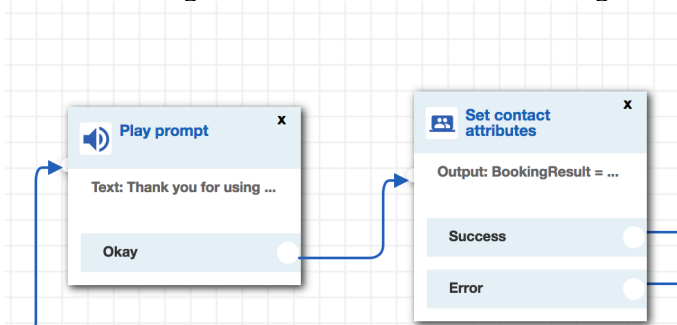
We are experiencing technical difficulties.

- ☐ Enter dynamically

Interpret as

Text

16. If the booking was a success, we are setting the Contact Attribute:



Set contact attributes ✕

Define and store key-value pairs as contact attributes. [Learn more](#)

Contact attributes are accessible by other areas of Amazon Connect, such as the Contact Control Panel (CCP) and Contact Trace Records (CTRs).

Attribute to save

Destination Type
✕

User Defined

Destination Attribute

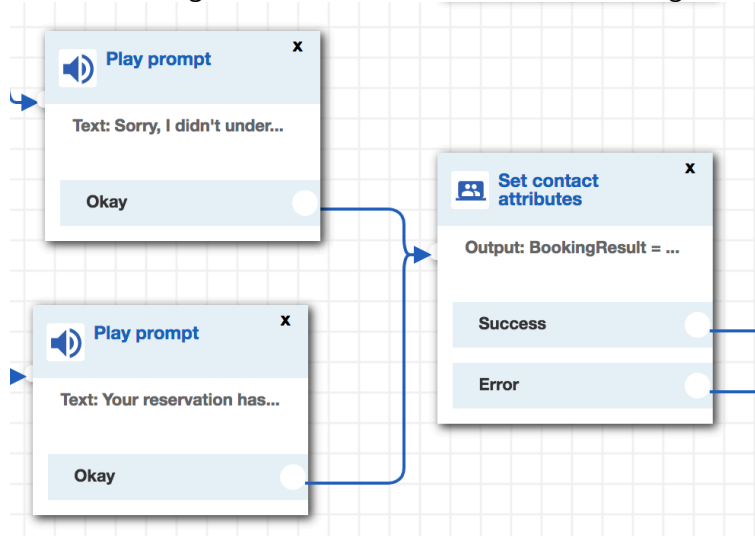
BookingResult

☒ Use text

Value

Success

17. If the booking was not successful, we are setting Contact Attributes:



Set contact attributes ✕

Define and store key-value pairs as contact attributes. [Learn more](#)

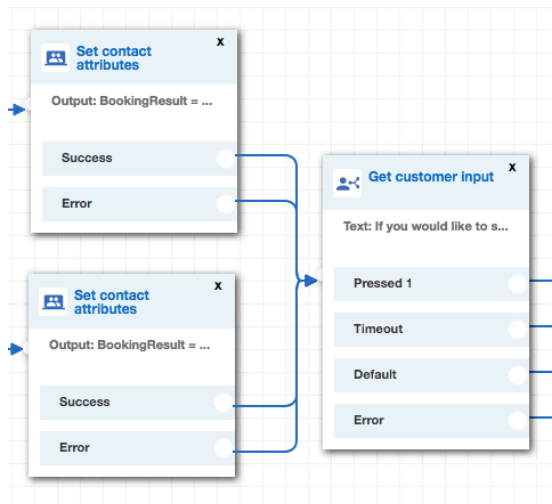
Contact attributes are accessible by other areas of Amazon Connect, such as the Contact Control Panel (CCP) and Contact Trace Records (CTRs).

Attribute to save

Destination Type ✕
User Defined ▼
Destination Attribute
BookingResult
☒ Use text
Value
Unsuccessful

18. Place the **Get Customer Input** node and set prompt as "If you would like to speak to an agent, please press 1." Scroll down and click on "Add another condition" and type 1

Get customer input ✕
Delivers an audio or chat message for a customer response.
The contact flow branches based on the customer input: DTMF, chat, or Amazon Lex. [Learn more](#).
☐ Select from the prompt library (audio)
☒ Text-to-speech or chat text
☒ Enter text
If you would like to speak to an agent, please press 1
☐ Enter dynamically
Interpret as
Text ⬆
DTMF Amazon Lex
Plays an audio prompt and branches based on DTMF or Amazon Lex intents. The audio prompt is interruptible when using DTMF.
Set timeout (Minimum one second)
5 seconds
Option:
x 1
[Add another condition](#)
Cancel Save



19. Add new block - Transfer to flow `EmployeeBooking_TransferToQueue`
Transfer to flow

Ends the current flow and transfers the customer to a flow of type contact flow.

Transfer

Select a flow

`EmployeeBooking_TransferToQueue` x ▾

20. Add new **Play Prompt** block, with error message:

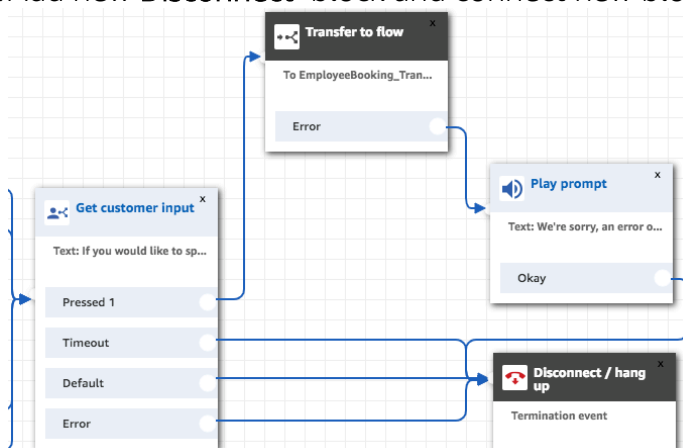
Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

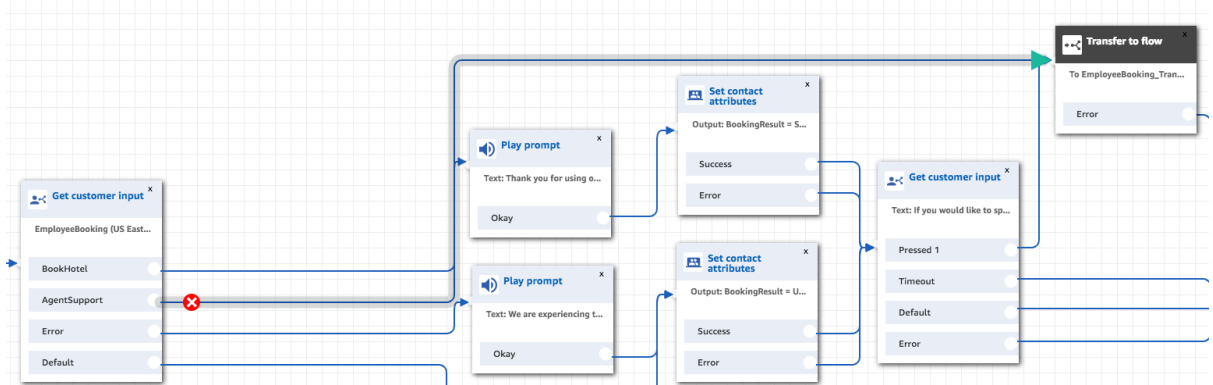
Enter text

We're sorry, an error occurred. Please try again later.

21. Add new **Disconnect** block and connect new blocks accordingly:

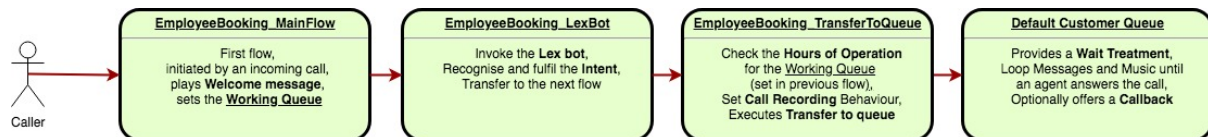


22. Connect the "AgentSupport" output with "Transfer to flow" block input:



23. Save and Publish the flow

As a final step, we have to invoke **EmployeeBooking_LexBot** flow from our existing **EmployeeBooking_MainFlow**. The new sequence is going to look like this:



1. Navigate to Routing > Contact Flows > **EmployeeBooking_MainFlow**
2. Edit the **Transfer to Flow** node by setting **EmployeeBooking_LexBot** as the destination flow:

Transfer to flow

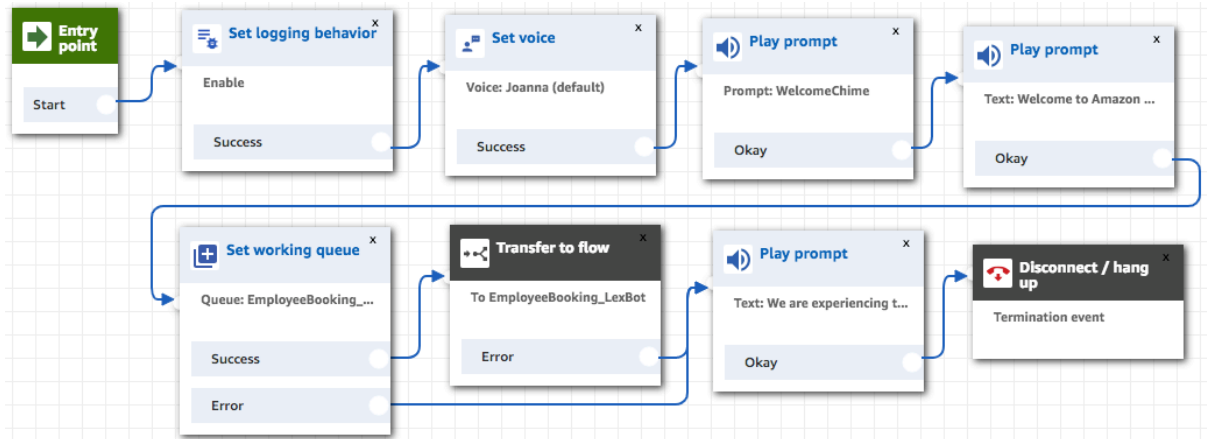
Ends the current flow and transfers the customer to a flow of type contact flow.

Transfer

Select a flow

EmployeeBooking_LexBot x v

3. Save and Publish the flow



Testing it all together

It is time to test our Lex bot:

1. Make a phone call to your Amazon Connect instance
2. The Welcome Message is going to be played
3. Then the Lex bot is going to ask "How can I help you today?"
4. Start by saying "book a hotel"
5. Follow the conversation and book your hotel
6. Make another call and say "speak to an agent", when prompted
7. The call should be transferred to an available agent.

Hands-on Module 3: Amazon Connect Dynamic Contact Flow

Objective: Improve the contact flow created in the previous module, design an Amazon Connect dynamic flow, allow employee authentication, and greet and transfer authenticated employees to designated queue. We will use AWS Lambda and Amazon DynamoDB.

AWS Lambda role and policy for Amazon DynamoDB access

In this module, you will be creating an AWS Lambda function that accesses an Amazon DynamoDB table for employee information. To execute correctly, AWS Lambda will need an Amazon Identity and Access Management (IAM) **role** and **policy** with the appropriate permissions.

1. In your AWS console search bar, type **"IAM"** and select **IAM** from the results
2. In the navigation pane, select **"Policies"** and **"Create Policy"**
3. Choose a service: **DynamoDB**
4. Actions: **GetItem, PutItem, Scan, Query** (use the filter Action box to find them)
5. Resources: **All Resources**

Note: if you need to limit access to certain tables, it's possible to specify a certain resource/table in the policy document

6. Click **Add Additional Permissions**
7. Choose a service: **CloudWatch Logs**
8. Access level: **CreateLogGroup, CreateLogStream, PutLogEvents**
9. Resources: **All Resources**

Note: if you need to limit access to certain resources, it's possible to specify a certain resource/table in the policy document

10. Click **Review Policy**
11. Enter Policy name: **Lambda_EmployeeBooking**
12. Enter a description
13. Click **Create Policy**

Next step is to create a role:

1. In the navigation pane, select **"Roles"** and **"Create Role"**
2. Choose the service that will use this role: **Lambda**
3. Click **Next: Permissions**
4. Select Policy: **Lambda_EmployeeBooking**
5. Click **Next: Tags**
6. At this point you can add custom Tags. More information can be found here: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_tags.html?icmpid=docs_iam_console#id_tags_procs
For the purpose of this lab, we don't need to set any Tags
7. Click **Next: Review**

8. Role name: **Lambda_EmployeeBooking**
9. Click **Create Role**

Creating and configuring the DynamoDB table

Amazon DynamoDB is a fast and flexible NoSQL database service for all applications that need consistent, single-digit millisecond latency at any scale. It is a fully managed cloud database and supports both document and key-value store models. Its flexible data model, reliable performance, and automatic scaling of throughput capacity make it a great fit for storing and referencing caller information from Amazon Connect.

1. In your AWS console search bar, type **"DynamoDB"** and select **DynamoDB** from the results

Note: Ensure that your selected region is "US East (N. Virginia)" by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.

2. Click **Create Table**
3. Table name: **Employee**
4. Primary key: **EmployeeID** (keep String as type)
Please use the correct case, as all parameters are case-sensitive
5. Click **Create**
6. Wait for the table to be created
7. Select **Indexes > Create Index**
8. Partition key: **PhoneNumber**
9. Index name: **PhoneNumber-index**
10. Click **Create Index**
11. Wait for the index to be created (If index is not created within 5 minutes continue to step 12 and check again after step 16)
12. Select **Employee** table > **Items > Create item**
13. In the top-left corner drop-down list, select **Text** (**Tree** is selected by default)
14. Enter the following JSON:

Please use the correct case, as all parameters are case-sensitive

Create item

Text ▾ ☐ DynamoDB JSON

```

1 {
2   "EmployeeID": "101",
3   "PhoneNumber": "+447700111111",
4   "EmployeeName": "John",
5   "EmployeePIN": "111"
6 }
```

15. Enter your mobile phone number, in the E.164 format

Note: PIN has been stored as a plain text. In a regular production system, the PIN should be encrypted.

16. Click **Save**

AWS Lambda Integration

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume - there is no charge when your code is not running. Just upload your code and Lambda takes care of everything required to run and scale your code with high availability. With AWS Lambda and Amazon Connect, you can pass and return custom key-value pairs allowing for multiple integration points within your contact flows. More information about using AWS Lambda with Amazon Connect can be found in the official documentation: <https://docs.aws.amazon.com/connect/latest/adminguide/connect-lambda-functions.html>

1. In your AWS console search bar, type "**Lambda**" and select Lambda from the results

Note: Ensure that your selected region is "US East (N. Virginia)" by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.

2. Select **Create a Function**
3. Select **Author from scratch**
4. Name: **EmployeeBooking_DBLookup**
5. Runtime: **Node.js 14.x**
6. Role: Use an existing role > **Lambda_EmployeeBooking**
7. Select **Create Function**

Function name

Enter a name that describes the purpose of your function.

EmployeeBooking_DBLookup

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime Info

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Node.js 14.x

Permissions Info

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ Change default execution role

Execution role

Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions
 ☒ Use an existing role
 ☐ Create a new role from AWS policy templates

Existing role

Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

Lambda_EmployeeBooking

View the [Lambda_EmployeeBooking](#) role on the IAM console.

► Advanced settings

Cancel

Create function

© 2021, Amazon Web Services, Inc. or its affiliates. All rights reserved.
Amazon Connect Bootcamp Lab Guide v3.7

Page **54** of **123**

8. Copy the code from:
https://s3.amazonaws.com/amazonconnectbootcampresources/v3.0/Module3/EmployeeBooking_DBLookup.js
9. Paste replacing the current `index.js` content

```

1  'use strict';
2
3  const AWS = require('aws-sdk');
4  const dynamodb = new AWS.DynamoDB.DocumentClient();
5
6  exports.handler = (event, context, callback) => {
7
8      console.log('Received event:', JSON.stringify(event, null, 2));
9
10     var PhoneNumber = event.Details.Parameters.PhoneNumber;
11     var EmployeeID = event.Details.Parameters.EmployeeID;
12     var EmployeePIN = event.Details.Parameters.EmployeePIN;
13     var ReturnResult = {};
14
15     EmployeeSearch();
16
17     function EmployeeSearch()
18     {
19         var DDBTable = "Employee";
20
21         if(PhoneNumber)
22         {
23             var params = {
24                 TableName: DDBTable,
25                 KeyConditionExpression: "PhoneNumber = :varNumber",
26                 IndexName: "PhoneNumber-index",
27                 ExpressionAttributeValues:
28                 {
29                     ":varNumber": PhoneNumber
30                 }
31             };
32
33         }
34     }
35 }
  
```

10. Go to the "Configuration -> General Configuration" then increase the Timeout to 7 seconds.
11. Click Deploy (top right of page)

This Lambda function is going to be used for employee authentication. Please examine the code and notice the following:

1. It uses the AWS SDK and DynamoDB Document Client

```

3  const AWS = require('aws-sdk');
4  const dynamodb = new AWS.DynamoDB.DocumentClient();
5
  
```

2. As a first step, for debugging purposes, it logs the event payload to the CloudWatch Logs

```

9      console.log('Received event:', JSON.stringify(event, null, 2));
10
  
```

3. During the authentication process, Amazon Connect will be sending parameters to this Lambda function. Parameters are stored in the local variables, `ReturnResult` is the object that will be returned back to Contact Flow and `EmployeeSearch()` function is invoked.

```

11     var PhoneNumber = event.Details.Parameters.PhoneNumber;
12     var EmployeeID = event.Details.Parameters.EmployeeID;
13     var EmployeePIN = event.Details.Parameters.EmployeePIN;
14     var ReturnResult = {};
15
16     EmployeeSearch();
  
```

4. This Lambda function is going to cover both scenarios from our use-case:

- a. As a first step, we have to recognize the caller, by phone number. In that case, only the phone number will be passed as a parameter to our Lambda function. If the caller is recognized, Contact Flow will ask the caller to enter their PIN, and in this case the same Lambda function will be invoked, passing PIN as an additional parameter. If the PIN matches, Lambda will return the result: "Authenticated"
 - b. If the caller is not recognized by phone number, the Contact Flow is going to ask for the Customer ID and PIN, then invoke the same Lambda function, passing EmployeeID and PIN as parameters
5. Based on our scenario, the **EmployeeSearch** function sets DynamoDB table name, then if the PhoneNumber has been sent as an input parameter, it sets parameters for querying the table by PhoneNumber:

```

19     function EmployeeSearch()
20     {
21         var DDBTable = "Employee";
22
23         if(PhoneNumber)
24         {
25             var params = {
26                 TableName: DDBTable,
27                 KeyConditionExpression: "PhoneNumber = :varNumber",
28                 IndexName: "PhoneNumber-index",
29                 ExpressionAttributeValues:
30                 {
31                     ":varNumber": PhoneNumber
32                 }
33             };
34
35             console.log("Searching by PhoneNumber: " + JSON.stringify(params));
36             QueryDynamoDB(params);
37         }
38     }

```

6. If PhoneNumber input parameter has not been sent, the function checks if the EmployeeID parameter has been sent, and if that's the case, it sets parameters for querying the table by EmployeeID:

```

39         else if(EmployeeID)
40         {
41             var params = {
42                 TableName: DDBTable,
43                 KeyConditionExpression: "EmployeeID = :varNumber",
44                 ExpressionAttributeValues:
45                 {
46                     ":varNumber": EmployeeID
47                 }
48             };
49             console.log("Searching by EmployeeID: " + JSON.stringify(params));
50             QueryDynamoDB(params);
51         }
52     }

```

7. If both parameters (PhoneNumber and EmployeeID) are missing, the function will return "Unknown" as the result.

```

53     else
54     {
55         setUnknown();
56         invokeCallbackFunction(ReturnResult);
57     }

```

8. QueryDynamoDB function uses AWS SDK and DynamoDB Document Client to search for an employee in our DynamoDB table:

```

61 function QueryDynamoDB(params)
62 {
63     dynamodb.query(params, function(err, data) {
64         if (err) {
65             console.error("Unable to read item. Error JSON:", JSON.stringify(err, null, 2));
66         } else {
67             console.log("GetItem succeeded:", JSON.stringify(data, null, 2));
68             if(data.Items.length == 1)
69             {
70                 ReturnResult.EmployeeID = data.Items[0].EmployeeID;
71                 ReturnResult.EmployeeName = data.Items[0].EmployeeName;
72                 ReturnResult.EmployeePIN = data.Items[0].EmployeePIN;
73             }
74         }
75     }
76     EmployeeSearchCB();
77 });
78 }
79 --

```

9. If the employee has been found, the functions will assign the employee's details to *ReturnResult* object, so these can be consumed by an Amazon Connect Contact Flow. Please note that the DynamoDB Query is an asynchronous operation, hence once the operation is completed, the **EmployeeSearchCB** function is invoked.
10. The **EmployeeSearchCB** function processes the result from DynamoDB and prepares the response to be sent back to Contact Flow:

```
81 function EmployeeSearchCB() {  
82     if(!ReturnResult.EmployeeID)  
83     {  
84         setUnknown();  
85     }  
86     else  
87     {  
88         if(ReturnResult.EmployeePIN && ReturnResult.EmployeePIN === EmployeePIN)  
89         {  
90             ReturnResult.EmployeeType = 1; //Authenticated  
91         }  
92         else  
93         {  
94             if(EmployeePIN)  
95             {  
96                 ReturnResult.EmployeeType = 3; //PIN is wrong  
97             }  
98             else  
99             {  
100                 ReturnResult.EmployeeType = 2; //Ask for PIN  
101             }  
102         }  
103     }  
104     invokeCallbackFunction(ReturnResult);  
105 }
```

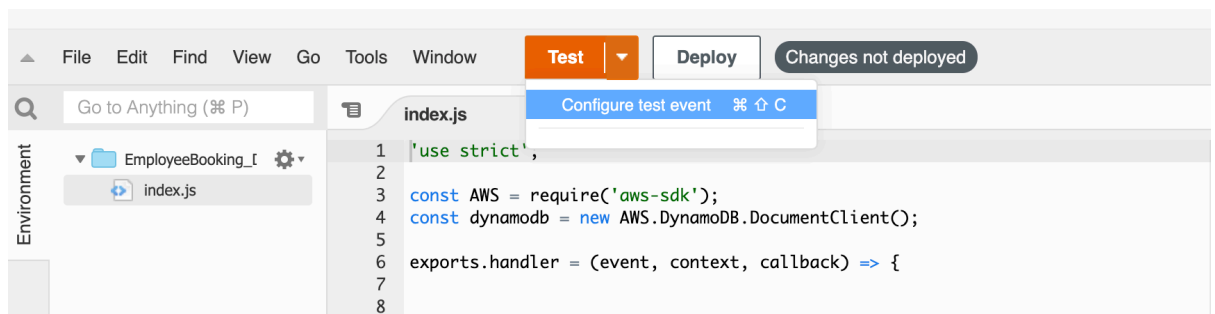
The **SetUnknown** function prepares the response for an unknown caller, which will be sent back to the Contact Flow:

```
107 function setUnknown()
108 {
109     console.log("setUnknown");
110     ReturnResult.EmployeeType = 0; //Unknown Employee
111     ReturnResult.EmployeeName = 'Unknown';
112     ReturnResult.EmployeePIN = null;
113 }
114
```

11. The **InvokeCallbackFunction** passes the result back to an Amazon Connect Contact Flow:

```
115 function invokeCallbackFunction(ResultObj)
116 {
117     ReturnResult.EmployeePIN = null;
118     console.log("invokeCallbackFunction: " + JSON.stringify(ResultObj));
119     callback(null, ResultObj);
120 }
121
122 };
```

12. It is strongly recommended that you test your Lambda function before you continue with the Amazon Connect integration. Click on **Configure Test Events**



13. To test our Lambda function, we are going to create 5 test events. First test event is going to cover the "Unknown" phone number scenario:

Event name

UnknownPhoneNumber

```
1 {
2   "Details": {
3     "ContactData": {},
4     "Parameters": {
5       "PhoneNumber": "+44123123123"
6     }
7   },
8   "Name": "ContactFlowEvent"
9 }
```

```
{
  "Details": {
    "ContactData": {},
    "Parameters": {
      "PhoneNumber": "+441231213123"
    }
  },
  "Name": "ContactFlowEvent"
}
```

14. Click **Create** and then click **Test**

Test event
Delete
Format
Save changes
Test

15. The Lambda function is going to return the "Unknown" employee:

Execution result: succeeded (logs)

Details

The area below shows the result returned by your function execution. [Learn more](#) about returning results from your function.

```
{
  "EmployeeType": 0,
  "EmployeeName": "Unknown",
  "EmployeePIN": null
}
```

Summary

Code SHA-256	Request ID
9UDQHjtY/Splq0okwlyJk5mvjqNTGg/kNta4X4AfrWo=	11ababfd-65f5-4b81-9b6a-44b543189d65
Duration	Billed duration
76.72 ms	77 ms
Resources configured	Max memory used
128 MB	91 MB

16. The next event is going to be with a PhoneNumber for an existing employee:

Event name
ExistingPhoneNumber

```
1 {
2   "Details": {
3     "ContactData": {},
4     "Parameters": {
5       "PhoneNumber": "+4477 [redacted]"
6     }
7   },
8   "Name": "ContactFlowEvent"
9 }
```

The json code is the same but with a number that exists on the DynamoDB table.

17. The Lambda function is going to return EmployeeID and EmployeeName, but EmployeeType = 2 (which means that employee has to enter their PIN):

EmployeeBooking_DBLookup Qualifiers ▾ Actions ▾ ExistingPhoneNumber ▾ Test Save

✓ Execution result: succeeded (logs) ✕

▼ Details

The area below shows the result returned by your function execution. [Learn more](#) about returning results from your function.

```
{
  "EmployeeID": "101",
  "EmployeeName": "John",
  "EmployeePIN": null,
  "EmployeeType": 2
}
```

Summary

Code SHA-256	IT17Nop80BChXN/HskDJZQtVN5dtCysuj3sNJTQ+xiY=	Request ID	b5b63768-206a-11e8-9bee-516103ad1464
Duration	339.76 ms	Billed duration	400 ms
Resources configured	128 MB	Max memory used	40 MB

18. In the next event, we are going to send EmployeePIN, so that we can complete the process of authentication.

Event name

ExistingPhoneNumberPIN

```
1 {
2   "Details": {
3     "ContactData": {},
4     "Parameters": {
5       "PhoneNumber": "+4477 [REDACTED]",
6       "EmployeePIN": "111"
7     }
8   },
9   "Name": "ContactFlowEvent"
10 }
```

19. The Lambda function is going to return EmployeeID and EmployeeName, and EmployeeType = 1 (which means that employee has been authenticated):

EmployeeBooking_DBLookup Qualifiers ▾ Actions ▾ ExistingPhoneNumber... ▾ Test Save

✓ Execution result: succeeded (logs) ✕

▼ Details

The area below shows the result returned by your function execution. [Learn more](#) about returning results from your function.

```
{
  "EmployeeID": "101",
  "EmployeeName": "John",
  "EmployeePIN": null,
  "EmployeeType": 1
}
```

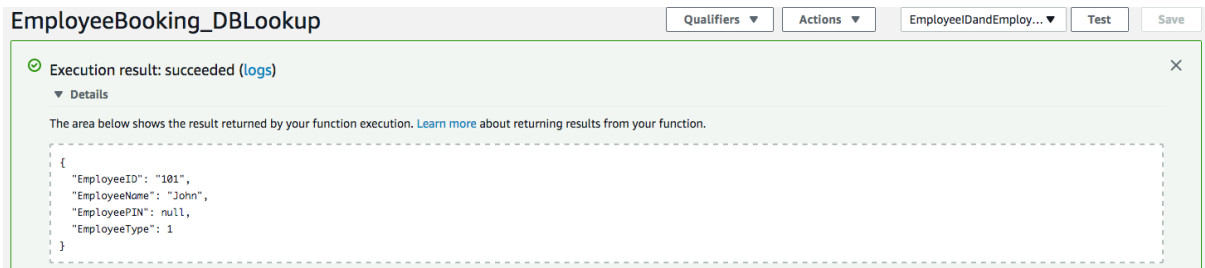
20. In this event, we are going to send EmployeeID and EmployeePIN (to cover the scenario when an employee is calling from an unknown phone number):

Saved Test Event

EmployeeIDandEmployeePIN ▼ ↺

```
1 {
2   "Details": {
3     "ContactData": {},
4     "Parameters": {
5       "EmployeeID": "101",
6       "EmployeePIN": "111"
7     }
8   },
9   "Name": "ContactFlowEvent"
10 }
```

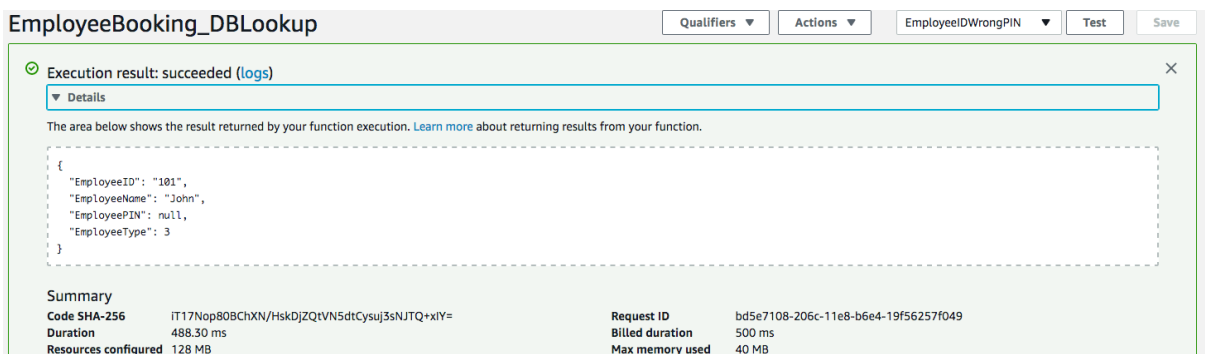
21. The Lambda function is going to return EmployeeID and EmployeeName, and EmployeeType = 1 (which means that employee has been authenticated):



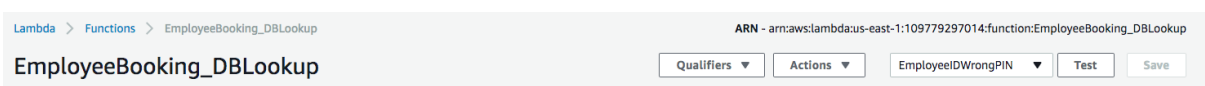
22. The final event is going to send EmployeeID, but wrong EmployeePIN:



23. The Lambda function is going to return EmployeeID and EmployeeName, but EmployeeType = 3 (which means that employee has entered the wrong PIN):



24. As a final step, please note the Function Name and Function ARN:



Giving Amazon Connect permission to invoke your Lambda function

When invoking a Lambda function from Amazon Connect Contact Flow, you need to give Amazon Connect permission to execute your AWS Lambda functions.

1. Navigate to Amazon Connect Service on the AWS console > Click on your instance name
2. From the left-hand menu, select **Contact Flows**
3. Select **Region** – in this case, **US East: N. Virginia**
4. In the AWS Lambda section, select the Lambda function name – in this case, **EmployeeBooking_DBLookup**

AWS Lambda

Amazon Connect can interact with your own systems and take different paths in IVR dynamically. To achieve this, invoke AWS Lambda functions in contact flows to interact with your own systems or other services, then build personalized and dynamic experiences based on data returned.

Note: By adding Lambda functions, you are granting Amazon Connect permission to invoke them [Create a new Lambda function](#)

Function

EmployeeBooking_DBLookup

+ Add Lambda Function

Lambda Functions

5. Click + Add Lambda Function

AWS Lambda

Amazon Connect can interact with your own systems and take different paths in IVR dynamically. To achieve this, invoke AWS Lambda functions in contact flows to interact with your own systems or other services, then build personalized and dynamic experiences based on data returned.

Note: By adding Lambda functions, you are granting Amazon Connect permission to invoke them [Create a new Lambda function](#)

Function

Select

+ Add Lambda Function

Lambda Functions

EmployeeBooking_DBLookup

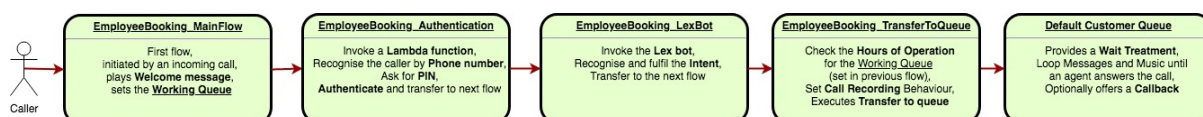
Remove

6. From this moment, our Amazon Connect instance has the permission to execute the **EmployeeBooking_DBLookup** Lambda function. In the next step, we are going to design a contact flow, which utilizes this Lambda function to recognise the caller.

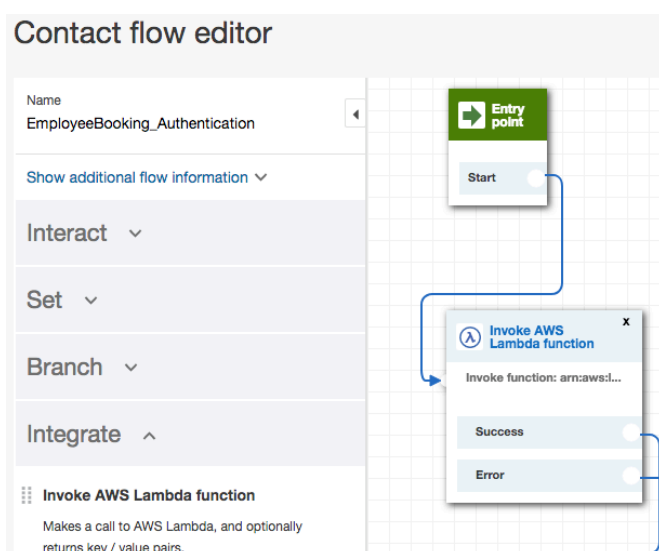
Employee Authentication Contact Flow

In this section, we are going to implement a Contact Flow that will guide employees through the authentication process.

1. Download the Contact Flow template from:
https://s3.amazonaws.com/amazonconnectbootcampresources/v3.0/Module3/EmployeeBooking_Authentication
2. Navigate to your Amazon Connect Instance > Routing > Contact Flows
3. Select **Create Contact Flow**
4. Click on the drop-down list next to the blue Save button and select **Import Flow**
5. Select the download Contact Flow Template (**EmployeeBooking_Authentication**)
6. Select **Import**
7. Check Contact Flow **Name** and **Description**, then click **Save**
8. Our existing Contact Flow **EmployeeBooking_MainFlow** is going to set basic configuration, play *WelcomeChime* and then transfer the call to the **EmployeeBooking_Authentication** Contact Flow. The modification of **EmployeeBooking_MainFlow** is described later in this section. The new flow sequence is going to look like this:



9. The first step in the **EmployeeBooking_Authentication** Contact Flow is to invoke our Lambda function, and we are using Integration > **Invoke AWS Lambda function** node from the palette.



10. In the node configuration, we have to set the Lambda ARN and the parameters that we want to send to the Lambda function. In this case, we are sending the System > Customer Number as input parameter. The timeout for the function is set to 4 seconds.

*Note: You will need to update the ARN field with your own Lambda ARN. There are 3 "Invoke AWS Lambda function" nodes that need to be changed to your Lambda function within the **EmployeeBooking_Authentication** contact flow.*

Invoke AWS Lambda function
✕

Makes a call to AWS Lambda, and optionally returns key / value pairs.

The returned key value pairs can be used to set contact attributes.

Function ARN

☒ Select a function

EmployeeBooking_DBLookup ▾

☐ Use attributes

Function input parameters

Use text
✕

☒ Use attribute

Destination key

PhoneNumber

Type

System ▾

Attribute

Customer Number ▾

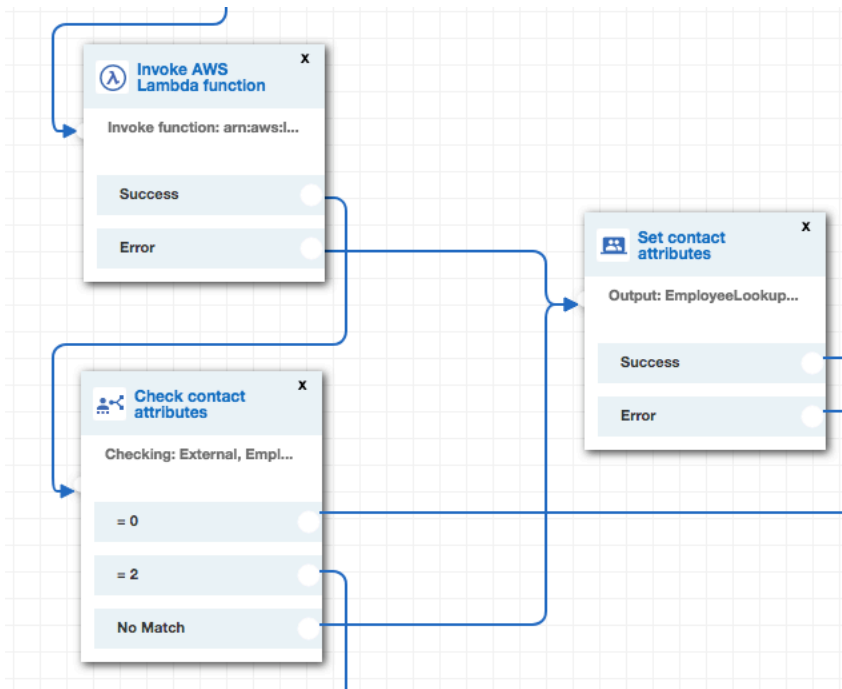
[Add another parameter](#)

Timeout (max 8 seconds)

4

Cancel
Save

11. The Lambda function can either be a success, or it can return an error, therefore we have to process the outcome.



12. If the Lambda function fails, we are going to set a Contact Attribute, which can be used later in the flow, displayed in agent application or stored in Contact Trace Records. In this case, we want to track what happened with the authentication process, hence setting **EmployeeLookup=PhoneNumber-Failed** (which means that we tried to find the employee by phone number, but there was an error)

Set contact attributes ^

Stores key / value pairs as contact attributes.

Contact attributes are accessible by other areas of Amazon Connect, such as the Contact Control Panel (CCP) and Contact Trace Records (CTRs).

Attribute to save

☒ Use text

Destination key
EmployeeLookup

Value
PhoneNumber-Failed

13. If Lambda was successful, we have to check the result returned by Lambda. In our case, we want the attribute **External > EmployeeType**. Expecting results:
- Unknown phone number (EmployeeType = 0) or
 - Existing Phone Number, but the employee has to enter EmployeePIN (EmployeeType = 2).

If we get anything else (EmployeeType not 0 nor 2), we are taking the fail path from point number 12.

Check contact attributes

Branches based on a comparison to the value of a contact attribute.

Examples of supported comparisons include: "equal to", "greater than", "less than", "starts with", "ends with", "contains".

Attribute to check

Type

External

Attribute

EmployeeType

Conditions to check

x	Equals	0
x	Equals	2

No Match

14. If our Lambda function returns Unknown (*EmployeeType = 0*), we are going to set the Contact Attribute, and move to the next step in our scenario – ask the customer for their EmployeeID (see point number 24).

Set contact attributes

Stores key / value pairs as contact attributes.

Contact attributes are accessible by other areas of Amazon Connect, such as the Contact Control Panel (CCP) and Contact Trace Records (CTRs).

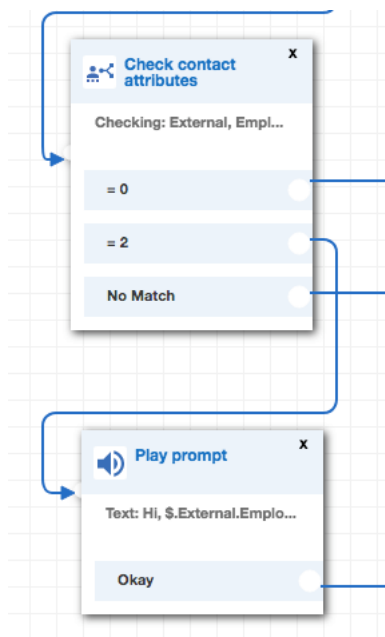
Attribute to save

☒ Use text

Destination key
 EmployeeLookup

Value
 PhoneNumber-Unknown

15. If our Lambda function returns "2" – Ask for PIN, we are going to play the Welcome message first, greeting the employee by their name.



Plays audio.

Prompts can be an audio file, stored in the prompt library, or text-to-speech, which can optionally be specified in a flow via a contact attribute.

Prompt

☐ Select from the prompt library

☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

Hi, \$.External.EmployeeName . Welcome to Amazon Connect Employee Booking service.

☐ Enter dynamically

Interpret as

Text

Note: We are using the Amazon Polly text-to-speech engine, and simply providing the employee's name as an external attribute, received from our Lambda function (\$.External.EmployeeName).

16. In the next step, we are asking for their PIN, by using **Store Customer Input** node:

Store customer input

Stores numerical input to contact attribute.

Plays an interruptible audio prompt and stores digits via DTMF as a contact attribute.

Prompt

☐ Select from the prompt library
 ☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

Please enter your PIN:

☐ Enter dynamically

Interpret as

Text

Prompt

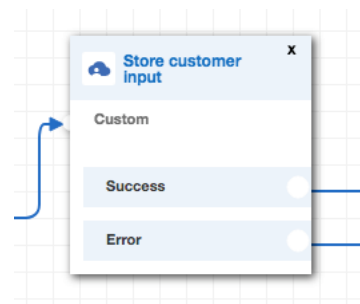
☒ Custom

Maximum Digits

3

Delay between entry

5



17. Using the stored customer input, we are going to invoke our Lambda function once again, trying to authenticate our caller. In this case, we are passing 2 parameters: Customer Number (as **PhoneNumber**) and System> **Stored Customer Input** (as **EmployeePIN**).

Note: You will need to update the ARN field with your own Lambda ARN.

Invoke AWS Lambda function

Makes a call to AWS Lambda, and optionally returns key / value pairs.

The returned key value pairs can be used to set contact attributes.

Function ARN

☒ Select a function

EmployeeBooking_DBLookup

☐ Use attributes

Function input parameters

☐ Use text
 ☒ Use attribute

Destination key

PhoneKey

Type

System

Attribute

Customer Number

☐ Use text
 ☒ Use attribute

Destination key

EmployeePIN

Type

System

Attribute

Stored customer input

Add another parameter

Timeout (max 8 seconds)

3

Cancel

Save

18. Next step is to check the result of our Lambda function. If it fails, we are going to set Contact Attribute **EmployeeLookup=EmployeePIN-Error**, which will help us to trace errors with Lambda function. We are expecting to get either "1" (Employee Authenticated) or "3" (Wrong PIN). If we don't get any of these, we are taking the fail path with setting the **EmployeeLookup=EmployeePIN-Error**.

Check contact attributes

Checking: External, Empl...

= 1

= 3

No Match

Attribute to check

Type

External

Attribute

EmployeeType

Conditions to check

x	Equals	1
x	Equals	3
No Match		

19. If the employee has been successfully authenticated (EmployeeType=1), we are setting the Contact Attributes (EmployeeLookup, EmployeeID and EmployeeName) and playing the Successful Authentication message.

Attribute to save

Use text
Destination key
EmployeeLookup
Value
PhoneNumber

Use attribute

Use text

Use attribute
Destination key
EmployeeID
Type
External
Attribute
EmployeeID

Use text

Use attribute
Destination key
EmployeeName
Type
External
Attribute
EmployeeName

Play prompt

Plays audio.

Prompts can be an audio file, stored in the prompt library, or text-to-speech, which can optionally be specified in a flow via a contact attribute.

Prompt

Select from the prompt library

Text to speech (Ad hoc)
Learn more about Amazon Connect's TTS capabilities
Enter text
Thank you. You have successfully authenticated.

Enter dynamically

Interpret as
Text

20. In the next step, since this is a successfully authenticated employee, we want to **Set working queue** as **EmployeeBooking_Autenticated**.

Set working queue

Specifies the queue to be used when Transfer to queue is invoked.

A queue must be specified prior to invoking Transfer to queue. It's also the default queue for checking attributes, such as: staffing, queue status, and hours of operation.

Outputs:

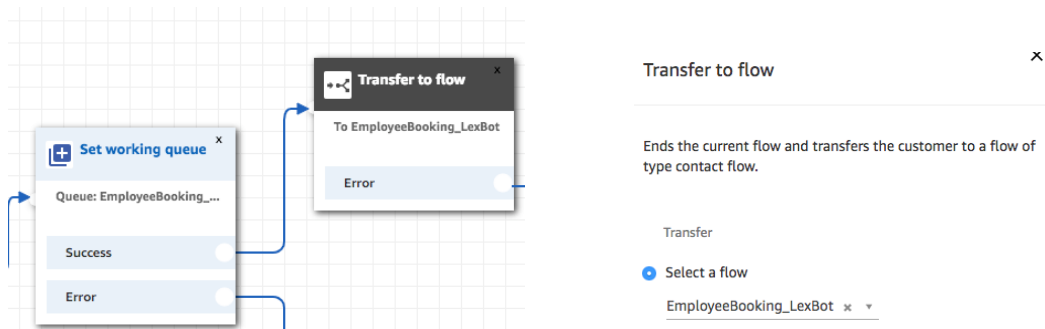
By queue

Select a queue
EmployeeBooking_Authenticated

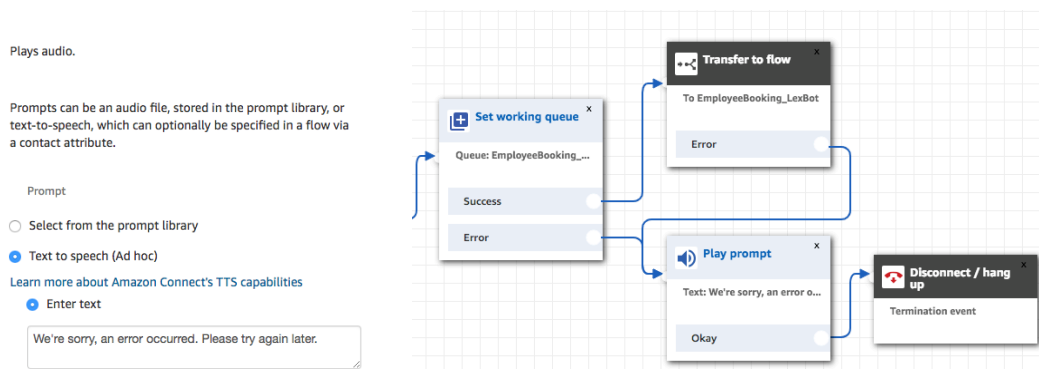
Play prompt
Text: Thank you. You have ...
Okay

Set working queue
Queue: EmployeeBooking_...
Success
Error

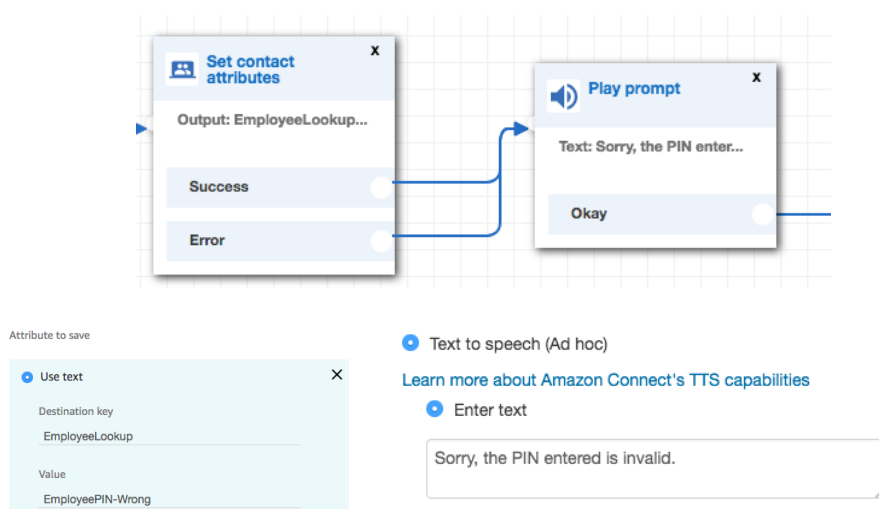
21. If the **Set working queue** was successful, we are transferring the call to **EmployeeBooking_LexBot** flow, using **Transfer to flow** node:



22. If there was an Error, we are disconnecting the call, after a message:



23. However, if the caller entered a wrong PIN, our Lambda function is going to return `EmployeeType=3`, hence we are going to set the Contact Attribute ***EmployeeLookup=EmployeePIN-Wrong***, then play a message that PIN was invalid. In a production environment, we would normally ask the caller to re-enter their PIN, however, for the purpose of this lab, we are offering a transfer to an agent.



24. Going back to the branch where the employee was not recognized by their phone number, we are going to play a Welcome message, and ask the employee to enter their ***EmployeeID***.

Get customer input

Configure the branches that a customer can choose based on their intent.

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

Welcome to Amazon Connect Employee Booking service.
We are not able to recognize your phone number. If you

☐ Enter dynamically

Interpret as

Text

DTMF

Amazon Lex

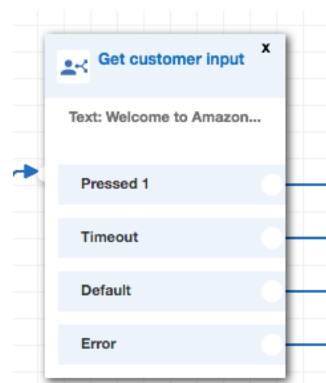
Configure the branches that a customer can choose based on their intent.

Set timeout (Minimum one second)

3 seconds

Option:

x 1



25. If the caller press number 1, we are going to ask for their EmployeeID and EmployeePIN:

Store customer input

Stores numerical input to contact attribute.

Plays an interruptible audio prompt and stores digits via DTMF as a contact attribute.

Prompt

☐ Select from the prompt library

☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

Please enter your Employee ID.

☐ Enter dynamically

Interpret as

Text

Prompt

☒ Custom

Maximum Digits

3

Delay between entry

5

Store customer input

Stores numerical input to contact attribute.

Plays an interruptible audio prompt and stores digits via DTMF as a contact attribute.

Prompt

☐ Select from the prompt library

☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

Please enter your PIN:

☐ Enter dynamically

Interpret as

Text

Prompt

☒ Custom

Maximum Digits

3

Delay between entry

5

26. We have stored our first customer input (EmployeeID) in Contact Attribute CI_EmployeeID, so that we can get another customer input (EmployeePIN) and then send both inputs to our Lambda function.

Attribute to save

☐ Use text
 ☒ Use attribute

Destination key
 Cl_EmployeeID

Type
 System

Attribute
 Stored customer input

27. Having both parameters collected from the caller, we are invoking our Lambda function, passing them as parameters.

Note: You have to update the ARN field with your own Lambda ARN.

Invoke AWS Lambda function
 ×

Makes a call to AWS Lambda, and optionally returns key / value pairs.

The returned key value pairs can be used to set contact attributes.

Function ARN

☒ Select a function
 EmployeeBooking_DBLookup

☐ Use attributes

Function input parameters

☐ Use text
 ☒ Use attribute

Destination key
 EmployeeID

Type
 User Defined

Attribute
 Cl_EmployeeID

☐ Use text
 ☒ Use attribute

Destination key
 EmployeePIN

Type
 System

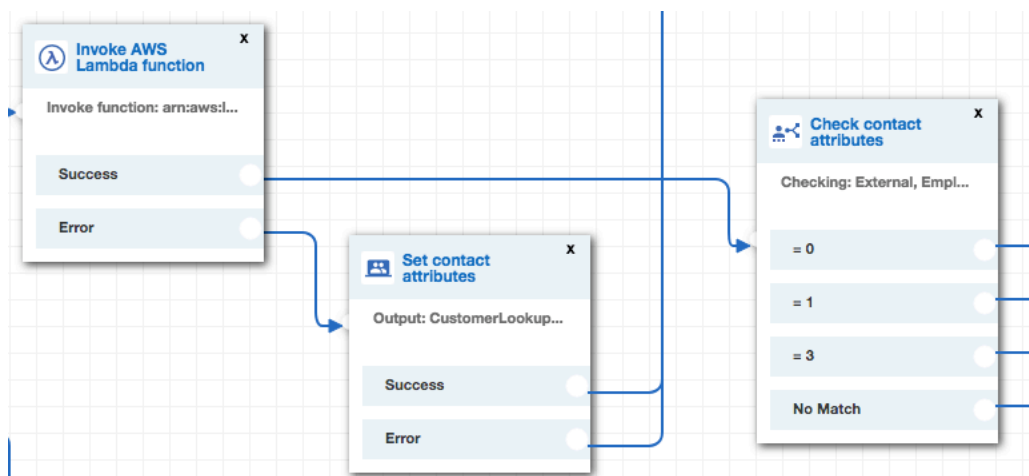
Attribute
 Stored customer input

[Add another parameter](#)

Timeout (max 8 seconds)
 4

Cancel Save

28. Similar to previous steps, we have to check the result of our Lambda function.



29. If our Lambda function returned **CustomerType=0**, our EmployeeID is not recognized. **CustomerType=1** indicates that employee has been authenticated and we can transfer it to our **EmployeeBooking_Authenticated** queue. **CustomerType=3** indicates that caller entered the wrong PIN.

Note: In a production flow, we would ask the caller to re-enter EmployeeID and EmployeePIN, however for this lab, we are simply offering a transfer to agents.

30. If the caller has been successfully authenticated, we are going to greet by Employee Name in the next Play Prompt node, and also set Contact Attributes **CustomerLookup=EmployeeID**, so that we can trace that caller has been

authenticated by entering EmployeeID. At the same time, we also set **EmployeeName**, so that we can play it to the agent.

Plays audio.

Prompts can be an audio file, stored in the prompt library, or text-to-speech, which can optionally be specified in a flow via a contact attribute.

Prompt

☐ Select from the prompt library
☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

Hi, \$.External.EmployeeName . You have successfully authenticated.

Attribute to save

☒ Use text

Destination key
EmployeeLookup

Value
PhoneNumber

☐ Use attribute

☐ Use text

☒ Use attribute

Destination key
EmployeeID

Type
External

Attribute
EmployeeID

☐ Use text

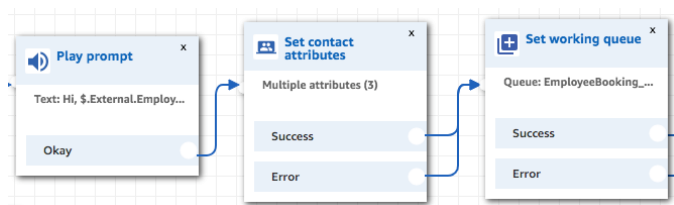
☒ Use attribute

Destination key
EmployeeName

Type
External

Attribute
EmployeeName

31. In the next step, since this is a successfully authenticated employee, we want to **Set working queue** as **EmployeeBooking_Autenticated**.



Set working queue

Specifies the queue to be used when Transfer to queue is invoked.

A queue must be specified prior to invoking Transfer to queue. It's also the default queue for checking attributes, such as: staffing, queue status, and hours of operation.

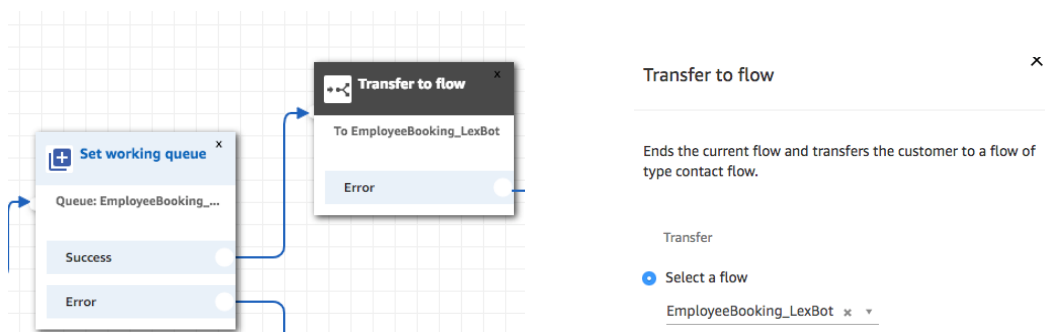
Outputs:

• By queue

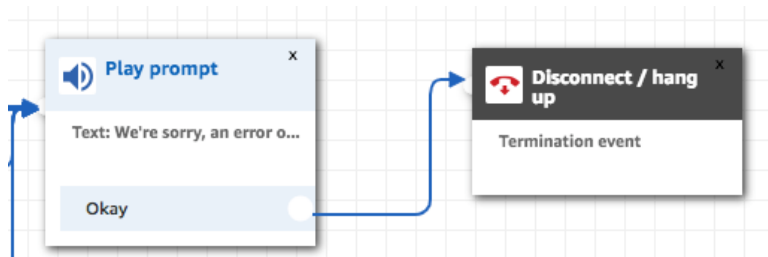
• Select a queue

EmployeeBooking_Authenticated

32. If the **Set working queue** was successful, we are transferring the call to **EmployeeBooking_LexBot** flow, using **Transfer to flow** node:

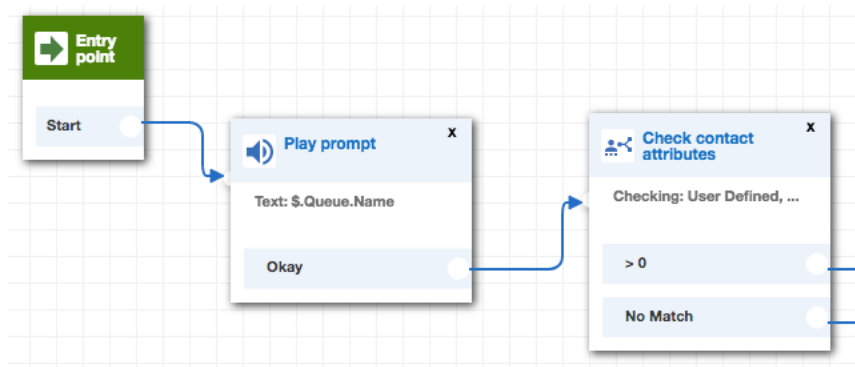


33. Otherwise, we are disconnecting the call, after an error message



According to our use-case, the agent has to greet by the employee's name. For that purpose, we are going to create a custom Agent Whisper Flow.

1. Navigate to Routing > Contact Flows > **Default Agent Whisper**
2. Click on the blue drop-down arrow, next to the Save button
3. Select **Save As**
4. New name: **EmployeeBooking_AgentWhisper**
5. Set Description
6. Click **Save as**
7. Keep existing **Play prompt** node that plays the queue name
8. Add new **Check contact attributes** node and check if the employee is authenticated:



Attribute to check

Type

User Defined

Attribute

EmployeeID

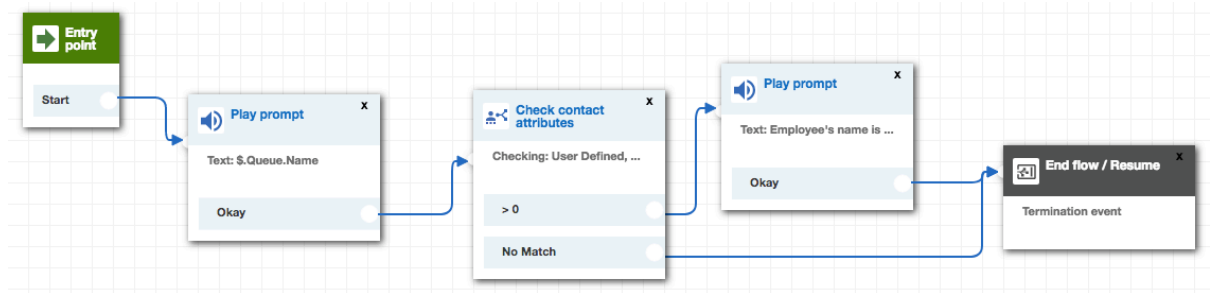
Conditions to check

x Is greater than

0

No Match

9. Add new **Play Prompt** node to play Employee's name



Play prompt

Plays audio.

Prompts can be an audio file, stored in the prompt library, or text-to-speech, which can optionally be specified in a flow via a contact attribute.

Prompt

☐ Select from the prompt library

☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

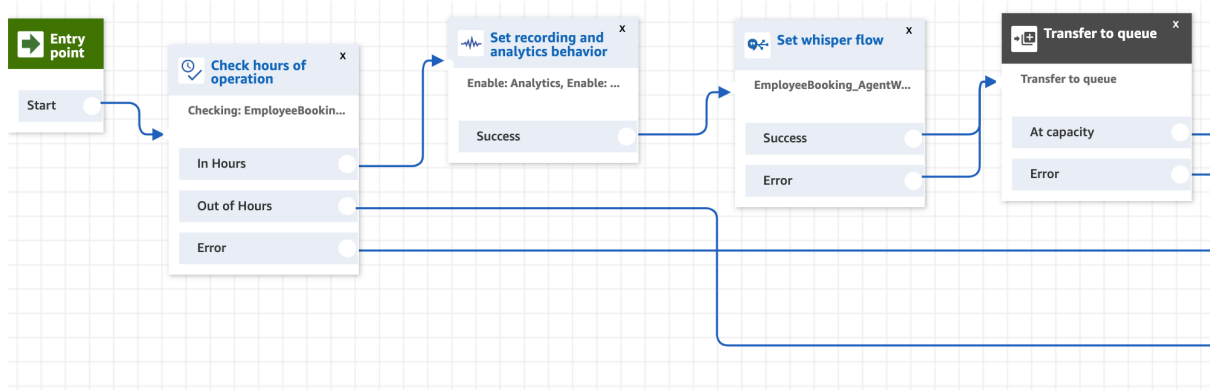
Employee's name is \$.Attributes.EmployeeName .

10. Click **Save and Publish**

11. Next, we need to add a reference to the new Agent Whisper Flow.

Navigate to Routing>Contact Flows> **EmployeeBooking_TransferToQueue**

12. Add new **Set Whisper Flow** node



13. Set Agent Whisper Flow `EmployeeBooking_AgentWhisper`

Set whisper flow x

Specifies the whisper to be played to customer on an outbound call, or to the customer or agent when the call is joined.

Whisper flow

☒ Agent

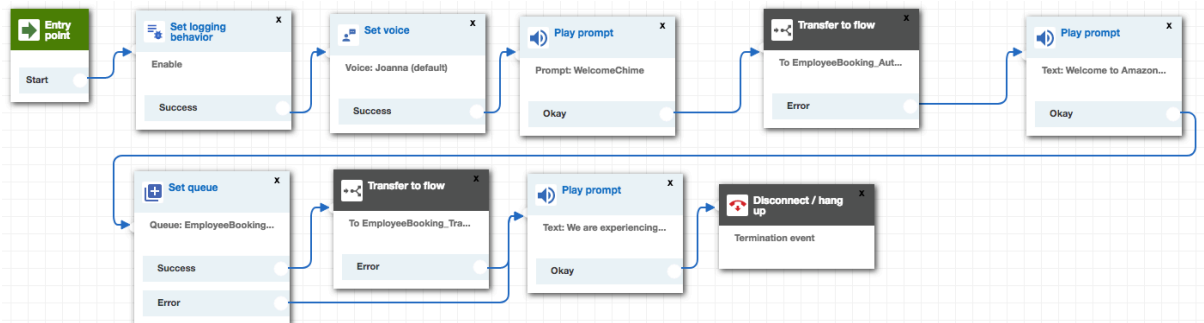
`EmployeeBooking_AgentWhisper` x ▼

☐ Customer

14. Click **Save and Publish**

As a final step in this module, we have to modify `EmployeeBooking_MainFlow`, to transfer a call to our Contact Flow for Authentication (`EmployeeBooking_Authentication`).

1. Navigate to Routing > Contact Flows > `EmployeeBooking_MainFlow`
2. Place a new **Transfer to flow** node after **Play Prompt – WelcomeChime**



3. Select a flow: **EmployeeBooking_Authentication**

Transfer to flow ✕

Ends the current flow and transfers the customer to a flow of type contact flow.

Transfer

☒ Select a flow

EmployeeBooking_Auth... ✕ ▼

☐ Use attributes

4. On Error, continue to existing **Play Prompt – Welcome Message**
5. Click **Save and Publish**

From this point, you can start making phone calls to your Amazon Connect instance! Please test different scenarios, with unknown and existing phone numbers, as well as authentication with *EmployeeID* and *EmployeePIN*.

Check the CloudWatch log files for Amazon Connect Contact Flow and AWS Lambda functions.

Transfer the authenticated employee to an agent and check if the Whisper Flow plays employee's name to the agent.

Hands-on Module 4: Amazon Lex Enhanced

Objective: Enhance previously created Amazon Lex chatbot by using a Lambda function to validate inputs and store bookings to a DynamoDB table.

Build your Lambda function

Lex bots leverage the use of AWS Lambda to place and request data from sources outside the Lex bot. You can create AWS Lambda functions to use as code hooks for your Amazon Lex bot. You can identify Lambda functions to perform initialization and validation, fulfillment, or both in your intent configuration. In this use case, the Lambda function is creating a code hook interface to serve a sample bot for fulfillment.

1. In your AWS console search bar, type “**Lambda**” and select **Lambda** from the results
2. *Ensure that your selected region is “US East (N. Virginia)” by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.*
3. Select **Create a function**
4. Select **Author from scratch**
5. Name: **EmployeeBooking_LexBookHotel**
6. Runtime: **Node.js 14.x**
7. Role: Use an existing role > **Lambda_EmployeeBooking**
8. Select **Create function**

Function name
Enter a name that describes the purpose of your function.

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions
☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

[View the Lambda_EmployeeBooking role](#) on the IAM console.

► **Advanced settings**

9. Copy/paste the code from https://s3.amazonaws.com/amazonconnectbootcampresources/v3.0/Module4/EmployeeBooking_LexBookHotel.js

10. Scroll down to the "Configuration -> General configuration " then increase the **Timeout** to 7 seconds.
11. Click **Deploy**

Please examine the code and notice the following:

1. It uses AWS SDK, DynamoDB Document Client and Crypto module

```

3 //load AWS-SDK
4 const AWS = require('aws-sdk');
5 //declare dynamoDB
6 const dynamoDB = new AWS.DynamoDB();
7 //load Crypto
8 const crypto = require("crypto");

```

2. The main handler is invoked each time an event is passed from Lex to Lambda. As a first step, for debugging purpose, it logs the event payload to CloudWatch Logs. It then sets the default Time Zone and invokes **dispatch** function so that intent can be determined.

```

12 // ----- Main handler -----
13 // -----
14
15 // Route the incoming request based on intent.
16 // The JSON body of the request is provided in the event slot.
17
18 exports.handler = (event, context, callback) => {
19
20     console.log("Event payload: " + JSON.stringify(event));
21
22     try {
23         // By default, treat the user request as coming from the America/New_York time zone.
24         process.env.TZ = 'America/New_York';
25         console.log(`event.bot.name=${event.bot.name}`);
26
27         dispatch(event, (response) => loggingCallback(response, callback));
28     } catch (err) {
29         callback(err);
30     }
31
32 };

```

3. In our use-case, we are going to have only a single intent – **BookHotel**, therefore our **dispatch** function supports only that intent. When the intent is recognized, it invokes **BookHotel** function – the main function that performs dialog management and fulfillment for booking a hotel.

```

72 function bookHotel(intentRequest, callback) {
73   console.log("DEBUG>>bookHotel : " + JSON.stringify(intentRequest, null, 2));
74   const location = intentRequest.currentIntent.slots.Location;
75   const checkInDate = intentRequest.currentIntent.slots.CheckInDate;
76   const nights = intentRequest.currentIntent.slots.Nights;
77   const sessionAttributes = intentRequest.sessionAttributes || {};
78
79   // Load confirmation history and track the current reservation.
80   const reservation = String(JSON.stringify({ ReservationType: 'Hotel', Location: location, CheckInDate: checkInDate, Nights: nights }));
81   sessionAttributes.currentReservation = reservation;
82
83   if (intentRequest.invocationSource === 'DialogCodeHook') {
84     // Validate any slots which have been specified. If any are invalid, re-elic for their value
85     const validationResult = validateHotel(intentRequest.currentIntent.slots);
86     if (!validationResult.isValid) {
87       const slots = intentRequest.currentIntent.slots;
88       slots['$validationResult.violatedSlot'] = null;
89       callback(elicitSlot(sessionAttributes, intentRequest.currentIntent.name,
90         slots, validationResult.violatedSlot, validationResult.message));
91       return;
92     }
93
94     // Otherwise, let native DM rules determine how to elicit for slots and prompt for confirmation.
95     // Pass price back in sessionAttributes once it can be calculated; otherwise clear any setting from sessionAttributes.
96     if (location && checkInDate && nights != null) {
97       // The price of the hotel has yet to be confirmed.
98       const price = generateHotelPrice(location, nights);
99       sessionAttributes.currentReservationPrice = price;
100     } else {
101       delete sessionAttributes.currentReservationPrice;
102     }
103     sessionAttributes.currentReservation = reservation;
104     callback(delegate(sessionAttributes, intentRequest.currentIntent.slots));
105     return;
106   }
107
108   // Booking the hotel. In a real application, this would likely involve a call to a backend service.
109   console.log("bookHotel under=${reservation}");
110
111   //store booking to DynamoDB
112   storeHotelBooking(JSON.parse(sessionAttributes.currentReservation), sessionAttributes.currentReservationPrice, sessionAttributes.CC_EmployeeID);
113
114   /*delete*/ sessionAttributes.currentReservationPrice;
115   delete sessionAttributes.currentReservation;
116   sessionAttributes.lastConfirmedReservation = reservation;
117
118   callback(close(sessionAttributes, 'Fulfilled',
119     { contentType: 'PlainText', content: 'Thanks, I have placed your reservation.' }));
120 }

```

- Our bot is going to ask 3 questions: Location (city), Check-In Date and number of Nights to stay. Each of these represents a **slot**, therefore we've got 3 slots: **Location**, **CheckInDate** and **Nights**

```

74 const location = intentRequest.currentIntent.slots.Location;
75 const checkInDate = intentRequest.currentIntent.slots.CheckInDate;
76 const nights = intentRequest.currentIntent.slots.Nights;

```

- Each time a caller gives an answer, the Lex bot will invoke the Lambda function, trying to validate the answer and either store the answer or ask the caller for another try.
- Once all answers are valid, the **generateHotelPrice** function will be invoked, which calculates the hotel price for given location and number of nights.

Note: This is just a random price, while in a real scenario the price would be retrieved from a database or a webservice.

```

170 // Generates a number within a reasonable range that might be expected for a hotel.
171 // The price is fixed for a location
172 function generateHotelPrice(location, nights) {
173   console.log("DEBUG>>generateHotelPrice location:" + JSON.stringify(location, null, 2) + ", nights:" + JSON.stringify(nights, null, 2));
174   let costOfLiving = 0;
175   for (let i = 0; i < location.length; i++) {
176     costOfLiving += location.toLowerCase().charCodeAt(i) - 97;
177   }
178   return nights * (100 + costOfLiving);
179 }
180

```

- Lex bot asks the employee for the final confirmation, and if the employee confirms, it stores the booking to DynamoDB. It uses the Crypto module to

generate a random *TripID* value.

```

125 function storeHotelBooking(reservation, price, employeeID)
126 {
127     console.log('DEBUG>>storeHotelBooking reservation:' + JSON.stringify(reservation, null, 2) + ', price:' + price);
128
129     //generate random TripID
130     var DDB_TripID = crypto.randomBytes(8).toString("hex") + '-' + crypto.randomBytes(8).toString("hex");
131
132     //set Parameters
133     var dynamoParams =
134     {
135         TableName: 'LexBookHotel',
136         Item: {
137             'TripID': {S:DDB_TripID},
138             'ReservationType': {S:reservation.ReservationType},
139             'Location': {S:reservation.Location},
140             'CheckInDate': {S:reservation.CheckInDate},
141             'Nights': {S:reservation.Nights},
142             'Price': {S:price},
143             'EmployeeID' : {S:employeeID}
144         }
145     };
146     console.log("DEBUG>>storeHotelBooking DDB Item: " + JSON.stringify(dynamoParams));
147     dynamoDB.putItem(dynamoParams, function(err, data){
148         if (err) {
149             console.log("DEBUG>>storeHotelBooking DynamoDB Error", err);
150         } else {
151             console.log("DEBUG>>storeHotelBooking DynamoDB Success", data);
152         }
153     });
154 }

```

8. EmployeeID is also stored for the booking. EmployeeID is passed as a parameter from Amazon Connect to the Lex Bot (`sessionAttributes.CC_EmployeeID`)
9. Function **ValidateHotel** is used to validate caller inputs (slots) and various helper functions are used to support the booking process: **parseLocalDate**, **isValidCity** (it uses a hard-coded list of cities), **isValidDate**, **buildValidationResult**.

```

207 function validateHotel(slots) {
208     console.log("DEBUG>>validateHotel : " + JSON.stringify(slots, null, 2));
209     const location = slots.Location;
210     const checkInDate = slots.CheckInDate;
211     const nights = slots.Nights;
212
213     if (location && !isValidCity(location)) {
214         return buildValidationResult(false, 'Location', 'We currently do not support ${location} as a valid destination. Can you try a different city?');
215     }
216
217     if (checkInDate) {
218         if (!isValidDate(checkInDate)) {
219             return buildValidationResult(false, 'CheckInDate', 'I did not understand your check in date. When would you like to check in?');
220         } if (parseLocalDate(checkInDate) < new Date()) {
221             return buildValidationResult(false, 'CheckInDate', 'Reservations must be scheduled at least one day in advance. Can you try a different date?');
222         }
223     }
224
225     if (nights != null && (nights < 1 || nights > 30)) {
226         return buildValidationResult(false, 'Nights', 'You can make a reservations for from one to thirty nights. How many nights would you like to stay for?');
227     }
228
229     return { isValid: true };
230 }

```

10. The Lambda function returns a response to the Lex Bot by using 3 helper functions that build responses in JSON format:
 - a. **ElicitSlot** is used in slot validation and re-prompting. It contains the *slot* for which to elicit data from the user and a *message* to convey to the user
 - b. **Delegate** is used by the Lambda function to delegate the responsibility for the next course of action to Amazon Lex.
 - c. **Close** is used once the Lambda function fulfills the intent, indicating that Amazon Lex does not expect a user response. It includes

fulfillmentState and an appropriate *message* to convey to the user.

```

239 function elicitSlot(sessionAttributes, intentName, slots, slotToElicit, message) {
240     return {
241         sessionAttributes,
242         dialogAction: {
243             type: 'ElicitSlot',
244             intentName,
245             slots,
246             slotToElicit,
247             message,
248         },
249     };
250 }
251
252 function close(sessionAttributes, fulfillmentState, message) {
253     return {
254         sessionAttributes,
255         dialogAction: {
256             type: 'Close',
257             fulfillmentState,
258             message,
259         },
260     };
261 }
262
263 function delegate(sessionAttributes, slots) {
264     return {
265         sessionAttributes,
266         dialogAction: {
267             type: 'Delegate',
268             slots,
269         },
270     };
271 }

```

11. The *currentReservationPrice* value is stored in *sessionAttributes* and will be passed back to an Amazon Connect Contact Flow, so it can be read to the caller.

Creating and configuring the DynamoDB table

To support our use-case, we have to create a DynamoDB table, which is going to store all bookings made through Amazon Lex bot.

1. In your AWS console search bar, type **"DynamoDB"** and select **DynamoDB** from the results
2. Ensure that your selected region is **"US East (N. Virginia)"** by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.
3. Click **Create table**
4. Table name: **LexBookHotel**
5. Primary key: **TripID**
6. Click **Create**

Create DynamoDB table

[Tutorial](#) ⓘ

DynamoDB is a schema-less database that only requires a table name and primary key. The table's primary key is made up of one or two attributes that uniquely identify items, partition the data, and sort data within each partition.

Table name* ⓘ

Primary key* Partition key

ⓘ

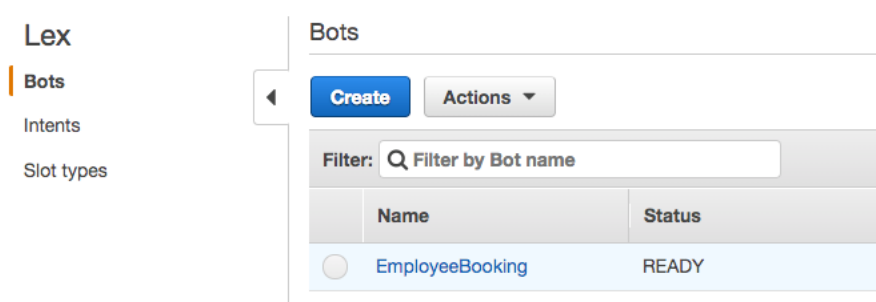
☐ Add sort key

7. You can continue with next section while the table is created

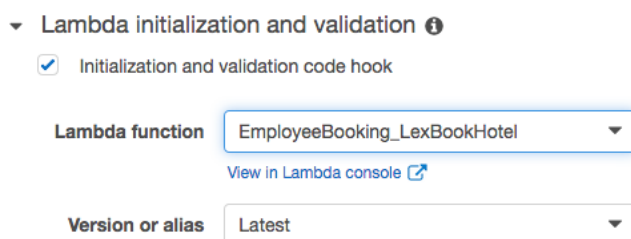
Reconfigure your Lex bot

We are going to reconfigure our Lex bot so it starts using previously created Lambda function for input validation and finally storing our booking to our DynamoDB table.

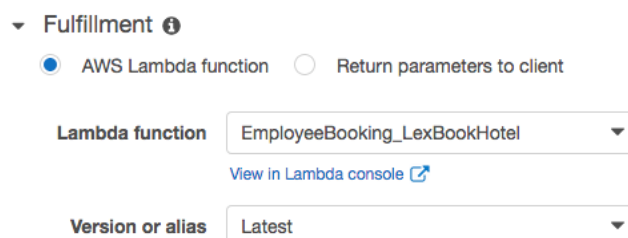
1. In your AWS console search bar, type **"Lex"** and select **Amazon Lex** from the results
2. *Ensure that your selected region is "US East (N. Virginia)" by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.*
3. Choose your bot from the list (EmployeeBooking) and select the BookHotel intent.



4. Set **Lambda Initialization and Validation** – select **EmployeeBooking_LexBookHotel**

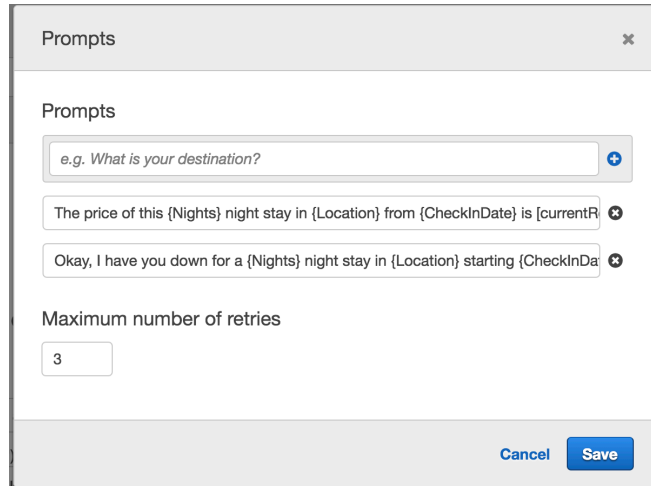


5. Set **Fulfillment** > **AWS Lambda Function** > **EmployeeBooking_LexBookHotel**



6. Add another **Confirmation prompt** that includes price
 - a. **Confirm**

- i. "Okay, I have you down for a {Nights} night stay in {Location} starting {CheckInDate}. Shall I book the reservation?"
- ii. "The price of this {Nights} night stay in {Location} from {CheckInDate} is [currentReservationPrice] dollars. Shall I book the reservation?"



The image shows a 'Prompts' dialog box with a close button (X) in the top right corner. Inside the dialog, there is a section titled 'Prompts' containing a text input field with the placeholder text 'e.g. What is your destination?' and a plus icon to its right. Below this, there are two more text input fields, each followed by a minus icon (X). The first of these fields contains the text 'The price of this {Nights} night stay in {Location} from {CheckInDate} is [currentR'. The second field contains the text 'Okay, I have you down for a {Nights} night stay in {Location} starting {CheckInDa'. Below the prompts, there is a section titled 'Maximum number of retries' with a text input field containing the number '3'. At the bottom right of the dialog, there are two buttons: 'Cancel' and 'Save'.

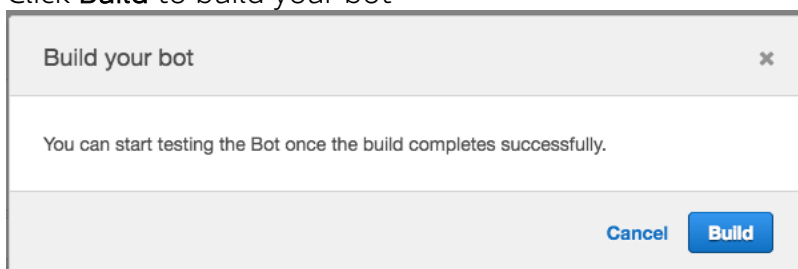
7. Remove the **Response** message set earlier, as response is produced by Lambda function:



The image shows a 'Response' dialog box with a dropdown arrow and an information icon (i) on the left. Inside the dialog, there is a dashed rectangular box containing a plus icon and the text 'Add Message'.

Please ensure that the "Cancel" prompt still has a message before trying to save.

8. Click **Save Intent**
9. Click **Build** to build your bot



The image shows a 'Build your bot' dialog box with a close button (X) in the top right corner. Inside the dialog, there is a text message that says 'You can start testing the Bot once the build completes successfully.' At the bottom right of the dialog, there are two buttons: 'Cancel' and 'Build'.

10. Once the bot is successfully built, you can test it by typing an utterance in the console on the left-hand side. Start by typing "book a hotel"

> Test Bot (Latest) 🟢 READY

book a hotel

What city will you be staying in?

Clear

Chat to your bot...

Inspect Response

Dialog State: ElicitSlot

☒ Summary ☐ Detail

Intent: BookHotel

Slots	(0/3)
CheckInDate	null
Location	null
Nights	null

11. Enter a city (*Location slot*)

> Test Bot (Latest) 🟢 READY

book a hotel

What city will you be staying in?

New York

What day do you want to check in?

Clear

Chat to your bot...

Inspect Response

Dialog State: ElicitSlot

☒ Summary ☐ Detail

Intent: BookHotel

Slots	(1/3)
CheckInDate	null
Location	New York
Nights	null

12. Enter a Check-In Date (*CheckInDate slot*)

> Test Bot (Latest) READY

book a hotel

What city will you be staying in?

New York

What day do you want to check in?

Saturday

How many nights will you be staying?

Clear

Chat to your bot...

Inspect Response

Dialog State: ElicitSlot

☒ Summary ☐ Detail

Intent: BookHotel

Slots	(2/3)
CheckInDate	2018-03-10
Location	New York
Nights	null

13. Enter how many nights you want to stay (*Nights slot*)

> Test Bot (Latest) READY

What day do you want to check in?

Saturday

How many nights will you be staying?

3

The price of this 3 night stay in New York from 2018-03-10 is 417 dollars. Shall I book the reservation?

Clear

Chat to your bot...

Inspect Response

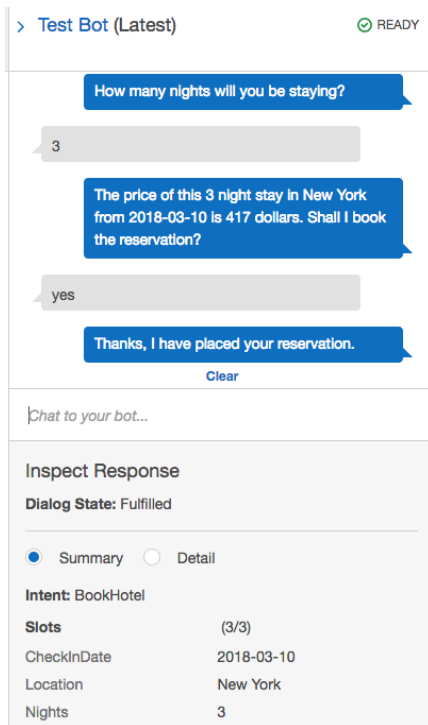
Dialog State: ConfirmIntent

☒ Summary ☐ Detail

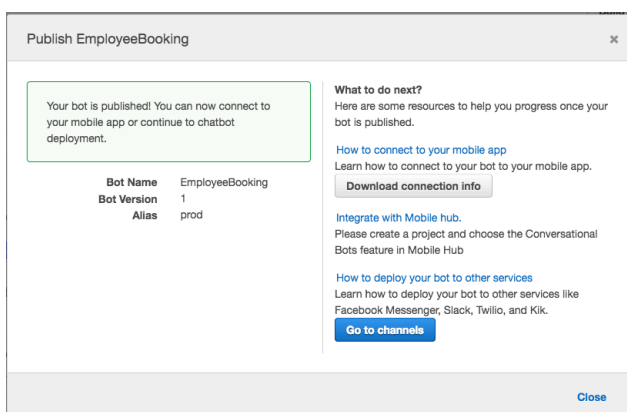
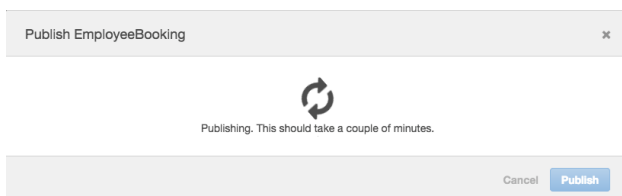
Intent: BookHotel

Slots	(3/3)
CheckInDate	2018-03-10
Location	New York
Nights	3

14. The Lex bot has calculated the price, and it asks you for the confirmation. Enter "yes"



15. The Lex bot confirms your reservation.
16. Navigate to your DynamoDB table, *LexBookHotel*, and check *Items*
17. The booking is missing – what could be the problem? Check the CloudWatch logs for your Lambda function!
18. Click **Publish**, set an alias as **prod**, click **Publish** again



Read booking price in your Amazon Connect Contact Flow

In this step, we are going to get the booking price from Amazon Lex bot and read it in our Amazon Connect Contact Flow. We just need to enhance our existing prompt.

1. In your Amazon Connect instance, navigate to Routing > Contact Flows
2. Select **EmployeeBooking_LexBot** flow
3. Open the first Play prompt block, the one that confirms booking
4. Edit the text by adding a reservation price returned from the Lex bot:

• Text to speech (Ad hoc)

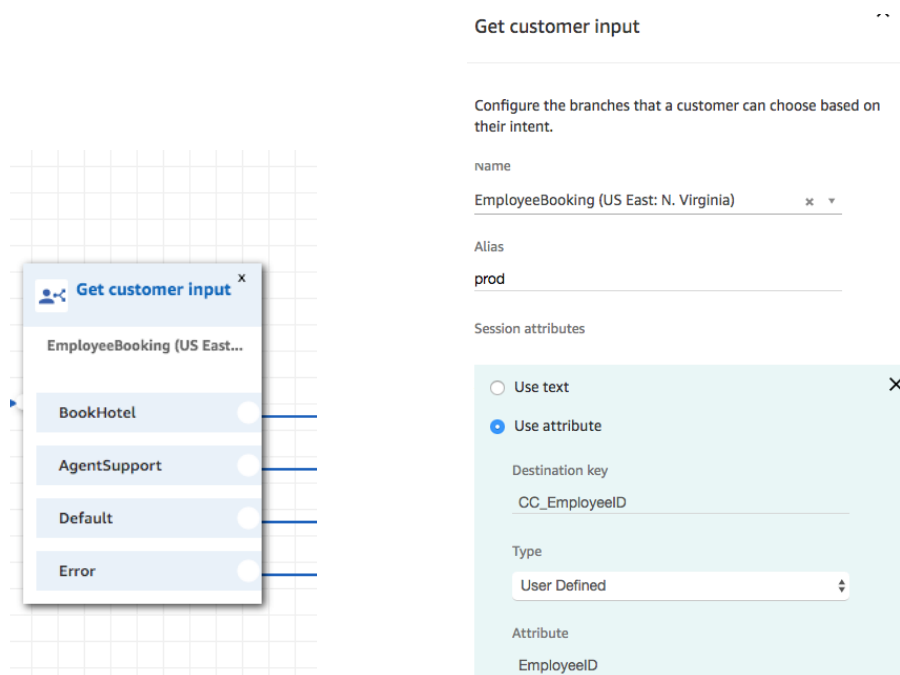
[Learn more about Amazon Connect's TTS capabilities](#)

• Enter text

Thank you for using our service to book a Hotel. Your account will be charged for \$.Lex.SessionAttributes.currentReservationPrice dollars.

"Thank you for using our service to book a hotel Your account will be charged for \$.Lex.SessionAttributes.currentReservationPrice dollars."

5. Open the first "Get Customer Input" block, that has Lex bot attached and add Session attribute as **CC_EmployeeID = User Defined > EmployeeID** (that will be passed to Lex bot and Lambda function attached to Lex bot, to be stored in DynamoDB table, as part of the trip reservation):



The screenshot displays the 'Get customer input' block configuration in the Amazon Connect console. The block is titled 'Get customer input' and is part of the 'EmployeeBooking (US East: N. Virginia)' flow. The configuration shows the 'Session attributes' section with 'Use attribute' selected, 'Destination key' set to 'CC_EmployeeID', 'Type' set to 'User Defined', and 'Attribute' set to 'EmployeeID'.

6. **Save** and **Publish** the flow

Testing it all together

It is time to test our use-case scenario:

1. Make a phone call to your Amazon Connect instance
2. When prompted, authenticate with your PhoneNumber and PIN (alternatively EmployeeID and PIN)
3. The Lex bot is going to ask "How can I help you today?"
4. Start by saying "book a hotel"
5. Follow the conversation and book your hotel
6. Check the DynamoDB table and find your booking

LexBookHotel [Close](#)

Overview **Items** Metrics Alarms Capacity Indexes Global Tables Backups Triggers Access control Tags

Create item Actions

Scan: [Table] LexBookHotel: TripID ^

Scan [Table] LexBookHotel: TripID ^

+ Add filter

Start search

	TripID	CheckInDate	EmployeeID	Location	Nights	Price	ReservationType
☐	ba191310a3bc5ad5-87f2b31345abe12b	2018-03-07	101	new york	4	556	Hotel

Adding Lex Variables to your Contact Flow

Amazon Lex passes a JSON output back to Amazon Connect, with the intent and slot information available for reference as a variable in Connect call flows.

You can reference the intent using the variable string ***\$.Lex.[intent]***, so in our example it would be ***\$.Lex.BookHotel***.

You can also reference the slots using the format ***\$.Lex.Slots.[slotName]***, so our example could return ***\$.Lex.Slots.Location***, ***\$.Lex.Slots.CheckInDate***, and ***\$.Lex.Slots.Nights*** as usable variables.

Lex will also return a variable of ***\$.Lex.DialogState*** to route a call as an attribute, for example, if a call is able to complete in Lex, the DialogState variable will be returned as "Fulfilled." In our use case, if the state returns as fulfilled, we will play a confirmation and disconnect. If the booking is NOT fulfilled, either because Lex could not understand, or if the user said "NO" to the confirmation prompt, Lex returns an error and we can use the error-handling option of the "Get Customer Input" block to route the call appropriately (in our case, to an agent).

In a contact flow, you would use a "Set Attribute" block to create a user-defined attribute of the returned variable, and then can use that attribute to route calls using a "***Check Attribute***" function. You may also refer to the variables in a text-to-speech field in any of the supported contact flow blocks to read back the returned information.

Hands-on Module 5 – Send SMS Confirmation

Objective: After the booking is successfully placed, send a SMS Confirmation which includes the booking price

In this module, we want to send a SMS Confirmation after the booking is completed. For this purpose, we are going to invoke AWS Lambda function, from Amazon Connect Contact Flow, and send an SMS by publishing it to AWS Simple Notification Service.

Amazon Simple Notification Service (SNS) is a flexible, fully managed pub/sub messaging and mobile notifications service for coordinating the delivery of messages to subscribing endpoints and clients. With SNS you can fan-out messages to a large number of subscribers, including distributed systems and services, and mobile devices. It is easy to set up, operate, and reliably send notifications to all your endpoints – at any scale. You can get started using SNS in a matter of minutes using the AWS Management Console, AWS Command Line Interface, or using the AWS SDK with just three simple APIs. SNS eliminates the complexity and overhead associated with managing and operating dedicated messaging software and infrastructure. More information can be found in the official documentation: <https://aws.amazon.com/documentation/sns/>

AWS Lambda role and policy for AWS SNS Publish

In this section, you will be creating an AWS Lambda function that publishes a SMS message that contains a booking confirmation and booking price. To function correctly, AWS Lambda will need an Amazon Identity and Access Management (IAM) **role** and **policy** with the appropriate permissions.

1. In your AWS console search bar, type **"IAM"** and select **IAM** from the results
2. In the navigation pane, select **"Policies"** and **"Create Policy"**
3. Choose a service: **SNS**
4. Actions: **Publish**
5. Resources: **All Resources**
6. Click **Add Additional Permissions**
7. Choose a service: **CloudWatch Logs**
8. Access level: **CreateLogGroup, CreateLogStream, PutLogEvents**
9. Resources: **All Resources**

Note: if you need to limit access to certain resources, it's possible to specify a certain resource/table in the policy document

10. Click **Review Policy**
11. Enter Policy name: **Lambda_EmployeeBooking_SMS**

12. Enter a description
13. Click **Create Policy**

Next step is to create a role:

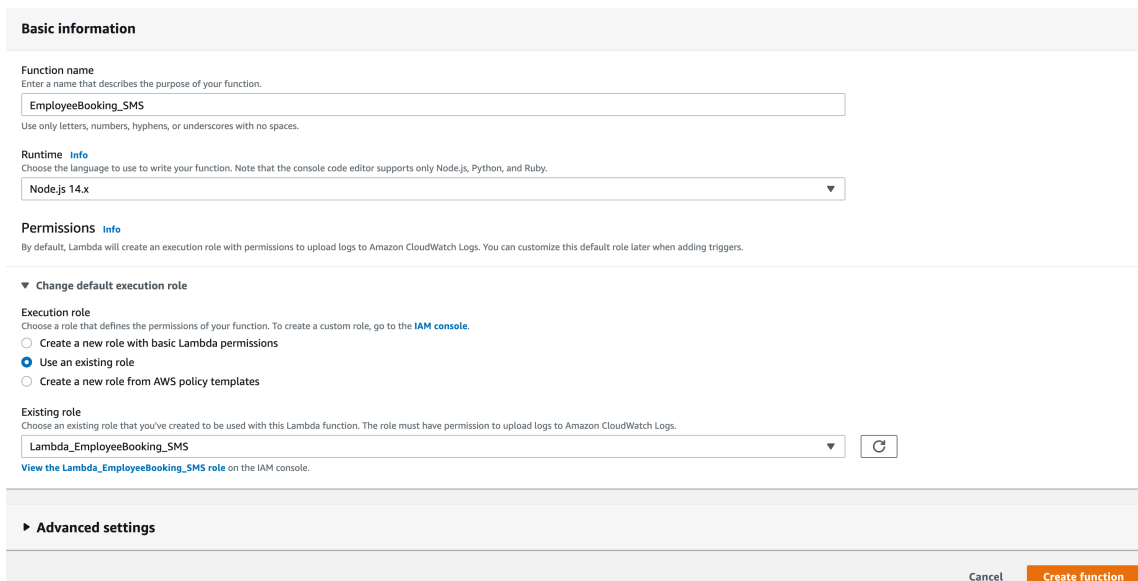
1. In the navigation pane, select **"Roles"** and **"Create Role"**
2. Choose the service that will use this role: **Lambda**
3. Click **Next: Permissions**
4. Select Policy: **Lambda_EmployeeBooking_SMS**
5. Click **Next: Tags**
6. At this point you can add custom Tags. More information can be found here: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_tags.html?icmpid=docs_iam_console#id_tags_procs
For the purpose of this lab, we don't need to set any Tags
7. Click **Next: Review**
8. Role name: **Lambda_EmployeeBooking_SMS**
9. Click **Create Role**

Once the role is created, we are ready to create our Lambda function.

1. In your AWS console search bar, type **"Lambda"** and select Lambda from the results

Note: Ensure that your selected region is "US East (N. Virginia)" by clicking on your region in the upper right-hand corner of the AWS console and selecting the region.

2. Select **Create a Function**
3. Select **Author from scratch**
4. Name: **EmployeeBooking_SMS**
5. Runtime: **Node.js 14.x**
6. Role: Use an existing role > **Lambda_EmployeeBooking_SMS**
7. Select **Create Function**



The screenshot shows the 'Basic information' tab of the 'Create Function' wizard in the AWS console. The 'Function name' field is populated with 'EmployeeBooking_SMS'. The 'Runtime' dropdown is set to 'Node.js 14.x'. Under the 'Permissions' section, the 'Change default execution role' dropdown is expanded, showing three options: 'Create a new role with basic Lambda permissions', 'Use an existing role' (which is selected), and 'Create a new role from AWS policy templates'. Below this, the 'Existing role' dropdown is set to 'Lambda_EmployeeBooking_SMS'. At the bottom right, there are 'Cancel' and 'Create function' buttons.

8. The Lambda function code can be downloaded from here:
https://s3.amazonaws.com/amazonconnectbootcampresources/v3.0/Module5/EmployeeBooking_SMS.js
9. Start examining your function, first step is loading AWS SDK

```

1  'use strict';
2
3  const AWS = require('aws-sdk');
4
5  exports.handler = (event, context, callback) => {
6
7
8      callback(null, 'Hello from Lambda');
9  }
10 }

```

10. As a next step, for debugging purposes, it logs the event payload to the CloudWatch Logs

```

5  exports.handler = (event, context, callback) => {
6
7      console.log('Received event:', JSON.stringify(event, null, 2));
8
9  }

```

11. To be able to communicate with SNS, we have to declare it, together with AWS Region (us-east-1 in our case): Please change AWS Region if you choose different region

```

7      console.log('Received event:', JSON.stringify(event, null, 2));
8
9      var AWSRegion = 'us-east-1';
10     AWS.config.update({region: AWSRegion});
11     var SNS = new AWS.SNS();

```

12. Next, we have to retrieve parameters sent by Amazon Connect and store them into local variables. In our case, Amazon Connect sends *PhoneNumber* and *BookingPrice*

```

12
13     var PhoneNumber = event.Details.Parameters.PhoneNumber;
14     var BookingPrice = event.Details.Parameters.BookingPrice;
15

```

13. We can then use BookingPrice to create our MessageBody

```

16     var MessageBody = "Thank you for using our service to book a Hotel." +
17     "Your account will be charged for " + BookingPrice + " dollars.";
18

```

14. We are creating a payload for SNS, with all the details, like PhoneNumber, Message and MessageAttributes (SenderId):

```

18
19     const params = {
20       PhoneNumber: PhoneNumber,
21       Message: MessageBody,
22       MessageAttributes: {
23         "AWS.SNS.SMS.SenderID": {
24           "DataType": "String",
25           "StringValue": "Booking"
26         }
27       }
28     };
29

```

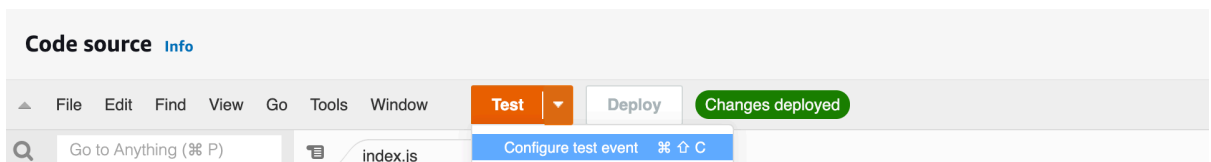
15. In the final step, we are submitting the payload to SNS service by invoking SNS.publish API. We are also sending the ReturnResult back to Amazon Connect, in form of true or false value.

```

29
30     var ReturnResult = {};
31
32     SNS.publish(params, function(err, data){
33       console.log('Sending message to ' + params.PhoneNumber.toString());
34       if (err){
35         console.log(err, err.stack);
36         ReturnResult.Success = false;
37         callback(null, ReturnResult);
38       }
39       console.log('Message sent to ' + params.PhoneNumber.toString());
40       ReturnResult.Success = true;
41       callback(null, ReturnResult);
42     });
43 }

```

16. Scroll down to the “**Configuration-> General configuration**” then increase the **Timeout** to 7 seconds.
17. Click **Save**
18. It is strongly recommended that you test your Lambda function before you continue with the Amazon Connect integration. Click on **Configure Test Events**



19. Set Event name as **TestSMS** then create an event payload like in the below example. Please make sure that you’ve set your phone number instead.

```

{
  "Details": {
    "ContactData": {},
    "Parameters": {
      "PhoneNumber": "+447123456789",
      "BookingPrice": "200"
    }
  },
  "Name": "ContactFlowEvent"
}

```

Configure test event

A function can have up to 10 test events. The events are persisted so you can switch to another computer or web browser and test your function with the same events.

☒ Create new test event
 ☐ Edit saved test events

Event template

Hello World

Event name

TestSMS

```

1 {
2   "Details": {
3     "ContactData": {},
4     "Parameters": {
5       "PhoneNumber" : "+44123123123",
6       "BookingPrice" : "217"
7     }
8   },
9   "Name" : "ContactFlowEvent"
10 }

```

Cancel

Create

20. Once you are done with the payload, click the **Create** button

21. Click the **Test** button to test you Lambda function

EmployeeBooking_SMS

Throttle

Qualifiers ▼

Actions ▼

TestSMS ▼

Test

Save

22. Lambda function should return a success message and the SMS message should arrive on your cell phone

EmployeeBooking_SMS Throttle Qualifiers Actions TestSMS Test Save

✓ Execution result: succeeded (logs) ✕

▼ Details

The area below shows the result returned by your function execution. [Learn more](#) about returning results from your function.

```
{
  "Success": true
}
```

Summary

Code SHA-256	YiStCrzbBJAKKtV+EJscM0Gk/+Vtq02jWvL+4adp/Y=	Request ID	316cee2d-5f68-11e8-a08d-f9078e8c2e42
Duration	1535.50 ms	Billed duration	1600 ms
Resources configured	128 MB	Max memory used	34 MB

Log output

The area below shows the logging calls in your code. These correspond to a single row within the CloudWatch log group corresponding to this Lambda function. [Click here](#) to view the CloudWatch log group.

```
START RequestId: 316cee2d-5f68-11e8-a08d-f9078e8c2e42 Version: $LATEST
2018-05-24T15:36:15.125Z 316cee2d-5f68-11e8-a08d-f9078e8c2e42 Received event: {
  "Details": {
    "ContactData": {},
    "Parameters": {
      "PhoneNumber": "+44123123123",
      "BookingPrice": "217"
    }
  },
  "Name": "ContactFlowEvent"
}
```

Giving Amazon Connect permission to invoke your Lambda function

When invoking a Lambda function from Amazon Connect Contact Flow, you need to give Amazon Connect permission to execute your AWS Lambda functions.

1. Navigate to Amazon Connect Service > Click on your instance name
2. From the left-hand menu, select **Contact Flows**
3. Select **Region** – in this case, **US East: N. Virginia**
4. In the AWS Lambda section, select the Lambda function name – in this case, **EmployeeBooking_SMS**

AWS Lambda

Amazon Connect can interact with your own systems and take different paths in IVR dynamically. To achieve this, invoke AWS Lambda functions in contact flows to interact with your own systems or other services, then build personalized and dynamic experiences based on data returned.

Note: By adding Lambda functions, you are granting Amazon Connect permission to invoke them [Create a new Lambda function](#) ↗

Function EmployeeBooking_SMS ▼ [+ Add Lambda Function](#)

Lambda Functions

EmployeeBooking_DBLookup ⓘ [Remove](#)

5. Click **+ Add Lambda Function**

AWS Lambda

Amazon Connect can interact with your own systems and take different paths in IVR dynamically. To achieve this, invoke AWS Lambda functions in contact flows to interact with your own systems or other services, then build personalized and dynamic experiences based on data returned.

Note: By adding Lambda functions, you are granting Amazon Connect permission to invoke them [Create a new Lambda function](#)

Function

Select

▼

+ Add Lambda Function

Lambda Functions

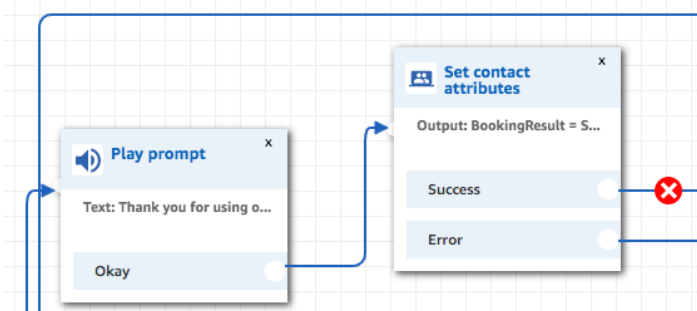
EmployeeBooking_DBLookup		Remove
EmployeeBooking_SMS		Remove

- From this moment, our Amazon Connect instance has the permission to execute the **EmployeeBooking_SMS** Lambda function. In the next step, we are going to update the contact flow, which utilizes this Lambda function to send an SMS to the caller.

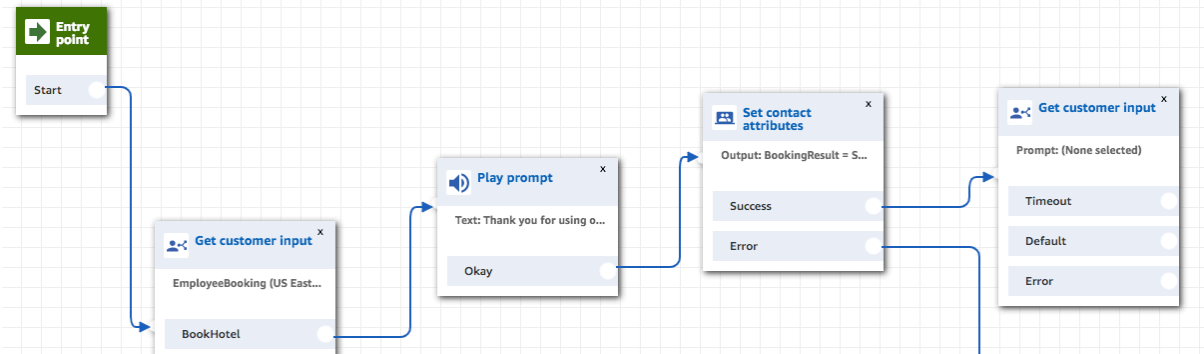
Update Amazon Connect Contact Flow

In this section, we are going to update our Contact Flow, by asking the caller if SMS Confirmation is needed and invoking our Lambda function to send the SMS out.

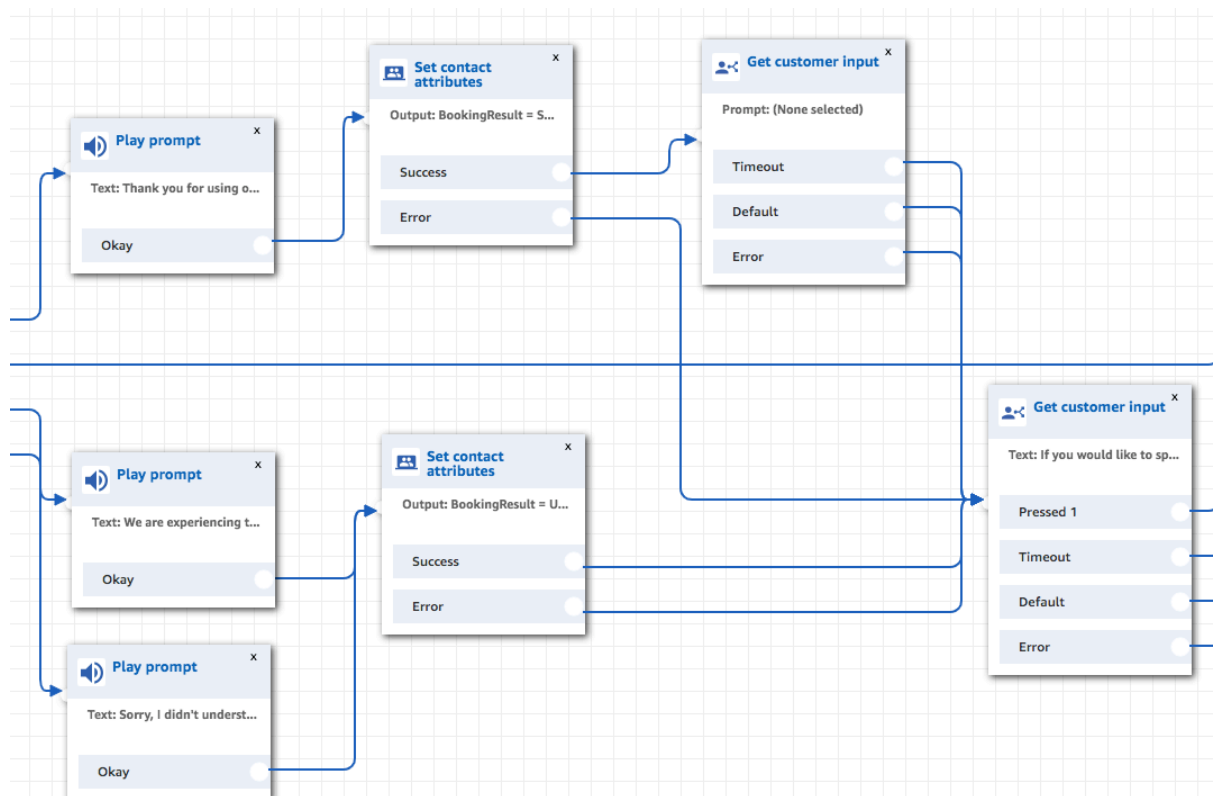
- Navigate to your Amazon Connect Instance > Routing > Contact Flows
- Open **EmployeeBooking_LexBot** contact flow
- Identify "Set contact attributes" block that sets "**BookingResult**"="**Success**" and break its output connection



- Place new "Get customer input" block and connect it to previous one



5. Connect "Timeout", "Default" and "Error" outputs to the next "Get customer input" block:



6. Configure new "Get customer input" block so that it asks the caller if SMS confirmation is needed:

Get customer input x

Configure the branches that a customer can choose based on their intent.

☒ Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

☒ Enter text

If you want to get SMS confirmation, please press 1.

☐ Enter dynamically

Interpret as

Text

DTMF

Amazon Lex

Configure the branches that a customer can choose based on their intent.

Set timeout (Minimum one second)

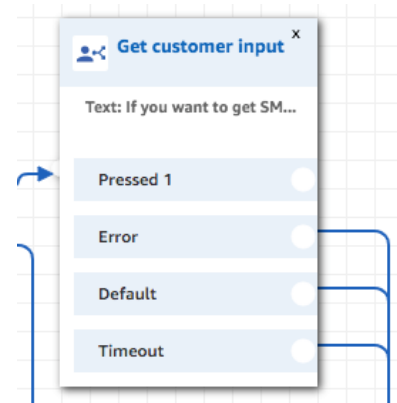
5 seconds

Option:

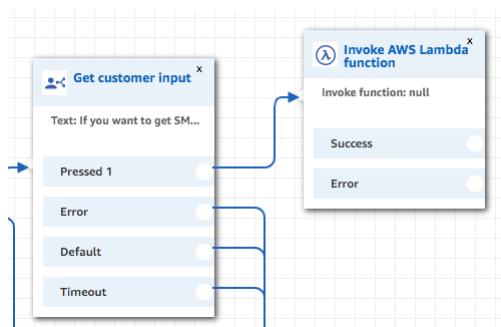
x 1

Cancel

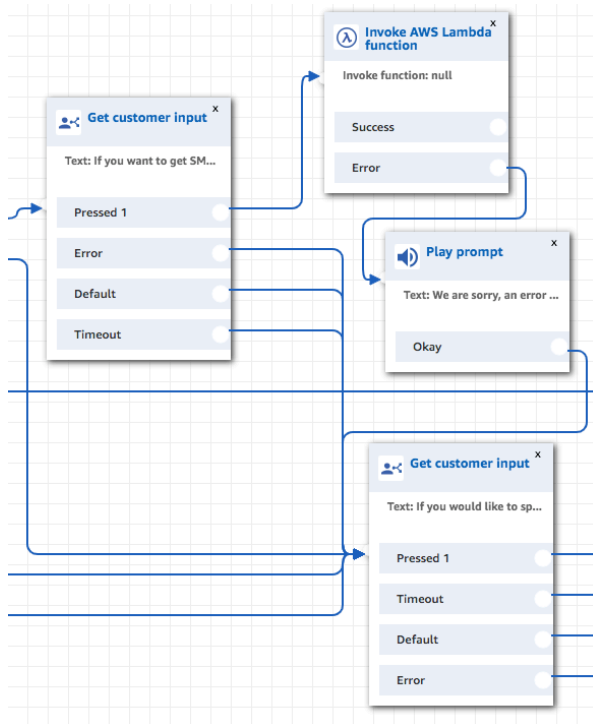
Save



- Place new "Invoke AWS Lambda function" block and connect it to Option 1 from the previous "Get customer input" block



- On Error, we are going to play a message and offer agent transfer:



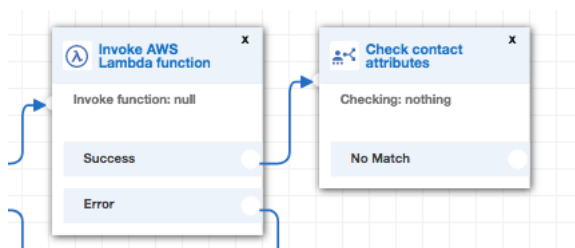
• Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

• Enter text

We're sorry, an error occurred.

1. The "Success" output indicates that Lambda has been successfully invoked, but we have to check the result returned by Lambda (true or false). Place a new "Check contact attributes" block and connect its input to the Success output from previous block.



2. Check if External > Success is equal to true:

Attribute to check

Type

External

Attribute

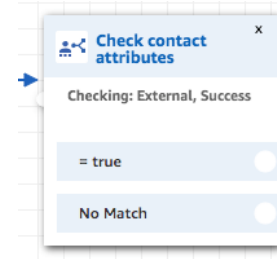
Success

Conditions to check

x Equals

true

No Match



3. In that case, play a confirmation message to the caller and offer agent transfer.

Text to speech (Ad hoc)

[Learn more about Amazon Connect's TTS capabilities](#)

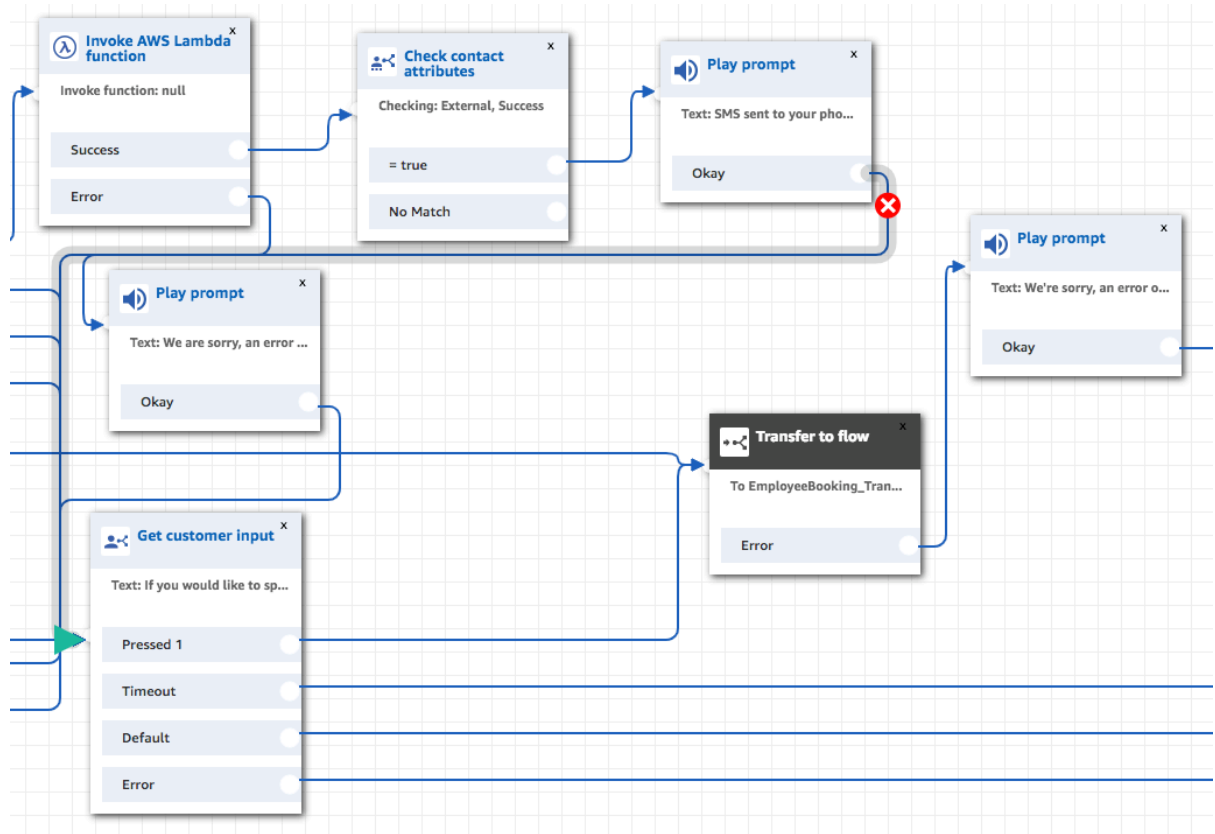
Enter text

SMS sent to your phone number.

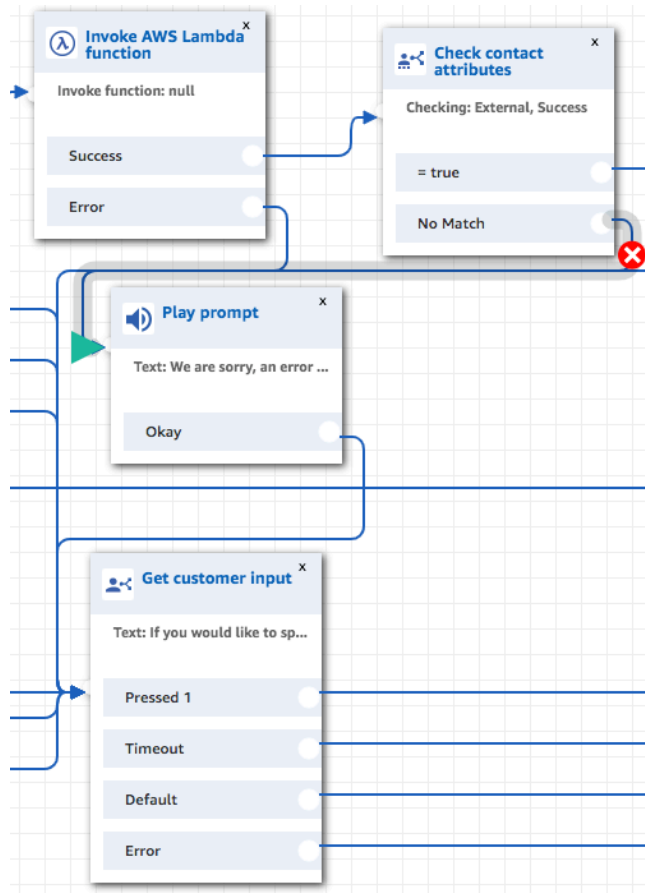
Enter dynamically

Interpret as

Text



4. Otherwise, if Successful is not equal to 'true', play the error message and offer the agent transfer:



5. Last piece is to configure "Invoke AWS Lambda function" block. We have to set the Lambda ARN and the parameters that we want to send to the Lambda function.

In this case, we are sending:

System > Customer Number as **PhoneNumber** input parameter and `$.Lex.SessionAttributes.currentReservationPrice` as **BookingPrice** input parameter.

The timeout for the function is set to 4 seconds.

Note: You will need to update the ARN field with your own Lambda ARN.

×

Invoke AWS Lambda function

Makes a call to AWS Lambda, and optionally returns key / value pairs.

The returned key value pairs can be used to set contact attributes.

Function ARN

☒ Select a function

EmployeeBooking_SMS ▾

☐ Use attributes

Function input parameters

×

☐ Use text
 ☒ Use attribute

Destination key
 PhoneNumber

Type
 System

Attribute
 Customer Number

×

☒ Use text
 ☐ Use attribute

Destination key
 BookingPrice

Value
 \$.Lex.SessionAttributes.currentReservationPrice

☐ Use attribute

[Add another parameter](#)

Timeout (max 8 seconds)

4

Cancel

Save

6. **Save** and **Publish** your contact flow
7. Make an inbound call, authenticate, make a hotel reservation and ask for SMS Confirmation, the check your cell phone for the SMS.

Note: If this is the first time using SMS, the phone number to receive the SMS will need to set up as sandbox number under Amazon SNS.

Hands-on Module 6 – Integrate CCP into a custom web page

Objective: Use Amazon Connect Streams API to integrate Contact Control Panel (CCP) into a custom web page and display User Defined Attributes to your agents.

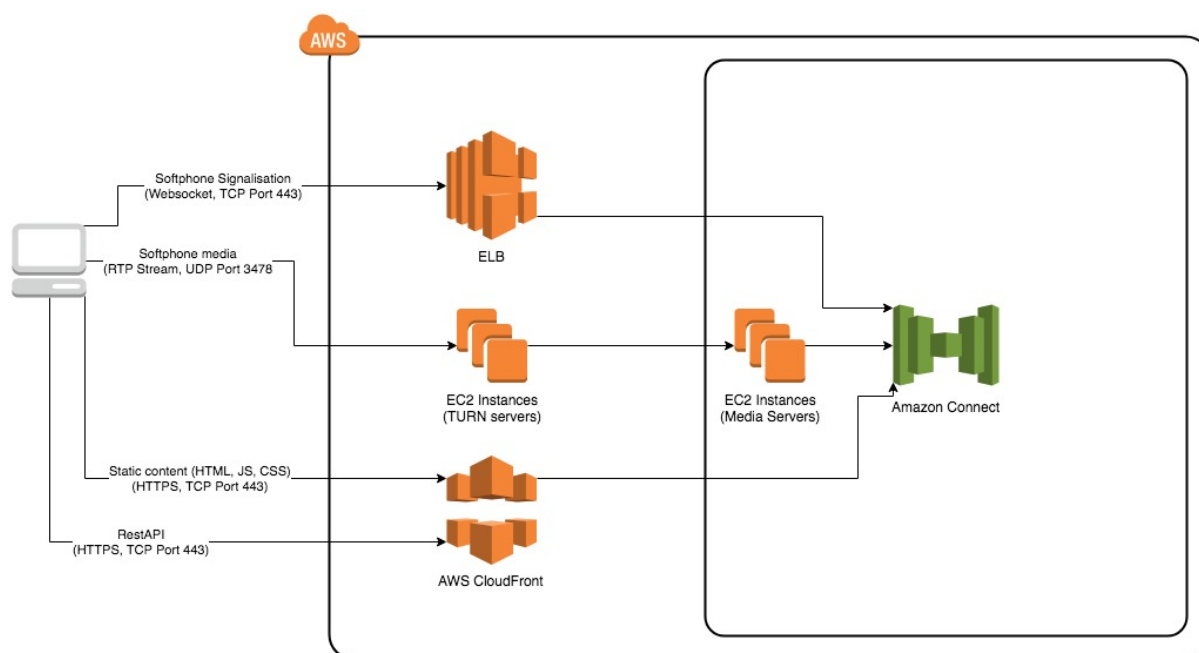
The Amazon Connect Contact Control Panel (CCP) is used by agents to communicate with customers (contacts). The CCP can be used with a built-in soft-phone or a desk-phone. Softphone is a browser-based telephony service that is not linked to a handset. It can be used remotely, provided that the agent is logged in to the Amazon Connect instance.

Amazon Connect CCP runs natively in a web browser and it doesn't require any special plugins. It uses WebRTC for direct peer-to-peer communication. In this case, it's a direct communication between CCP (which runs in agent's browser) and Amazon Connect platform in AWS Cloud.

With CCP, we can identify 4 different types of communication:

- **Static content** – HTML/JavaScript/CSS/Image files, which are downloaded directly from AWS CloudFront into the agent's web browser. This is a standard HTTPS, on **TCP Port 443**.
- **RestAPI** – CCP communicates with Amazon Connect platform via RestAPI (using Amazon Connect Streams API). In addition to handling API requests from downstream, it performs long-polling of agent state data and incremental polling of agent configuration data. All requests go through AWS CloudFront, via standard HTTPS, on **TCP Port 443**. The long-polling connection is established on agent login, and it's active all the time the agent is logged in. All events from the server to the browser will be delivered through this connection, including "new call" event.
- **Softphone signalling** – CCP uses Websocket protocol (WSS – Websocket Secure) for softphone signalisation between agent's browser and Amazon Connect platform. The Websocket connection is established whenever a new call has to be established and stays active until the call is disconnected. The event about the new call is delivered via long-polling connection and that triggers a new Websocket connection. Websocket is established from agent's browser to AWS Elastic Load Balancer, through **TCP connection, port 443**.
- **Softphone media stream** – CCP uses WebRTC protocol to establish real-time media between agent's browser and Amazon Connect platform. It leverages TURN protocol to establish RTP stream for voice channel, once the signalling connection is established (websocket). It always starts with 2 TURN servers, but the final connection is established with a single TURN server, on **UDP Port 3478**.

The communication diagram is given in the following picture:



Amazon Connect Streams

The Amazon Connect Streams API (Streams) gives you the power to integrate your existing web applications with Amazon Connect. Streams enables you to embed the Contact Control Panel (CCP) UI components into your page, but also to build your own from scratch by handling agent and contact state events directly through an object-oriented event driven interface. It's open-source and available on GitHub: <https://github.com/aws/amazon-connect-streams>

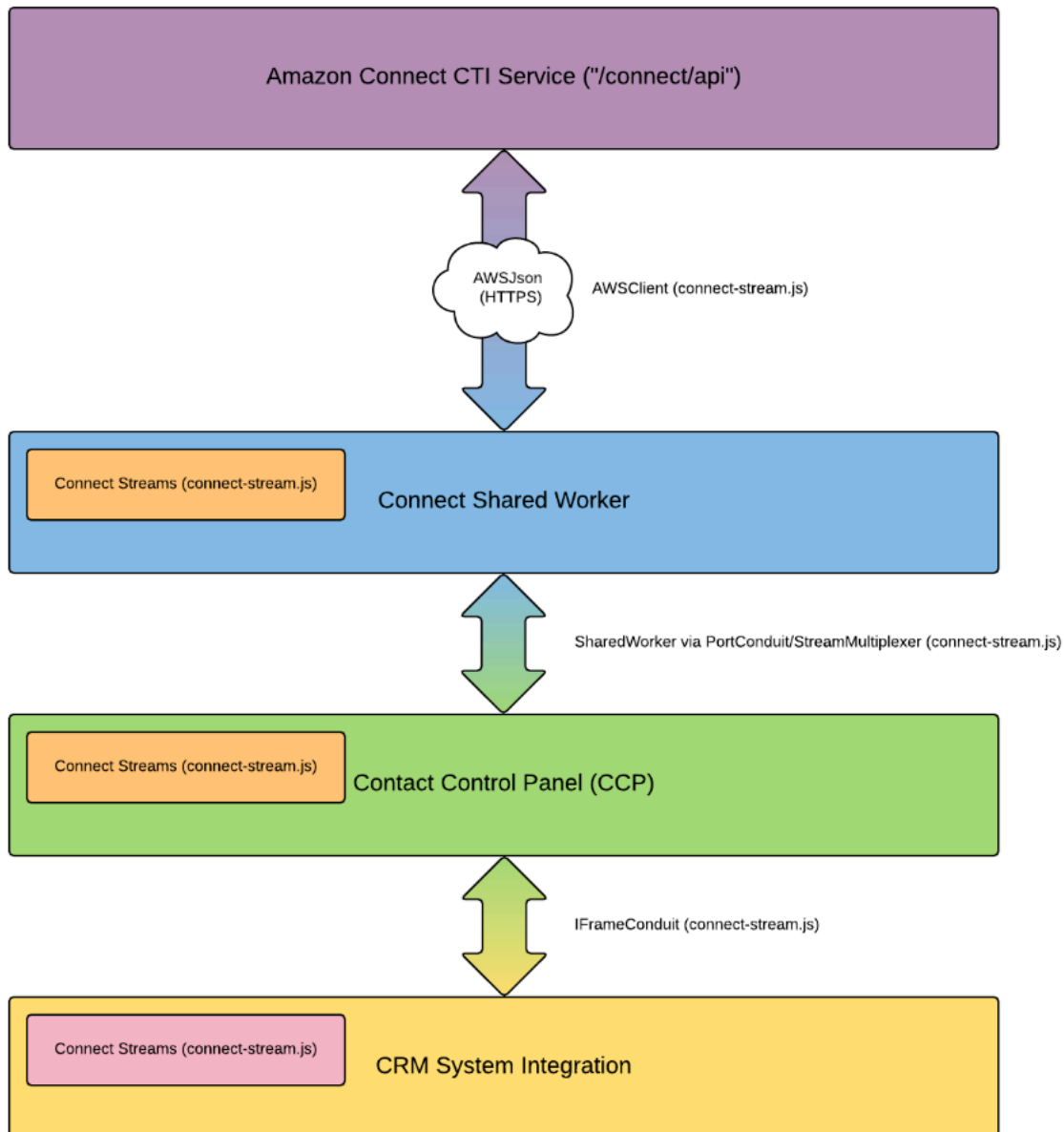
Amazon Connect Streams uses a multi-layered approach to provide a consistent level of control and customizability for CRM integrations, allowing you to build as sophisticated or as limited of an integration as is necessary for your business needs.

Amazon Connect Streams uses the concept of *Streams* to represent the connection points between each browser-side layer of the integration. Multiple *Streams* are connected together to create a bi-directional *Conduit* through which events and requests flow. There are three browser-side layers:

- **CRM Integration** – CRM client code initializes Connect Streams. The client initialization code builds an `<iframe/>` into which CCP is loaded. This may be visible or hidden, based on your preferences. For instance, despite the CCP is loaded in your CRM/Web page, you can hide all it completely and build your own UI components for call control.
- **Contact Control Panel (CCP)** – built-in agent experience, plays the role of a middle-man for requests between the CRM integration and the *Connect Shared Worker* which makes AJAX requests to the *Amazon Connect CTI Service*
- **Connect Shared Worker** – an HTML5 *SharedWorker* that handles API requests and performs long-polling of agent state data and incremental polling of agent configuration data. Whenever the agent state or configuration data changes,

these changes are synchronized across all open CCP tabs, iframes, and connected CRM integration pages.

The Amazon Connect Streams high-level diagram is given in the following picture:



An example of embedded CCP is given in the following pictures:

Ready:

Change status ▾

⚙

Available

Dial number

Quick connects

Amazon Connect CCP Integration Demo

Demonstrates the ability to embed the Call Control Panel and pass data to a form

Contact Details

Phone Number

+15551112222

Full Name

First Name

Last Name

↺

Contact ID

Contact ID

📄

Show Contact Attributes

Alerting:

Change status ▾

⚙

Inbound call

From

+44 7769 161124

Time elapsed

0:00:07

📞

❌

Amazon Connect CCP Integration Demo

Demonstrates the ability to embed the Call Control Panel and pass data to a form

Contact Details

Phone Number

+447769161124

Full Name

First Name

Last Name

↺

Contact ID

fdbadf6d-3e2f-4fc0-9aa2-67b8942aaa67

📄

Show Contact Attributes

In a Call:

Change status ▾

⚙

Connected

With

+44 7769 161124

Time elapsed

0:01:49

⏸ Hold

Dial number

🔄 Transfer

🔴 End call

Amazon Connect CCP Integration Demo

Demonstrates the ability to embed the Call Control Panel and pass data to a form

Contact Details

Phone Number

+447769161124

Full Name

First Name

Last Name

↺

Contact ID

fdbadf6d-3e2f-4fc0-9aa2-67b8942aaa67

📄

Hide Contact Attributes

	Name	Value
greetingPlayed		true
testAttribute1		value1

An example of Custom CCP, with hidden default UI:

Ready:

I'M READY!

Available ▾

Agent Details

Agent Name: Milos
Current Status: Available
Routing Profile: Basic Routing Profile

Hide Milos's Queues

Queue Name: BasicQueue

Contact Details

Customer Number:
Full Name First Name Last Name
Contact ID Contact ID

Show Contact Attributes

Alerting:

I'M READY!

Available ▾

Answer

Reject

Agent Details

Agent Name: Milos
Current Status: PendingBusy
Routing Profile: Basic Routing Profile

Show Milos's Queues

Contact Details

Customer Number: +447769161124
Full Name First Name Last Name
Contact ID 1e230e24-f0fc-4ec0-a7a1-8454f6060b21

Show Contact Attributes

In a Call:

I'M READY!

Available ▾

Hang UP

Agent Details

Agent Name: Milos
Current Status: Busy
Routing Profile: Basic Routing Profile

Show Milos's Queues

Contact Details

Customer Number: +447769161124
Full Name First Name Last Name
Contact ID 1e230e24-f0fc-4ec0-a7a1-8454f6060b21

Show Contact Attributes

Getting Started – Downloading Streams

The first step to embedding Amazon Connect CCP into your application is to download the Streams library from GitHub. You can do that by cloning our public repository here:

Mac OS Instructions:

```
$ git clone https://github.com/aws/amazon-connect-streams
```

If you don't have Git installed on your PC, please follow this guide:
<https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>

Once you have downloaded streams, change into the directory and build it:

```
$ cd amazon-connect-streams  
$ make
```

This will generate a file called `connect-streams-${VERSION}.js` which is the full version of Amazon Connect Streams API to be included in your web page. You will serve `connect-streams-${VERSION}.js` within your web application.

Windows Instructions:

Run Windows Powershell under Administrator permission. Install latest LTS version of NodeJS from <https://nodejs.org/en/>

```
PS C:\tests> npm install -g gulp  
PS C:\tests> git clone https://github.com/aws/amazon-connect-streams  
PS C:\tests> cd amazon-connect-streams  
PS C:\tests> npm install  
PS C:\tests> npm run release
```

Find build artifacts in release directory - This will generate a file called `connect-streams.js` and the minified version of the same `connect-streams-min.js` - this is the full Connect Streams API which you will want to include in your page.

Initialization

Initializing the Streams API is the first step to verify that you have everything setup correctly and that you will be able to listen for events

```
connect.core.initCCP(containerDiv, {
  ccpUrl:      ccpUrl,          /*REQUIRED*/
  loginPopup:  true,           /*optional, default TRUE*/
  loginUrl:    loginUrl,       /*optional*/
  region:      region          /*REQUIRED*/
  softphone:   {               /*optional*/
    disableRingtone: true,     /*optional*/
    ringtoneUrl: ringtoneUrl   /*optional*/
  }
});
```

The parameters for initializations are:

- **ccpUrl**: The URL of the CCP. This is the page you would normally navigate to in order to use the CCP in a standalone page, it is different for each instance.
- **region**: Amazon connect instance region. ex: us-east-1 for PDX ccp instance. only required for chat channel.
- **loginPopup**: Optional, defaults to `true`. Set to `false` to disable the login popup which is shown when the user's authentication expires.
- **loginUrl**: Optional. Allows custom URL to be used to initiate the ccp, as in the case of SAML authentication.
- **softphone**: This object is optional and allows you to specify some settings surrounding the softphone feature of Connect.
 - **allowFramedSoftphone**: Normally, the softphone microphone and speaker components are not allowed to be hosted in an `iframe`. This is because the softphone must be hosted in a single window or tab. The window hosting the softphone session must not be closed during the course of a softphone call or the call will be disconnected. If `allowFramedSoftphone` is `true`, the softphone components will be allowed to be hosted in this window or tab.
 - **disableRingtone**: This option allows you to completely disable the built-in ringtone audio that is played when a call is incoming.
 - **ringtoneUrl**: If the ringtone is not disabled, this allows for overriding the ringtone with any browser-supported audio file accessible by the user.

You have the option to show or hide the pre-built UI by showing or hiding the `containerDiv` into which you place the `iframe`

The pre-built CCP UI is portrait oriented and capable of contracting horizontally to fit smaller widths. It can expand from a width of 200px to a maximum of 320px based on the size of its container div. If the CCP is placed into a container where it would be sized under 221px in width, the CCP switches to a different layout style with smaller buttons and fonts. In its normal larger style, the CCP is 465px tall. In its smaller form, the CCP is reduced to 400px tall

CSS styles you add to your site will NOT be applied to the CCP because it is rendered in an `iframe`.

Event Subscription

Event subscriptions link your app into the heartbeat of Amazon Connect by allowing your code to be called when new agent information is available.

`connect.agent()` - Subscribe a method to be called when the agent is initialized

`connect.contact()` - Subscribe a method to be called for each newly detected contact

Simple – Serverless – Custom CCP

In this section we are going to create our custom web page and embed the CCP into it. The web page will be hosted in a S3 bucket and distributed via CloudFront (Content Delivery Network - CDN, for more information visit official documentation: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html>).

The sample HTML page can be downloaded here:

<https://s3.amazonaws.com/amazonconnectbootcampccp/SimpleAgent.html>

The StreamsAPI library v1.2 can be downloaded here:

<https://s3.amazonaws.com/amazonconnectbootcampccp/amazon-connect-v1.2.0-23-g430d835.js>

Start by opening your text editor and create *SimpleAgent.html* file.

In the `<head>` section, include:

- `amazon-connect-${version}.js` that you have created in the 'Downloading Streams'
- jQuery library

```

1 <html>
2   <head>
3     <title>Simple Agent Console</title>
4     <meta name="viewport" content="width=device-width, initial-scale=1">
5     <meta content="utf-8" http-equiv="encoding">
6
7     <script src="https://code.jquery.com/jquery-3.1.0.min.js"></script>
8     <script type="text/javascript" src="amazon-connect-v1.2.0-23-g430d835.js"></script>
9   </head>

```

In the `<body>` section:

- Add CSS style for `ccpContainer` (this is container that holds CCP UI)
- Add CSS style for `section-main`
- Add `h1` title to be displayed on the page

```

11 | <body>
12 |   <style>
13 |     #ccpContainer {
14 |       width:265px;
15 |       height:465px;
16 |       overflow:hidden;
17 |       float:left;
18 |       margin-left:22px;
19 |       padding:25px;
20 |     }
21 |     #section-main {
22 |       padding:10px;
23 |       width:960px;
24 |       height:495px;
25 |       margin-left:0px;
26 |       float:left;
27 |       overflow: hidden;
28 |     }
29 |   </style>
30 |
31 |   <h1>Welcome to Amazon Connect Agent Console</h1>

```

- Add *ccpContainer* section which will host the CCP UI
- Add *section-main* section which will be displaying the call-related data (User - defined Attributes set in Contact Flows)

```

33 |   <section id="ccpContainer">
34 |   </section>
35 |
36 |   <section id="section-main">
37 |     <header><h1>Amazon Connect CCP Integration Details</h1></header>
38 |     <fieldset>
39 |       <legend>Contact Details</legend>
40 |       <div id="divDetails"></div>
41 |     </fieldset>
42 |   </section>
43 |
44 | </body>
45 | </html>
46 |

```

- Inside the *ccpContainer* section, initialize the CCP, by providing the URL for your Amazon Connect instance

```

33 |   <section id="ccpContainer">
34 |     <script>
35 |       //-----init CCP-----
36 |       var container = document.getElementById("ccpContainer");
37 |
38 |       connect.core.initCCP(
39 |         container, {
40 |           ccpUrl: 'https://yourinstance.awsapps.com/connect/ccp#/',
41 |           softphone: {
42 |             allowFramedSoftphone: true,
43 |             disableRingtone: false
44 |           }
45 |         }
46 |       )
47 |       //-----End init CCP-----
48 |     </script>
49 |   </section>

```

- Once CCP is initialized, we have to subscribe *onConnecting* event to be able to receive call-related data for each incoming call. Once the contact is received, we are writing the *contactId* to the browser's console log

```

47 | | | | | //-----End init CCP-----
48 | | | | |
49 | | | | | //-----CCP Events-----
50 | | | | | connect.contact(function(contact){
51 | | | | | //-----CCP OnConnecting-----
52 | | | | | contact.onConnecting(function(contact){
53 | | | | | console.log("CDEBUG >> " + "onConnecting() >> contactId = " + contact.contactId);
54 | | | | | });
55 | | | | | });
56 | | | | |
57 | | | | |
58 | | | | | </script>

```

- Next step is to extract the incoming phone number

```

49 | | | | | //-----CCP Events-----
50 | | | | | connect.contact(function(contact){
51 | | | | | //-----CCP OnConnecting-----
52 | | | | | contact.onConnecting(function(contact){
53 | | | | | console.log("CDEBUG >> " + "onConnecting() >> contactId = " + contact.contactId);
54 | | | | |
55 | | | | | var phoneNumber = contact.getConnections()[1].getAddress()['phoneNumber'];
56 | | | | |
57 | | | | | });
58 | | | | | });
59 | | | | |
60 | | | | | </script>

```

- Extracting the User-Defined Attribute that was set in the Contact Flow (EmployeeName)

```

51 | | | | | //-----CCP OnConnecting-----
52 | | | | | contact.onConnecting(function(contact){
53 | | | | | console.log("CDEBUG >> " + "onConnecting() >> contactId = " + contact.contactId);
54 | | | | |
55 | | | | | var phoneNumber = contact.getConnections()[1].getAddress()['phoneNumber'];
56 | | | | |
57 | | | | | var attributes = contact.getAttributes();
58 | | | | | var EmployeeName = attributes["EmployeeName"].value;

```

- As a final step, we want to display received data on the page, by simply appending it to *divDetails*

```

51 | | | | | //-----CCP OnConnecting-----
52 | | | | | contact.onConnecting(function(contact){
53 | | | | | console.log("CDEBUG >> " + "onConnecting() >> contactId = " + contact.contactId);
54 | | | | |
55 | | | | | var phoneNumber = contact.getConnections()[1].getAddress()['phoneNumber'];
56 | | | | |
57 | | | | | var attributes = contact.getAttributes();
58 | | | | | var EmployeeName = attributes["EmployeeName"].value;
59 | | | | |
60 | | | | | //add details to DIV
61 | | | | | $("#divDetails").append("<hr/> ALERTING CALL " + contact.contactId +
62 | | | | | " from " + EmployeeName + "<"+ phoneNumber + "> at " + new Date().toLocaleString() + "<br/><br/>");
63 | | | | | });
64 | | | | | });

```

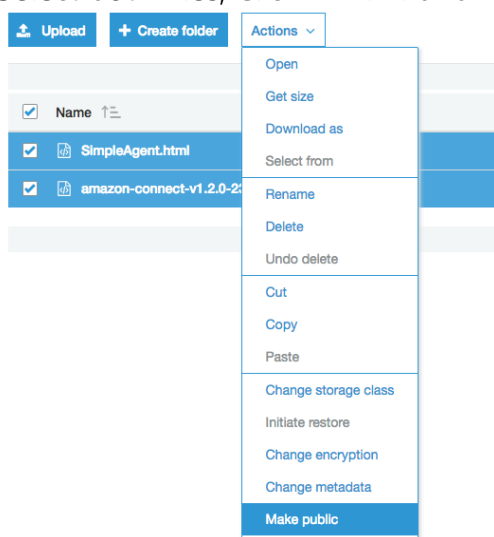
- Open your AWS Console and select S3 from the service list
- Click **Create Bucket**

- Enter a bucket name (i.e. amazonconnectbootcampccp)

- Click **Create**
- Click on your newly created bucket and then click **Upload**
- Add files > select your *SimpleAgent.html* and *amazon-connect- $\${version}.js$*
- Click **Upload**

☐	Name ↑	Last modified ↑	Size ↑	Storage class ↑
☐	SimpleAgent.html	Sep 12, 2018 6:49:24 PM GMT+0100	2.8 KB	Standard
☐	amazon-connect-v1.2.0-23-g430d835.js	Sep 12, 2018 6:49:26 PM GMT+0100	645.9 KB	Standard

- Select both files, Click **Actions** and **Make Public**



- At this step, we are creating Amazon CloudFront distribution.

Amazon CloudFront is a web service that speeds up distribution of your static and dynamic web content, such as .html, .css, .js, and image files, to your users. CloudFront delivers your content through a worldwide network of data centers called edge locations. When a user requests content that you're serving with CloudFront, the user is routed to the edge location that provides the lowest latency (time delay), so that content is delivered with the best possible performance.

- Open your AWS Console and select CloudFront
- Click **Create Distribution**

Amazon CloudFront Getting Started ?

Either your search returned no results, or you do not have any distributions. Click the button below to create a new CloudFront distribution. A distribution allows you to distribute content using a worldwide network of edge locations that provide low latency and high data transfer speeds ([learn more](#))

[Create Distribution](#)

- Click **Get Started for Web**

Select a delivery method for your content.

Web

Create a web distribution if you want to:

- Speed up distribution of static and dynamic content, for example, .html, .css, .php, and graphics files.
- Distribute media files using HTTP or HTTPS.
- Add, update, or delete objects, and submit data from web forms.
- Use live streaming to stream an event in real time.

You store your files in an origin - either an Amazon S3 bucket or a web server. After you create the distribution, you can add more origins to the distribution.

[Get Started](#)

- Set **Origin Domain Name** by selecting your S3 bucket (hosting custom web page)

Create Distribution

Origin Settings

Origin Domain Name	amazonconnectbootcampccp.s3.amazo	i						
Origin Path		i						
Origin ID	S3-amazonconnectbootcampccp	i						
Restrict Bucket Access	☐ Yes ☒ No	i						
Origin Custom Headers	<table border="0"> <thead> <tr> <th>Header Name</th> <th>Value</th> <th>i</th> </tr> </thead> <tbody> <tr> <td> </td> <td> </td> <td>+</td> </tr> </tbody> </table>	Header Name	Value	i			+	
Header Name	Value	i						
		+						

- For the training purpose, we are going to decrease the default amount of time for objects to stay in caches – set Default TTL to 30 seconds

Object Caching ☐ Use Origin Cache Headers
☒ Customize
[Learn More](#)

Minimum TTL

Maximum TTL

Default TTL

- Click **Create Distribution**
- Distribution will take few moments to complete – Status: In Progress

CloudFront Distributions

Create Distribution

Distribution Settings

Delete

Enable

Disable

Viewing:

Any Delivery Method

Any State

<<

>

Viewing

	Delivery Method	ID	Domain Name	Comment	Origin	CNAMEs	Status	State
☐	Web	E1V0N7FMVAUFL	d18q4mcf6nojru.cloudfront.net	-	amazonconnectb	-	In Progress	Enabled

- In the meantime, we are going to whitelist Distribution Domain Name in the Amazon Connect Instance.
 - o For our customer's security, we require that all domains which embed the CCP for a particular instance are explicitly whitelisted.
 - o Each domain entry identifies the protocol scheme, host, and port.
 - o Any pages hosted behind the same protocol scheme, host, and port will be allowed to embed the CCP components which are required to use the Streams library.
 - o Whitelisted domains must be HTTPS
 - o All open tabs that contain an initialized Streams library or any other CCP tabs opened will be synchronized. This means that state changes made in one open window will be communicated to all open windows.
- Please copy Domain Name
 (i.e. d18q4mcf6nojru.cloudfront.net)

Delivery Method	ID	Domain Name
☒ Web	E1V0N7FMVAUFL	d18q4mcf6nojru.cloudfront.net

- In AWS Console, navigate to Amazon Connect and select your instance

- Choose "Approved Origins" link on the left side

Amazon Connect > amazon-connect-yomithird

Overview
Telephony
Data storage
Data streaming
Analytics tools
Tasks
Customer profiles
Approved origins
Contact flows

i We are upgrading the Amazon Connect console
To maintain your current level of access, make sure that you have required permissions. [Learn more](#)

Application integration

Amazon Connect can integrate with other products including Customer Relationship Management (CRM) and Workforce Management (WFM) products. Click on the link for details on how to set up integrations with Amazon Connect. [Learn more](#)

Approved origins

Once you have integrated with a CRM product, add the origins (scheme + host + port) that Amazon Connect will need to have access to.

[+ Add origin](#)

- Click "+ Add origin" and enter the Distribution Domain Name (including https://)

Add origin ✕

Enter origin URL

https://d18q4mcf6nojru.cloudfront.net

Cancel
Add

- Once completed, you should be able to see the CloudFront domain under Approved origins

Amazon Connect > amazonconnectbootcamp1

Overview
Telephony
Data storage
Data streaming
Application integration
Contact flows

Application integration

Amazon Connect can integrate with other products including Customer Relationship Management (CRM) and Workforce Management (WFM) products. Click on the link for details on how to set up integrations with Amazon Connect. [Learn more](#)

Approved origins

Once you integrated with a CRM product, add the origins (scheme + host + port) that Amazon Connect will need to have access to.

https://d18q4mcf6nojru.cloudfront.net [remove](#)

[+ Add origin](#)

- Check for CloudFront Distribution Status: Deployed

Domain Name	Comment	Origin	CNAMEs	Status
d18q4mcf6nojru.cloudfront.net	-	amazonconnectb	-	Deployed

- Open your web browser and navigate to <https://CloudFrontDomainName/SimpleAgent.html>
(i.e. <https://d18q4mcf6nojru.cloudfront.net/SimpleAgent.html>)

- Our custom web page should be displayed, with containers for CCP and Details
Welcome to Amazon Connect Agent Console

Amazon Connect CCP Integration Details

Contact Details

- The CCP Login screen should pop-up in a new tab (check the pop-up blocker)



Amazon Connect

Please log in with your
amazonconnectbootcamp1 credentials

Username

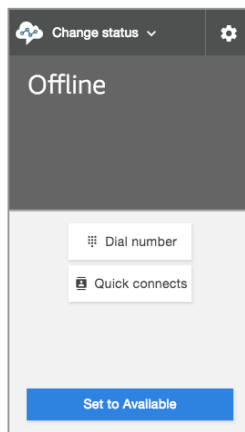
Password

Sign In

Forgot Password?

- Enter your CCP credentials and **Sign In**
- You may close the pop-up tab or leave it opened and go back to original tab with the custom web page. The CCP should be loaded in the left container

Welcome to Amazon Connect Agent Console

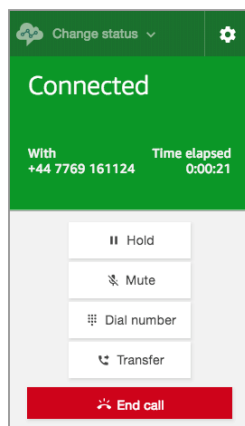


Amazon Connect CCP Integration Details

Contact Details

- Set your agent to Available and make an inbound call, then request to speak to an agent. Answer the call and check the Contact Details container. It should display ContactId, Caller's Name and Phone Number.

Welcome to Amazon Connect Agent Console



Amazon Connect CCP Integration Details

Contact Details

ALERTING CALL bb3851c9-e618-4d34-a456-373dfc63fb03 from John <+447769161124> at 13/09/2018, 11:39:51

Hands-on Module 7 – Trying it all Together

Objective: Independently create an Amazon Connect instance and implement a dynamic Contact Flow with database dip and call recording

In this module, we are challenging all participants to show the gained knowledge and implement the most common use-case scenario, by leveraging Amazon Connect, Lambda and DynamoDB services.

The challenge has few requirements and each successfully implemented requirement gives a certain number of points. Fulfill all the requirements and get the highest score!

1. Launch new Amazon Connect instance, claim a phone number and make your first call through built-in, sample Contact Flow – 10 points
2. Create your own Contact Flow, with a static and a dynamic prompt – 10 points
3. Create a new Queue and an Agent, assign the agent to your new queue – 10 points
4. Enable Contact Flow logs and Call Recording – 5 points
5. Check the Hours of operation and deliver the call to your agent – 10 points
6. Query the database by phone number – 30 points
7. Greet customer by his name – 10 points

You are allowed to use the Bootcamp Guide and online documentation. Have fun and good luck!